

Patterns and Frameworks - Leitfaden für die Projektarbeit

<https://vfhpf.eduloop.de>

Stand 2026-09-21 14:41:40

Inhalt

Patterns and Frameworks - Leitfaden für die Projektarbeit	3
1 Einführung und Organisatorisches	4
2 Ein initiales SpringBoot-Projekt aufsetzen	10
2.1 SpringBoot-Projekt mit Spring Initializr erzeugen	10
2.2 SpringBoot-Projekt in Eclipse importieren	12
2.3 Anwendung strukturieren	17
3 User verwalten mit REST, Spring Security und JPA	21
3.1 User-Klasse erstellen und mit JPA verknüpfen	21
3.2 REST-API für die User-Verwaltung definieren	25
3.3 Mit Spring Security den Zugang zur Web-Anwendung schützen	30
4 WebSockets zur bidirektionalen Client-Server-Kommunikation	38
4.1 Serverseitig WebSockets bereitstellen	38
4.2 Clientseitig mit WebSockets arbeiten	43
5 JWT verwenden	49
5.1 Serverseitig JWT implementieren	49
5.2 Clientseitig JWT implementieren	55
5.3 Fortgeschrittenes JWT-Handling	61
6 Eine Single-Page-Application entwickeln	66
6.1 Serverseitige SPA-Anpassungen vornehmen	66
6.2 Clientseitig die SPA umsetzen	69
6.3 Clientseitige JWT-Erweiterungen	78
7 Projektspezifische Aspekte	84
7.1 Schutz vor CSRF implementieren	84
7.2 HTTPS-Verbindung einrichten	87
7.3 Bilder zwischen Client und Server transferieren	91
7.4 Mit Countdowns arbeiten	102
7.5 Auf User-seitige Abbrüche reagieren	107
7.6 Typische Fallstricke im Projektverlauf	115
8 Mit Threads arbeiten	117
8.1 Motivation und Grundlagen von Threads	117
8.2 Zeitgesteuerte Ausführung von Threads	117
8.3 Thread Pools	118
8.4 Synchronisierung von Threads	118
8.5 Deadlocks und Starvation	118

Anhang

I Abbildungsverzeichnis	120
-------------------------------	-----

Patterns and Frameworks - Leitfaden für die Projektarbeit

Patterns and Frameworks - Leitfaden für die Projektarbeit

- 1 Einführung und Organisatorisches
- 2 Ein initiales SpringBoot-Projekt aufsetzen
- 3 User verwalten mit REST, Spring Security und JPA
- 4 WebSockets zur bidirektionalen Client-Server-Kommunikation
- 5 JWT verwenden
- 6 Eine Single-Page-Application entwickeln
- 7 Projektspezifische Aspekte
- 8 Mit Threads arbeiten

Dieser Leitfaden soll Ihnen helfen, Ihre Projektarbeit im Modul "Patterns and Frameworks" erfolgreich zu absolvieren. Anhand eines minimalistischen Beispielprojekts werden alle wesentlichen Schritte von der Initialisierung bis zur fertigen Umsetzung einer Client-Server-Anwendung skizziert. Alle projektrelevanten Technologien kommen in dem Beispielprojekt zum Einsatz. Für bekannte Probleme und Fallstricke, die im Projektverlauf typischerweise auftreten, werden Lösungsansätze vorgestellt.

Verbesserungsvorschläge und Korrekturwünsche zu diesem Leitfaden können Sie gerne über den Link "Fehler zu dieser Seite melden" (links unten auf jeder Seite) einspeisen. Vielen Dank für Ihre Mithilfe!

Und nun viel Erfolg und hoffentlich auch Spaß bei der Bearbeitung Ihres Projekts!

1 Einführung und Organisatorisches

Um nachvollziehen zu können, welche Schwerpunkte wir bei der Entwicklung des Beispielprojekts setzen werden, müssen wir zunächst einen Blick auf die Anforderungen werfen, die im Modul "Patterns and Frameworks" an die Projektarbeit gestellt werden.



Die Ausführungen in diesem Abschnitt beziehen sich auf die Lehrveranstaltung, wie sie an der TH Lübeck durchgeführt wird. Diese deckt sich weitestgehend mit dem didaktischen Konzept des ursprünglichen Modulautors.

Andere VFH-Standorte mögen abweichende Rahmenbedingungen definieren. Selbstverständlich gilt für Ihr Projekt das, was die Lehrperson an Ihrem VFH-Standort mit Ihnen besprochen hat.



Erfahrungsgemäß ist die Lernkurve in "Patterns and Frameworks" relativ steil. Lassen Sie sich bitte hiervon nicht abschrecken. Fangen Sie frühzeitig an und arbeiten Sie, sofern möglich, lieber mehrere Tage am Stück am Projekt anstatt immer wieder nur eine Stunde am Abend.



Bedenken Sie bei der Aufgabenverteilung im Team, dass immer das Risiko besteht, dass ein Teammitglied während des Semesters aussteigt. Wer bereits jetzt abschätzen kann, dass er/sie nicht ausreichend Zeit für das Projekt haben wird, kommuniziere das bitte frühzeitig, damit das Team sich noch neu organisieren kann.

Anforderungen an das Softwareprojekt

In dem Projekt soll ein Softwaresystem – konkret: ein einfaches Multiplayer-Spiel – erstellt werden, das in Client- und Server-Komponente unterteilt ist.

Ziel des Projekts ist es also, eine komponentenbasierte, verteilte Softwarearchitektur zu entwerfen und programmiertechnisch zu implementieren. Dabei sollen gezielt Entwurfs- und Architekturmuster sowie Methoden der verteilten Programmierung zum Einsatz kommen, insb. zur Client-Server-Kommunikation. Mindestens serverseitig soll mit einem Framework gearbeitet werden.

Damit führt das Projekt Kompetenzen zusammen (und baut diese aus), die Sie in vorangegangenen Modulen Ihres Medieninformatik-Studiums erworben haben, insb.

- Grundlagen der Programmierung 1 und 2
- Datenbanken
- Web-Programmierung

- Softwaretechnik
- Internetanwendungen für mobile Geräte
- Internetserver-Programmierung



Anmerkung

Falls Sie IT-Sicherheit Online studieren und "Patterns and Frameworks" als Wahl(pflicht)modul belegen, haben Sie vermutlich die Module *Web-Programmierung*, *Internetanwendungen für mobile Geräte* und *Internetserver-Programmierung* nicht absolviert. Das ist aber unkritisch, wenn Sie alternativ an dem Modul *Internet-Technologie* aus Ihrem Studiengang teilgenommen haben.

Der Schwerpunkt in Ihrem Projekt wird auf den Phasen Entwurf und Implementierung liegen. Es ist sowohl ein Versionsverwaltungssystem (Git) als auch ein Build Management Tool (Maven) zu verwenden. In der Wahl der IDE sind Sie frei (Eclipse, IntelliJ, ...). Gaming-Engines wie Unity sind in diesem Modul nicht zugelassen.

Das Projekt soll im Team (2-3 Studierende) bearbeitet werden.

Funktionale Anforderungen

Es soll ein einfaches Multiplayer-Spiel für (mindestens) 2 Spieler realisiert werden. Als Projektidee wird die Implementierung eines bekannten Arcade-Klassikers vorgeschlagen, z. B. Tetris, Tank, Snake, Pac-Man, Dig Dug oder Blobby Volley. Es kann auch ein Brettspiel oder ein rundenbasiertes Quizspiel umgesetzt werden. Weitere Spieltypen sind nach Absprache möglich.

Die Spielregeln können gerne individuell angepasst werden. Bezüglich der angestrebten Funktionalität der Anwendung wird dementsprechend Freiraum für kreative Ideen gelassen. Ein Single-Player-Modus ist nicht erforderlich.

Bevor Sie loslegen, stimmen Sie bitte Ihre Projektidee mit der Lehrkraft ab.

Unabhängig von der gewählten Projektidee sollen folgende Anforderungen berücksichtigt werden:

- Ein/e Spieler/in soll sich registrieren, einloggen (mit Username + Passwort) und ausloggen können. Die Registrierung kann einfach gehalten werden (z. B. keine E-Mail-Bestätigung, sondern einfach nur prüfen, ob Username bereits existiert).
- Ein/e eingeloggte/r Spieler/in muss auf definierte Weise (z. B. über eine Lobby) eine/n Mitspieler/in finden können, bevor das eigentliche Spiel beginnen kann.
- Für jede/n Spieler/in wird eine Historie der gespielten Spiele gespeichert und mittels einfacher Auswertungen dargestellt (gewonnene und verlorene Spiele, Highscores o. ä.).

- Es sollen an mindestens einer sinnvollen Stelle auch Bilder über die API an den Server gesendet bzw. vom Client angefragt und dargestellt werden (z. B. die Profilbilder der Spieler/innen).

Technische Anforderungen an die Server-Komponente

Als serverseitige Programmiersprache ist ausschließlich Java zugelassen - in diesem Leitfaden wird das JDK in der neuesten LTS-Version 25 verwendet. Ebenfalls obligatorisch ist die Verwendung von Spring Boot als (Web-)Framework für die Server-Seite Ihrer Anwendung. Bitte verwenden Sie Spring Boot in einer aktuellen, stabilen Version (siehe SpringBoot-Projekt mit Spring Initializr erzeugen).

Die registrierten Spieler/innen und ihre Spielhistorie sollen zentral in einer relationalen Datenbank gespeichert werden. Sie können hierzu einen DB-Server der Hochschule verwenden, sofern angeboten. Es soll ein Framework für das objekt-relationale Mapping eingesetzt werden. Die API soll eine Authentifizierung von Spieler/innen über Spring Security unterstützen, entweder sessionbasiert oder per JSON Web Token (JWT). Kommunikation zum Registrieren, Einloggen, und Abfragen der Spielhistorie soll über ein serverseitiges REST-API erfolgen, Kommunikation zum Finden von Mitspieler/innen und für Aktionen während des laufenden Spiels über ein Websocket-STOMP-API.

Technische Anforderungen an die Client-Komponente

Der Client soll ein grafisches UI umfassen und die obigen funktionalen Anforderungen abbilden. Bei drei Studierenden im Team sind zwei Clients zu erstellen, die beide auf gemeinsame serverseitige APIs zugreifen. Bei Teams aus zwei Studierenden ist ein Client ausreichend.

Programmiersprachen und Frameworks/Bibliotheken können für beide Clients beliebig gewählt werden, z. B.:

- Java / JavaFX (z. B. JDK 25 mit JavaFX SDK 25)
- TypeScript / Angular
- JavaScript / Vue.js
- JavaScript / Thymeleaf

Bei Verwendung eines Frontend-Framework wie Vue.js oder Angular soll der Client als Single Page Application (SPA) realisiert werden. Bei Verwendung einer serverseitigen Template Engine wie Thymeleaf wird sich eine klassische Multi Page Application ergeben.



WICHTIG für Teams aus drei Personen: Die beiden Clients müssen *mit unterschiedlichen Sprachen / Frameworks* realisiert werden. Zudem müssen sie *komplett unabhängig voneinander entwickelt* werden. Es ist z. B. nicht erlaubt, erst einen Client fertigzustellen und den zweiten durch Migration auf eine andere Sprache / ein anderes Framework aus dem ersten abzuleiten.

Prüfungsleistung: Projektarbeit plus Präsentation

Im Zentrum der Prüfungsleistung für das Modul „Patterns and Frameworks“ steht die fristgerechte Abgabe eines im Team entwickelten Softwareprojekts gemäß der oben beschriebenen Anforderungen. Zum Nachweis ihrer Eigenleistung präsentieren Teammitglieder ihr Projekt inkl. anschließender Diskussion mit der Lehrperson.

Jedes Teammitglied kann seine Präsentation auf die eigenen Schwerpunkte ausrichten. Falls also z. B. jemand vorwiegend mit der Entwicklung eines Clients befasst war, ist es okay, diesen in den Mittelpunkt der Präsentation zu stellen. Es wird allerdings erwartet, dass jedes Teammitglied einen Überblick über die Gesamtarchitektur des Spiels sowie über die Schnittstellen zwischen Client(s) und Server hat. Zu Beginn einer Präsentation sollte ein Überblick über das Gesamtprojekt gegeben werden.

Benotungskriterien (Abweichungen je nach VFH-Standort möglich)

Folgende Kriterien werden schließlich zur Benotung der Projektarbeit herangezogen:

- Grad der Erfüllung der vorgegebenen Anforderungen an das Softwareprojekt
- Code-Qualität
 - Lesbarkeit des Codes
 - Einhaltung von Coding Conventions
 - Sinnvoller Einsatz von OOP
 - Fehlerbehandlung / Robustheit gegen Fehleingaben
 - Kohäsion / Single-Responsibility-Prinzip
 - maßvolle interne Dokumentation
- Bewusster und sinnvoller Einsatz von Entwurfsmustern, Architekturmustern und Frameworks
- Verwendung geeigneter Protokolle zur Client-Server-Kommunikation
- Parallelisierung von Abläufen, soweit im Spiel sinnvoll, z. B. um das UI nicht zu blockieren
- Inhalt der Präsentation: Ergebnis vorstellen, Architektur vorstellen, beispielhaft im Quelltext durch die Architektur gehen (z. B. Login-Prozess), Lessons Learned

- Qualität der Präsentation: Verständlichkeit, Struktur, inhaltliche Tiefe, Einhaltung der zeitlichen Vorgaben, Beantwortung von Fragen

Bei nachweislich stark abweichenden Leistungen innerhalb eines Teams können individuelle Noten vergeben werden, d. h. nicht alle Teammitglieder erhalten zwingend dieselbe Note.

Prüfungsvorleistung (Abweichungen je nach VFH-Standort möglich)

Die Prüfungsvorleistung umfasst drei zeitlich vorgegebene Meilensteine, die jeweils Zwischenstände des Projekts repräsentieren.

1. ein UML-Klassendiagramm für die Klassen des fachlichen Datenmodells (also das, was Sie per ORM in die Relationale Datenbank abbilden) sowie ein UML-Komponentendiagramm zur Darstellung der Zielarchitektur. Bitte achten Sie darauf, dass Ihre Diagramme selbsterklärend sind (klare Struktur, sprechende Bezeichner usw.).
2. Dokumentation der vereinbarten Schnittstellen zwischen Client(s) und Server, also insb. der REST- und Websocket/STOMP-Endpunkte
3. Vorversionen von Client(s) und Server als Prototypen, d. h. ausgewählte Anwendungsfälle sind bereits realisiert, wenn auch ggf. noch nicht in eine gemeinsame Anwendung integriert.

Die Prüfungsvorleistungen sind unbenotet (nur „bestanden“ oder „nicht bestanden“). Die Lehrperson gibt den Teams Feedback, das diese im weiteren Projektverlauf berücksichtigen können.

Ausblick auf den Leitfaden

In diesem Leitfaden beleuchten wir verschiedene Aspekte einer (bewusst einfach gehaltenen) Web-Anwendung, insbesondere solche, die auch in Ihrem Projekt eine Rolle spielen werden. Die gezeigten Lösungsansätze können Sie als Orientierungshilfe verwenden und an Ihre konkrete Projektsituation anpassen.

Zunächst werden wir eine einfache, datenbankbasierte User-Verwaltung implementieren. Anschließend werden wir unsere Anwendung über Spring Security absichern. Dies machen wir beispielhaft in verschiedenen Varianten, nämlich zum einen über serverseitige Sessions und zum anderen über JWTs. Die Kommunikation zwischen Clients und dem Server realisieren wir dabei über ein REST-API.

Anschließend werden wir die bidirektionale, asynchrone Kommunikation mittels Web-Sockets kennenlernen. Diese Technologie wird Sie in die Lage versetzen, ein Matching zwischen Spieler/innen zu implementieren, Spielzüge zu realisieren sowie alle Clients über geänderte Spielzustände zu informieren.

Während wir im ersten Teil des Leitfadens ein (minimalistisches) Multi-Page-Frontend mit Hilfe der serverseitigen Templating Engine Thymeleaf umsetzen, werden wir später alternativ dazu eine Single Page Application (SPA) entwickeln. Diese werden wir mit dem Web-Framework Vue.js realisieren. Ebenso wie der Thymeleaf-Client wird auch der Vue-Client mit unserem SpringBoot-Server kommunizieren.

Nebenläufige Programmierung mit Threads sowie das Absichern unserer Anwendung gegen Cyberangriffe behandeln wir gegen Ende des Leitfadens. Hinzu kommen Lösungsansätze für verschiedene spezielle Aspekte Ihres Projekts, z. B. der Übertragung von Bildern zwischen Client und Server und der Umgang mit User-seitigen Abbrüchen (z. B. Browser-Tab schließen).

Insgesamt soll Ihnen der Leitfaden den Umgang mit den z. T. sehr komplexen Frameworks erleichtern und die potenziell frustrierende Suche nach Lösungen für typische Probleme im Projektverlauf ersparen.



2 Ein initiales SpringBoot-Projekt aufsetzen

2 Ein initiales SpringBoot-Projekt aufsetzen

2.1 SpringBoot-Projekt mit Spring Initializr erzeugen

2.2 SpringBoot-Projekt in Eclipse importieren

2.3 Anwendung strukturieren

In diesem Abschnitt erzeugen wir uns eine initiale Projektinfrastruktur, auf deren Basis wir dann nach und nach unser Projekt weiter ausbauen werden.



Es wird empfohlen, dass Sie Ihr Projekt unter dezentrale Versionskontrolle, vorzugsweise Git , stellen. Dadurch wird zum einen das Risiko eines Datenverlusts minimiert und zum anderen die Softwareentwicklung im Team erleichtert. Ob Sie darauf aufbauend GitHub , GitLab oder Bitbucket einsetzen, wird den Lehrpersonen an den einzelnen VFH-Standorten überlassen. Da Ihre Hochschule eventuell eigene Services zur Versionsverwaltung hostet, wird dieses Thema aus dem Leitfaden ausgeklammert.



Welche IDE Sie für Ihr Projekt verwenden, bleibt Ihrem Team entweder selbst überlassen oder wird von Ihrer Lehrperson vorgegeben. Dieser Leitfaden wurde am Beispiel von Eclipse in Verbindung mit Git und einem von der TH Lübeck gehosteten GitLab-Server erstellt. Die gezeigten Sachverhalte sollten sich aber in einfacher Weise auf andere Entwicklungsumgebungen übertragen lassen.

2.1 SpringBoot-Projekt mit Spring Initializr erzeugen

Zunächst werden wir ein initiales SpringBoot-Projekt erzeugen, welches wir dann unter Git-Kontrolle stellen und in unser GitLab-Repository pushen werden.

Auf der Website <https://start.spring.io/> spezifizieren wir unser Projekt: Es soll ein Maven-Projekt werden, als Programmiersprache wird Java ausgewählt (im Screenshot in der derzeit aktuellsten LTS-Version 25), dazu Spring Boot in der jeweils aktuellsten stabilen Version (im Screenshot 4.1.1). Als Metadaten hinterlegen wir insbesondere den Gruppennamen (bitte Benennungsvorgaben der Lehrperson beachten!) sowie den Spielnamen (Artifact). Hieraus ergibt sich der Package-Name unseres Java-Projekts. Als Packaging-Variante wählen wir "Jar", als Configuration "Properties".

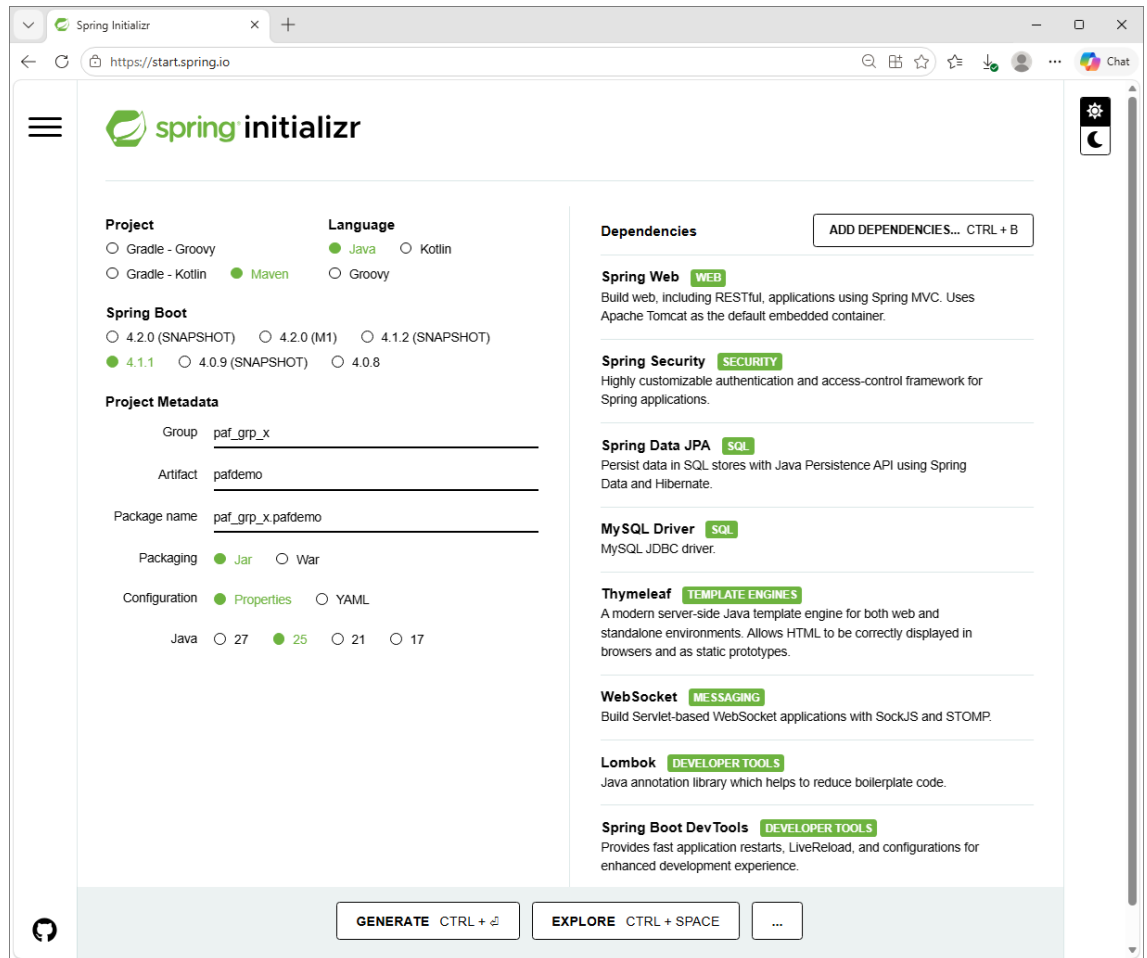


Abb. 2.1: Ein initiales SpringBoot-Projekt spezifizieren

Über den ADD...-Button rechts oben fügen wir nun eine Reihe von Dependencies hinzu, um unser Projekt mit extern bereitgestellten Funktionalitäten auszustatten, die wir später benötigen werden, hier:

- Spring Web
- Spring Security
- Spring Data JPA
- MySQL Driver
- Thymeleaf
- WebSocket
- Lombok
- Spring Boot DevTools



Falls in Ihrer Lehrveranstaltung ein anderes Datenbankmanagementsystem als MySQL verwendet wird, wählen Sie bitte den dazu passenden Treiber anstelle des o. g. MySQL-Treibers aus.



Um Lombok verwenden zu können, muss es zusätzlich in der verwendeten IDE installiert werden. Hinweise hierzu finden Sie auf der [Lombok-Website](#).

Über den Generate-Button laden wir nun das Projekt als zip-Datei herunter und entpacken es in einem Ordner unserer Wahl.

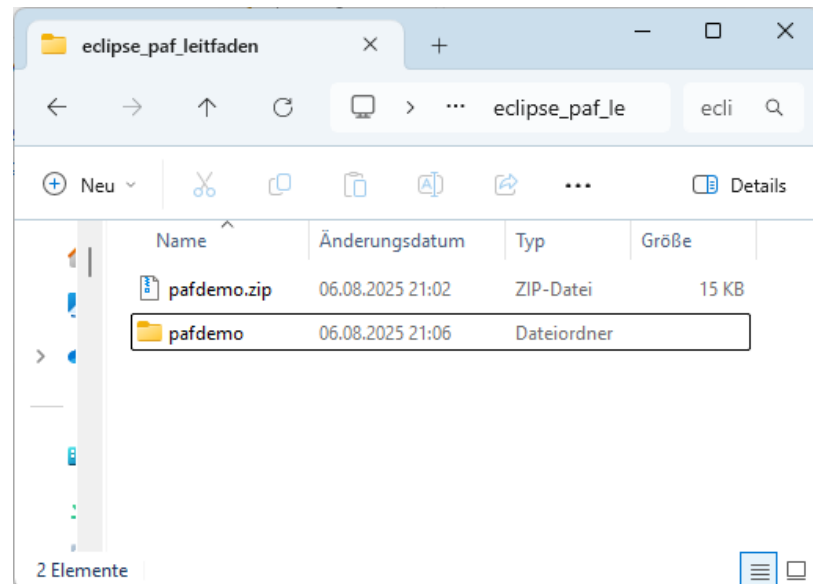


Abb. 2.2: Entpackte zip-Datei aus dem Spring Initializr

2.2 SpringBoot-Projekt in Eclipse importieren

Das im vorigen Schritt vom Spring Initializr erzeugte und von uns heruntergeladene und entpackte Projektverzeichnis werden wir nun in eine IDE, hier konkret Eclipse, importieren.

Hierzu klicken wir im File-Menü von Eclipse auf "Import..." und wählen dann im Abschnitt "Maven" den Import Wizard "Existing Maven Projects" aus. Im nächsten Dialog des Wizard wählen wir als Root Directory das Verzeichnis aus, in dem sich unser entpacktes Projekt befindet.

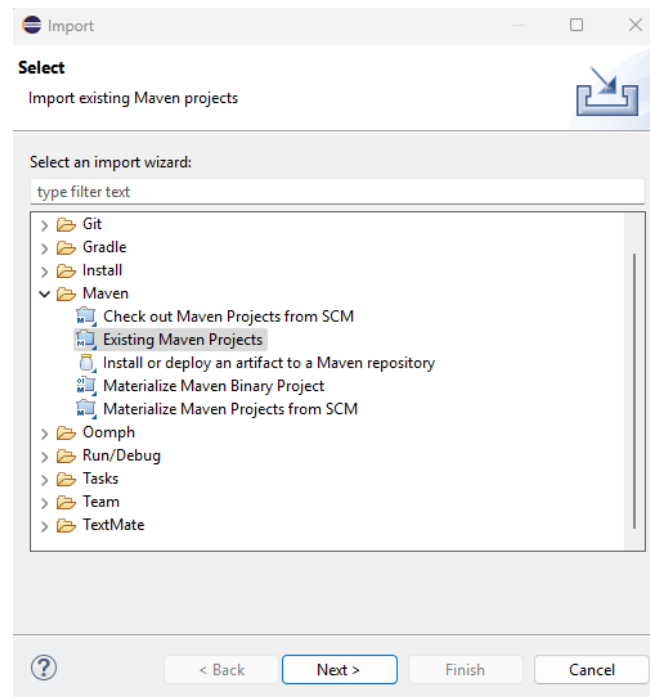


Abb. 2.3: Ein existierendes Maven-Projekt in Eclipse importieren, Teil 1

Eclipse sollte nun die enthaltene Datei pom.xml automatisch finden und markieren, so dass wir per Klick auf "Finish" den eigentlichen Import veranlassen können.

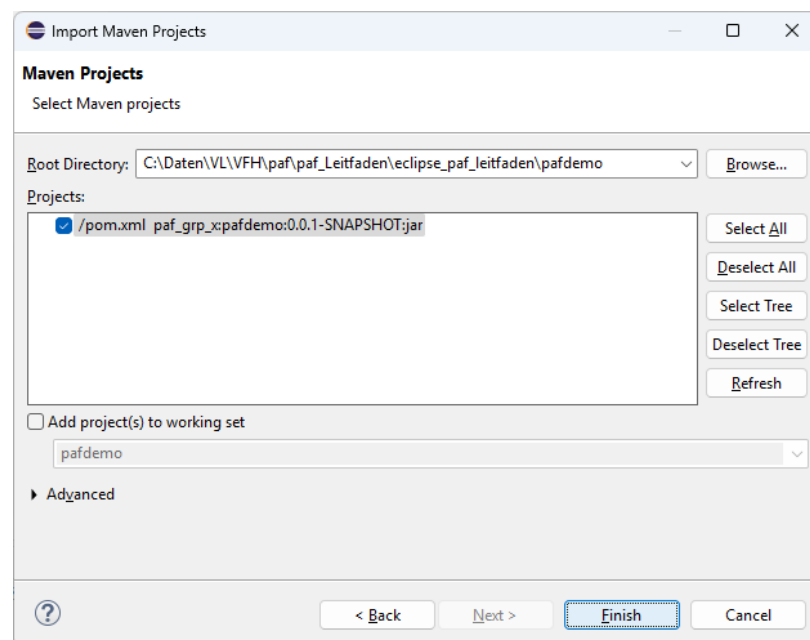


Abb. 2.4: Ein existierendes Maven-Projekt in Eclipse importieren, Teil 2

Der Package Explorer (typischerweise in der linken Seitenleiste) zeigt nun die initiale Projektstruktur an. Unter anderem gibt es in unserem Beispiel im Package paf_grp_x.pafdemo im Unterordner src/main/java eine Datei PafdemoApplication.java, die den Einstiegspunkt in unsere Anwendung darstellt.

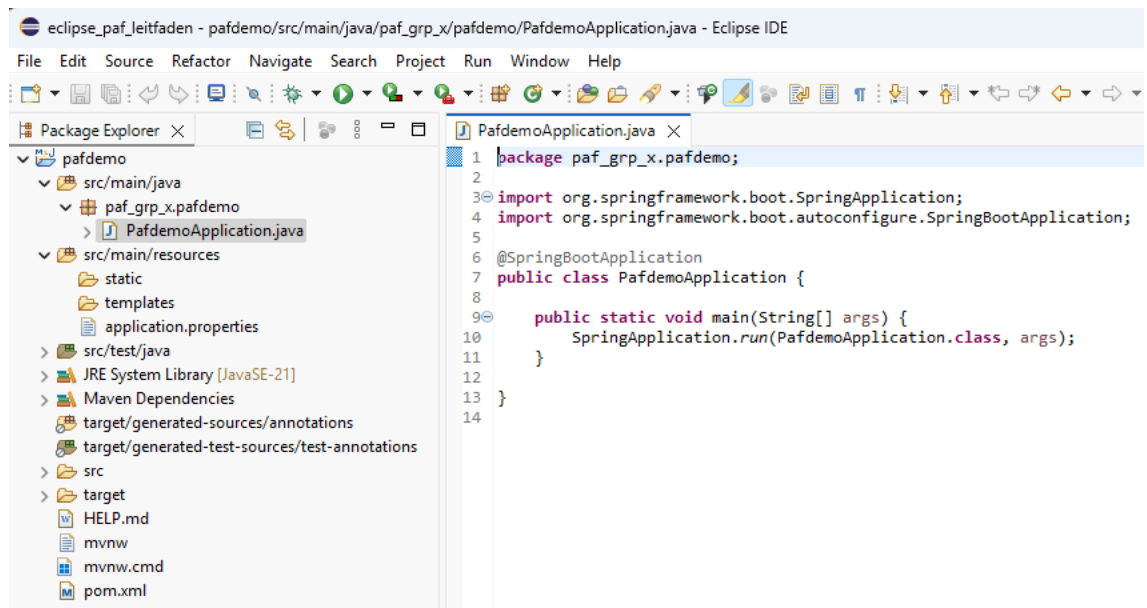


Abb. 2.5: Initiales Maven-Projekt nach dem Import

Um beim Starten der Web-Anwendung keine Fehlermeldung zu erhalten, dass die zu verwendende Datenbank noch nicht spezifiziert wurde, ziehen wir diesen Schritt vor, auch wenn wir aktuell noch keinen Datenbankzugriff brauchen:

Zentrale Konfigurationen einer SpringBoot-Anwendung werden in der Datei `application.properties` vorgenommen, die sich im Ordner `src/main/resources` befindet. Bisher ist hier nur der Anwendungsname (`pafdemo`) spezifiziert. Wir ergänzen nun die Datenbankkonfiguration wie in der folgenden Abbildung gezeigt. In der letzten Zeile deaktivieren wir außerdem die automatische Konfiguration von Spring Security, da wir diese erst später verwenden wollen.

```
*application.properties
1 spring.application.name=pafdemo
2
3 # create database tables automatically on application start
4 spring.jpa.hibernate.ddl-auto=create
5
6 spring.jpa.properties.hibernate.format_sql=true
7
8 spring.datasource.url=jdbc:mysql://localhost:3306/pafdemo
9
10 # never store credentials in configuration files when application is in productive use!
11 # here: dev mode only!
12 spring.datasource.username=pafdemo
13 spring.datasource.password=pafdemo
14
15 #disable Spring Security auto-configuration
16 spring.autoconfigure.exclude=org.springframework.boot.autoconfigure.security.servlet.SecurityAutoConfiguration
```

Abb. 2.6: Datenbankkonfiguration in `application.properties`

Wir gehen hier davon aus, dass...

- auf dem Entwicklungsrechner ein MySQL- oder MariaDB-Server läuft,
- es einen Datenbank-User `pafdemo` mit gleichnamigem Dummy-Passwort gibt,
- dieser User alle Rechte auf der Datenbank `pafdemo` hat.

Bei Bedarf können die Datenbank sowie der User neu angelegt und mit den entsprechenden Rechten ausgestattet werden, z. B. mit Hilfe des MySQL Command-Line Interface, phpMyAdmin oder der MySQL Workbench:

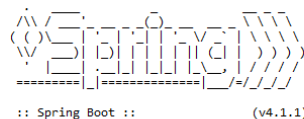


Code

```
CREATE DATABASE pafdemo;

CREATE USER pafdemo IDENTIFIED BY "pafdemo";
GRANT ALL ON pafdemo.* TO pafdemo;
```

Obwohl wir (bis auf die Datenbank-Konfiguration) noch keinen eigenen Quellcode beigesteuert haben, können wir die Anwendung nun bereits ausführen ("Run" auf Klasse PafdemoApplication). In der Eclipse-Konsole werden dann ungefähr die folgenden Zeilen ausgegeben:



:: Spring Boot :: (v4.1.1)

```
2025-08-08T17:59:51.739+02:00 INFO 34156 --- [pafdemo] [ restartedMain] paf_grp_x.pafdemo.PafdemoApplication : Starting PafdemoApplication using Java 25
2025-08-08T17:59:51.743+02:00 INFO 34156 --- [pafdemo] [ restartedMain] paf_grp_x.pafdemo.PafdemoApplication : No active profile set, falling back to 1 de
2025-08-08T17:59:51.797+02:00 INFO 34156 --- [pafdemo] [ restartedMain] .e.DevToolsPropertyDefaultsPostProcessor : Devtools property defaults active! Set 'spr
2025-08-08T17:59:51.797+02:00 INFO 34156 --- [pafdemo] [ restartedMain] .e.DevToolsPropertyDefaultsPostProcessor : For additional web related logging consider
2025-08-08T17:59:52.646+02:00 INFO 34156 --- [pafdemo] [ restartedMain] .s.d.r.c.RepositoryConfigurationDelegate : Bootstrapping Spring Data JPA repositories :
2025-08-08T17:59:52.664+02:00 INFO 34156 --- [pafdemo] [ restartedMain] .s.d.r.c.RepositoryConfigurationDelegate : Finished Spring Data repository scanning in
2025-08-08T17:59:53.258+02:00 INFO 34156 --- [pafdemo] [ restartedMain] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port 8080 (http)
2025-08-08T17:59:53.277+02:00 INFO 34156 --- [pafdemo] [ restartedMain] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2025-08-08T17:59:53.277+02:00 INFO 34156 --- [pafdemo] [ restartedMain] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/10.1
2025-08-08T17:59:53.336+02:00 INFO 34156 --- [pafdemo] [ restartedMain] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring embedded WebApplicationC
2025-08-08T17:59:53.336+02:00 INFO 34156 --- [pafdemo] [ restartedMain] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization c
2025-08-08T17:59:53.653+02:00 INFO 34156 --- [pafdemo] [ restartedMain] o.hibernate.jpa.internal.util.LogHelper : HHH000204: Processing PersistenceUnitInfo [r
2025-08-08T17:59:53.717+02:00 INFO 34156 --- [pafdemo] [ restartedMain] org.hibernate.Version : HHH000412: Hibernate ORM core version 6.6.2
2025-08-08T17:59:53.741+02:00 INFO 34156 --- [pafdemo] [ restartedMain] o.h.c.internal.RegionFactoryInitiator : HHH00026: Second-level cache disabled
2025-08-08T17:59:54.012+02:00 INFO 34156 --- [pafdemo] [ restartedMain] o.s.o.j.p.SpringPersistenceUnitInfo : No LoadTimeWeaver setup: ignoring JPA class
2025-08-08T17:59:54.043+02:00 INFO 34156 --- [pafdemo] [ restartedMain] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Starting...
2025-08-08T17:59:54.638+02:00 INFO 34156 --- [pafdemo] [ restartedMain] com.zaxxer.hikari.pool.HikariPool : HikariPool-1 - Added connection com.mysql.c
2025-08-08T17:59:54.640+02:00 INFO 34156 --- [pafdemo] [ restartedMain] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Start completed.
2025-08-08T17:59:54.719+02:00 WARN 34156 --- [pafdemo] [ restartedMain] org.hibernate.dialect.Dialect : HHH000511: The 5.5.5 version for [org.hiberr
2025-08-08T17:59:54.740+02:00 INFO 34156 --- [pafdemo] [ restartedMain] org.hibernate.orm.connections.pooling : HHH10001005: Database info:
Database JDBC URL [Connecting through datasource 'HikariDataSource (HikariPool-1)']
Database driver: undefined/unknown
Database version: 5.5.5
Autocommit mode: undefined/unknown
Isolation level: undefined/unknown
Minimum pool size: undefined/unknown
Maximum pool size: undefined/unknown
2025-08-08T17:59:54.996+02:00 INFO 34156 --- [pafdemo] [ restartedMain] o.h.e.t.j.p.i.JtaPlatformInitiator : HHH000489: No JTA platform available (set 'j
2025-08-08T17:59:55.000+02:00 INFO 34156 --- [pafdemo] [ restartedMain] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for pe
2025-08-08T17:59:55.051+02:00 WARN 34156 --- [pafdemo] [ restartedMain] JpaBaseConfiguration$JpaWebConfiguration : spring.jpa.open-in-view is enabled by default
2025-08-08T17:59:55.422+02:00 WARN 34156 --- [pafdemo] [ restartedMain] ion$DefaultTemplateResolverConfiguration : Cannot find template location: classpath:/t
2025-08-08T17:59:55.716+02:00 INFO 34156 --- [pafdemo] [ restartedMain] o.s.b.d.a.OptionalLiveReloadServer : LiveReload server is running on port 35729
2025-08-08T17:59:55.806+02:00 INFO 34156 --- [pafdemo] [ restartedMain] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with con
2025-08-08T17:59:55.830+02:00 INFO 34156 --- [pafdemo] [ restartedMain] paf_grp_x.pafdemo.PafdemoApplication : Started PafdemoApplication in 4.506 seconds
```

Abb. 2.7: Ausgaben in der Serverkonsole nach Start der Anwendung

In unserem Beispiel erkennen wir z. B., welche Java-Version verwendet wird, dass der TomcatWebServer auf Port 8080 gestartet wurde, dass der LiveReload-Server (bereitgestellt durch die Spring Boot DevTools) auf Port 35729 läuft usw. Offenbar läuft unsere Anwendung auf Basis einer Default-Konfiguration, in die wir bei Bedarf eingreifen können (wie z. B. mit den oben gezeigten Datenbank-Einstellungen). Spring Boot arbeitet, wie viele andere Frameworks auch, mit dem Grundsatz "Convention over Configuration", d. h. es werden diverse typische Grundannahmen getroffen, die schnell lauffähige Ergebnisse ermöglichen. Im Projektverlauf werden wir weitere Anpassungen an der Konfiguration vornehmen.

Die folgende Abbildung zeigt, wie sich unsere Anwendung derzeit im Browser präsentiert, wenn wir sie über die URL `http://localhost:8080/` aufrufen.

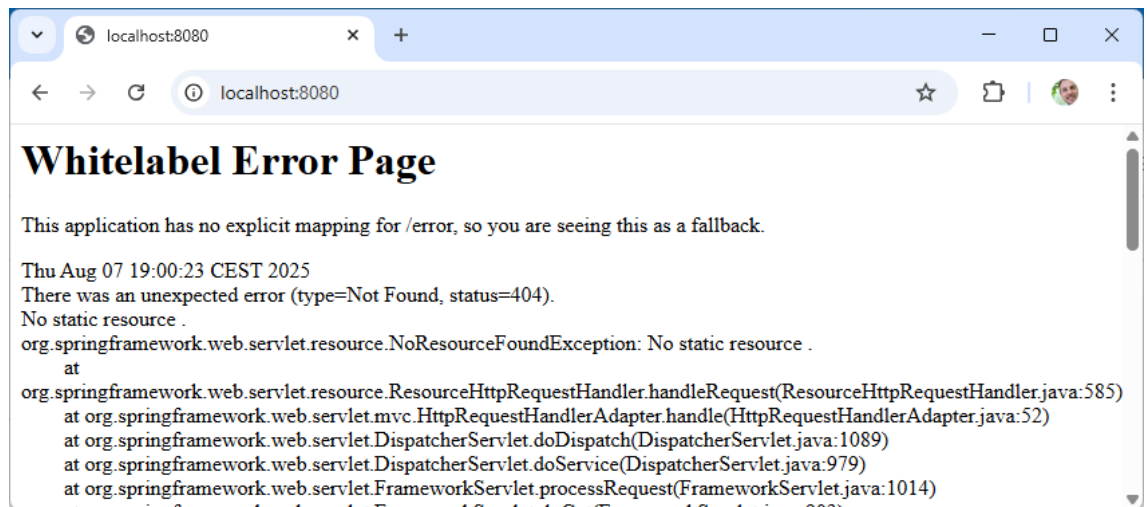


Abb. 2.8: Whitelabel Error Page

Die Fehlermeldung erscheint auf den ersten Blick unerfreulich, aber immerhin zeigt sie, dass der Server auf unseren (Get-)Request, den wir durch das Senden der URL ausgelöst haben, geantwortet hat. Da wir serverseitig noch keinen Endpunkt spezifiziert haben, der auf `/` reagiert, versucht der Browser, eine Fehlerseite unter `/error` aufzurufen. Da auch dieser Endpunkt nicht definiert wurde, erscheint die Whitelabel Error Page.

Wir können unsere Anwendung in einfacher Weise um eine Hello-World-Funktionalität ergänzen. Hierzu legen wir im Ordner `src/main/java` ein Package `paf_grp_x.pafdemo.controller` und darin eine Klasse `PafdemoController` mit dem in der folgenden Abbildung gezeigten Quellcode an.

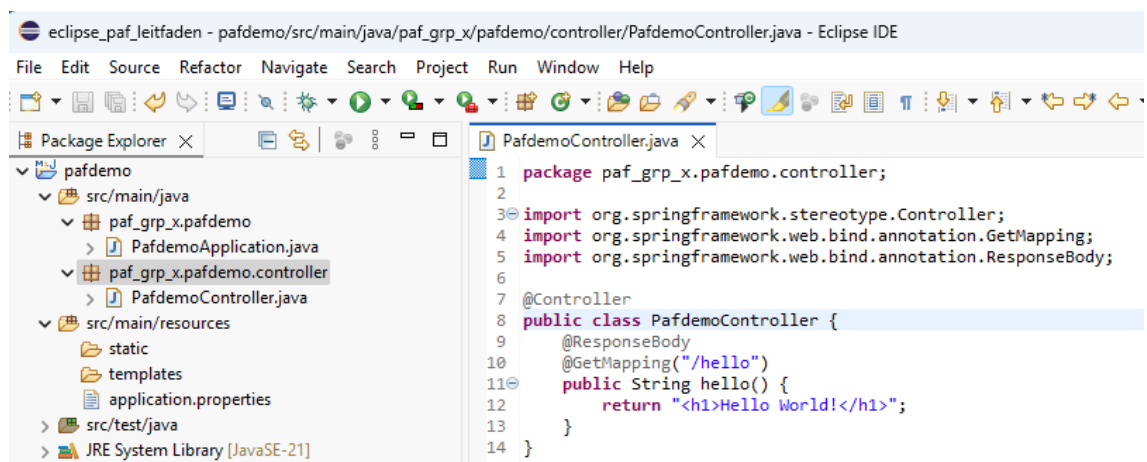


Abb. 2.9: Klasse PafdemoController

Wir arbeiten hier mit Annotationen aus dem Spring-Framework, um unsere Klasse und die darin enthaltene Methode mit Eigenschaften zu versehen, die zur Ausführungszeit zu einem bestimmten Verhalten führen. Konkret erhält die Klasse die Annotation `@Controller`, die Methode die Annotationen `@ResponseBody` und `@GetMapping`. Damit wird spezifiziert, dass die Methode `hello()` genau dann ausgeführt werden soll, wenn der REST-Endpunkt `/hello` mit einem (HTTP-)GET-Request angesprochen wird. Die Annotation `@ResponseBody` legt dabei fest, dass der Return-Wert der Methode an den Response-Body gebunden werden soll, also nicht den Namen eines HTML-Templates darstellt (dazu später mehr). Auf diese Weise können verschiedene Endpunkte eines REST-API definiert werden.

Dass dies funktioniert, können wir durch Aufruf der URL `http://localhost:8080/hello` zeigen.

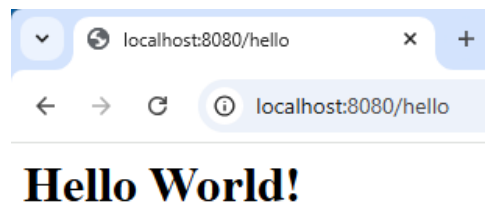


Abb. 2.10: Hello-World-Seite im Browser

Damit haben wir eine erste, minimale, lauffähige SpringBoot-Anwendung.

2.3 Anwendung strukturieren

Dieser Abschnitt enthält Hinweise zur Strukturierung Ihrer Softwareanwendung, zum einen serverseitig, zum anderen clientseitig.

Serverseitige Projektstruktur

Serverseitig ist durch die Verwendung von Maven eine gewisse Projektstruktur bereits vorgegeben. Typisch für die Top-Level-Ebene sind insbesondere die folgenden Ordner:

- `src/main/java` für unsere Java-Packages sowie deren Klassen und Interfaces
- `src/main/resources` für statische Frontend-Ressourcen, Thymeleaf-Templates sowie die SpringBoot-Konfigurationsdatei `application.properties`
- `src/test/java` für JUnit-Tests

Erwähnenswert auf Top-Level-Ebene ist weiterhin die zentrale Maven-Konfigurationsdatei `pom.xml`, welche insbesondere die Projektabhängigkeiten spezifiziert.

Package-Struktur

Innerhalb von *src/main/java* empfiehlt sich eine Package-Struktur entlang der wesentlichen Komponenten einer SpringBoot-Serveranwendung, z. B.:

- controller
- model
- repository
- security
- service
- websocket

Viele dieser Packages werden wir im Laufe des Leitfadens erzeugen und mit entsprechenden Klassen und Interfaces füllen. Innerhalb der Packages ist eine fachliche Untergliederung sinnvoll, z. B. innerhalb des controller-Package Klassen wie GameController, PlayerController usw.

Anwendungslogik in Service-Klassen

Bitte vermeiden Sie möglichst Anwendungslogik in den Controller-Klassen. Gemäß des MVC-Musters sollen Controller-Klassen sich auf die Steuerung der Views (z. B. Thymeleaf-Templates oder Sichten in Vue.js) und auf den Datenaustausch zwischen Views und Model konzentrieren. Anwendungslogik können Sie besser in entsprechende Service-Klassen (GameService, PlayerService usw., jeweils mit SpringBoot-Annotation `@Service`) auslagern. Strenggenommen gehören diese dann mit zum Model der Anwendung (Model = Daten + Anwendungslogik), Sie können Service-Klassen aber auch in einem eigenen service-Package bündeln.

Ein weiterer Vorteil der Platzierung von Anwendungslogik in Service-Klassen ist die bessere Wiederverwendbarkeit. So können Sie z. B. eine Anwendungsfunktion einmal implementieren und aus verschiedenen Controllern heraus nutzen.

Ein Service-Objekt können Sie per Dependency Injection in einen Controller einbinden:



Code

```
@Autowired
private final GameService gameService;
//...
Game game = gameService.createGame(...);
```

DTOs zum Datenaustausch

Client(s) und Server werden über die von Ihnen spezifizierten APIs (insb. REST und WebSocket) Daten austauschen. Hierfür empfiehlt sich die Verwendung sog. Data Transfer Objects (DTOs). Diese fassen mehrere Attributwerte zusammen, die bei Aufruf eines bestimmten Endpunkts en bloc zwischen Client und Server übertragen werden sollen. DTOs werden serverseitig über einfache Java-Klassen spezifiziert, wie das folgende Beispiel zeigt.



Code

```
@Getter
@Setter
public class CredentialsDto {
    private String username;
    private String password;
}
```

Ein großer Vorteil von DTOs ist, dass sie von SpringBoot automatisch vor dem Versand an den Client in JSON-Objekte umgewandelt werden. Umgekehrt werden beim Server eingehende JSON-Objekte automatisch in DTOs gewandelt. Dieser Mechanismus sorgt dafür, dass man sich als Anwendungsentwickler*in serverseitig gar nicht um JSON kümmern muss, sondern rein in der objektorientierten Welt bleiben kann. Wir werden ihm im weiteren Verlauf des Leitfadens immer wieder begegnen.

Objekte von Model-Klassen sind meist weniger geeignet für den Datentransfer zwischen Client und Server, da sie oft mehr Informationen enthalten als nötig, was sowohl unnötige Netzlast erzeugt als auch datenschutzrechtlich kritisch sein kann.



Beispiel

Beim Server geht ein Request ein, der einen Usernamen anfordert. Würde der Server in seiner Response mit dem kompletten User-Objekt (also einem Objekt der Model-Klasse User) antworten, würde er mehr Daten als nötig übertragen (z. B. auch den Vornamen des Users).

Wenn man sich vor Augen führt, dass das User-Objekt vermutlich auch das (hoffentlich) gehashte User-Passwort enthält, werden die mit diesem Ansatz verbundenen Datenschutz- und Sicherheitsprobleme offensichtlich.

Die Verwendung eines DTO, das ausschließlich den Usernamen als String-Attribut enthält, vermeidet hingegen die genannten Probleme.

DTOs bündeln also Use-Case-spezifische Einzelwerte für den Transfer über APIs. DTO-Klassen sollten zur besseren Abgrenzung am besten in einem separaten dto-Package abgelegt werden und nicht im model-Package.

Clientseitige Projektstruktur

Je nachdem, ob Sie einen Client mit Thymeleaf-Templates oder mit einem Frontend-Framework wie Vue.js bauen, unterscheidet sich die jeweilige Projektstruktur.

Thymeleaf-Projektstruktur

Das Thymeleaf-Frontend ist Teil des Maven-Projekts, liegt also auf Serverseite. Unterhalb von *src/main/resources* gibt es typischerweise einen Ordner *static* mit statischen Assets, ggf. untergliedert in Verzeichnisse wie *css*, *js* und *img*. Die Thymeleaf-Templates werden in einem Ordner *templates* abgelegt, ebenfalls unterhalb von *src/main/resources*.

Der Thymeleaf-Client steht nach dem Start der Serveranwendung auf dem Port zur Verfügung, auf dem der Application Server (Tomcat) der SpringBoot-Anwendung gestartet wurde. Defaultmäßig ist das der Port 8080, dies kann aber bei Bedarf in *application.properties* über die Variable *server.port* geändert werden.

Vue.js-Projektstruktur

Bei Verwendung von Vue.js (oder einem anderen Frontend-Framework) liegt die Projektstruktur typischerweise in einem eigenen Ordner, unabhängig vom Maven-Projekt. Die konkrete Struktur kann sich je nach Framework unterscheiden. Für ein Vue.js-Projekt sind z. B. folgende Ordner unterhalb von *src* typisch:

- *api*
- *assets*
- *components*
- *router*
- *services*
- *stores*
- *utils*
- *views*

Direkt in *src* liegen meist zentrale Dateien wie *App.vue* und *main.js*. Eine Ebene über *src* finden sich Konfigurationsdateien wie *package.json* und *vite.config.js* sowie *index.html* als zentraler Einstiegspunkt in die Client-Anwendung.

Der Vue-Client muss separat vom Server gestartet werden und läuft dementsprechend auf einem anderen Port. Bei Verwendung des Build-Werkzeugs Vite ist dies per Default der Port 5173, was bei Bedarf in *package.json* geändert werden kann.



Gliederung

3 User verwalten mit REST, Spring Security und JPA

3 User verwalten mit REST, Spring Security und JPA

3.1 User-Klasse erstellen und mit JPA verknüpfen

3.2 REST-API für die User-Verwaltung definieren

3.3 Mit Spring Security den Zugang zur Web-Anwendung schützen

In diesem Abschnitt des Leitfadens werden wir zunächst eine einfache User-Klasse erstellen, deren Objekte in unserer Datenbank mittels JPA persistiert werden können. Diese User-Klasse stellt dann die Basis für einen über Spring Security abgesicherten, sessionbasierten Zugang zu (Teilen) unserer Anwendung dar.

Das Registrieren von neuen Usern, das Einloggen in den geschützten Bereich sowie ggf. weitere Funktionen der User-Verwaltung werden über ein REST-API realisiert.



Hinweis

Im Kapitel [JWT verwenden](#) werden wir mit den sog. JSON Web Tokens (JWT) eine alternative Art der Zugangskontrolle kennenlernen, die ohne serverseitige Sessions auskommt.

3.1 User-Klasse erstellen und mit JPA verknüpfen

Zuerst erzeugen wir in unserem Projekt ein neues Package `paf_grp_x.pafdemo.model` und darin eine Klasse `User` mit dem in der folgenden Abbildung gezeigten Quellcode.

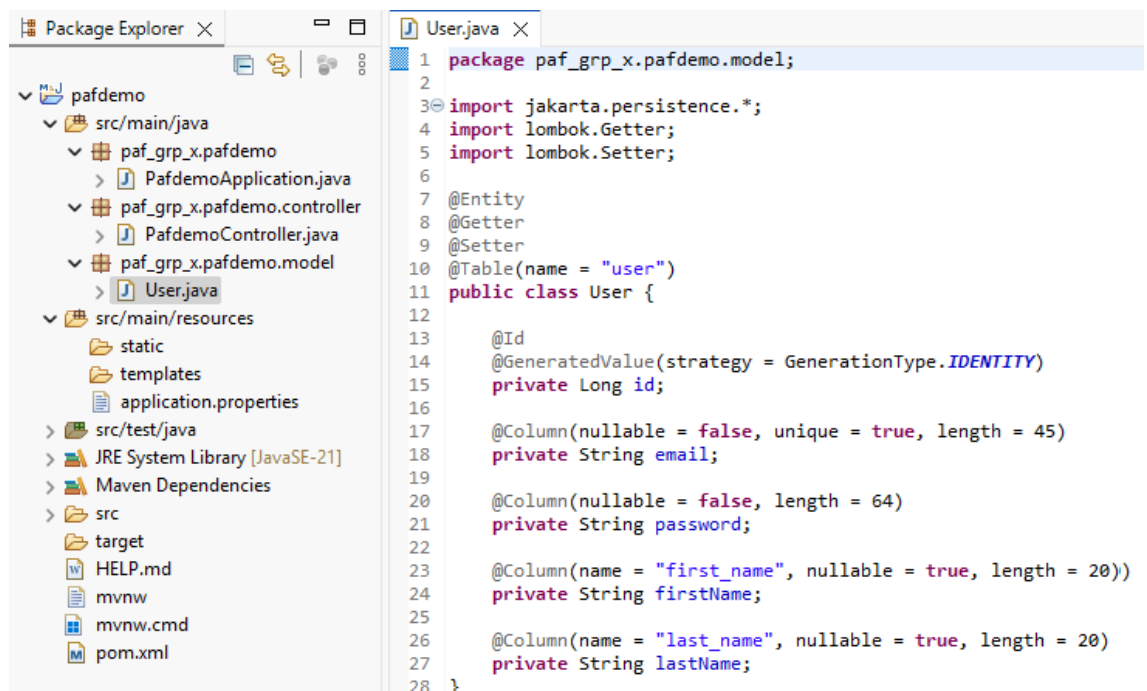


Abb. 3.1: Einfache User-Klasse mit JPA- und Lombok-Annotationen

Die Klasse selbst und auch ihre Attribute sind mit diversen JPA-Annotationen versehen. Diese dienen dem objektrelationalen Mapping (ORM), d. h. der Abbildung von objektorientierten Datenstrukturen in das relationale Datenmodell. So wird die Klasse über `@Entity` als Entity-Klasse deklariert, deren Objekte in der Datenbank zu persistieren sind. Mit `@Table` wird der mit der Klasse User zu assoziierende Tabellename (hier: `user`) spezifiziert. `@Id` und `@GeneratedValue` legen das Primärschlüsselattribut fest und sorgen für ein Auto-Increment der generierten Primärschlüssel. Die `@Column`-Annotationen sollten weitgehend selbsterklärend sein, wenn man sich mit SQL (z. B. `CREATE TABLE`) auskennt.



Hinweis

Bei Modellen, die aus mehr als einer Klasse bzw. Datenbanktabelle bestehen - in "echten" Anwendungen natürlich der Normalfall -, müssen zusätzlich Beziehungen zwischen Klassen auf Beziehungen zwischen Tabellen abgebildet werden. Hierfür stehen JPA-Annotationen wie `@OneToMany`, `@ManyToOne`, `@OneToOne`, `@ManyToMany`, `@JoinTable` und `@JoinColumn` zur Verfügung. An dieser Stelle betrachten wir beispielhaft die Beziehungen zwischen einer Klasse `Game` und einer Klasse `Player`. In `Game` gebe es Attribute `player1` und `player2`, jeweils vom Typ `Player`. D. h. jedes `Game`-Objekt hat genau einen `Player 1` und einen `Player 2`. Umgekehrt soll ein `Player`-Objekt mit mehreren `Game`-Objekten verknüpft sein können, seien es zeitgleich laufende oder vergangene Spiele. Es ergibt sich also aus `Game`-Sicht eine many-to-one-Beziehung zu `Player`. Diese können wir in der Klasse `Game` modellieren, wie nachfolgend gezeigt.



Code

```
@ManyToOne
@JoinColumn(name = "player1_id", nullable = false)
private Player player1;

@ManyToOne
@JoinColumn(name = "player2_id", nullable = true)
private Player player2;
```

`@JoinColumn` legt jeweils fest, über welches Fremdschlüssel-Attribut in der `Game`-Tabelle die Verknüpfung zum Primärschlüsselattribut der `Player`-Tabelle erfolgen soll. Bei `player2` steht `nullable` auf `true`, weil man Spielzustände modellieren möchte, bei denen `player1` ein neues Spiel erzeugt hat, für das aber noch kein `player2` gefunden wurde.

Die Annotationen `@Getter` und `@Setter` gehören zu Lombok, einer Java-Bibliothek, die es Ihnen erspart, sog. Boilerplate-Code selbst schreiben zu müssen. In diesem Beispiel erzeugt Lombok im Hintergrund automatisch die Getter- und Setter-Methoden für die einzelnen Attribute der Klasse. Wir können diese Methoden wie üblich aufrufen, obwohl sie nicht explizit im Quellcode deklariert sind.

Um nun User-Objekte tatsächlich in die Datenbank schreiben und von dort wieder einlesen zu können, benötigen wir noch ein sog. Repository. Wir legen ein Package `paf_grp_x.pafdemo.repository` an und darin ein Interface `UserRepository`, das das generische Interface `JpaRepository` beerbt und es auf die Model-Klasse `User` und deren Id-Typ `Long` spezialisiert.

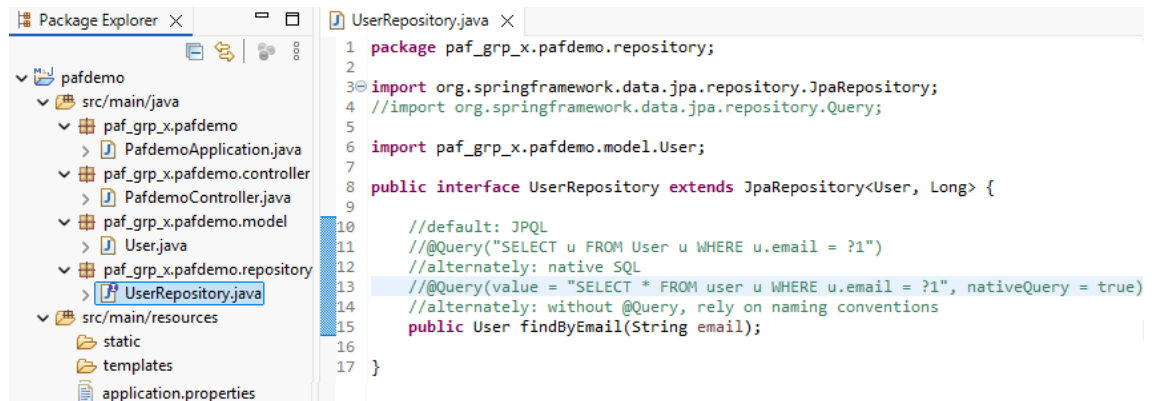


Abb. 3.2: Interface `UserRepository`

Das Interface spezifiziert die Methode `findByEmail()`. Bemerkenswert ist, dass diese Methode nirgends von uns implementiert wird. Dennoch können wir - wie wir noch sehen werden - diese Methode aufrufen. Es handelt sich hier um ein weiteres Beispiel für "Convention over Configuration": Spring Boot interpretiert den Methodennamen (`findByEmail`), den Parameter (`String email`) und den Rückgabebetyp (`User`) so, dass ein Methodenaufruf offenbar einen User anhand seiner email(-Adresse) aus der Datenbank selektieren soll. Diese Implementierung des Interface erfolgt automatisch im Hintergrund durch das Framework.

Im weiteren Projektverlauf kann es erforderlich werden, komplexere Datenbankabfragen zu spezifizieren. Hierzu kann man die Annotation `@Query` nutzen in Verbindung mit einer JPQL-Anfrage (Hybrid aus SQL und Objektorientierung) oder einer nativen SQL-Anfrage. Beides wird in den Kommentarzeilen oberhalb von `findByEmail()` angedeutet.

Die verschiedenen Möglichkeiten, Datenbankabfragen mit Spring Data JPA zu erzeugen, werden in der [Spring-Dokumentation](#) genauer erläutert.

Da unsere Anwendung derzeit noch über keine Funktionalität verfügt, mit der man das `UserRepository` ausprobieren könnte, erstellen wir uns hierfür einen JUnit-Test. Im Ordner `src/test/java` und dem darin enthaltenen Package `paf_grp_x.pafdemo` gibt es bereits eine (vom Spring Initializr) erzeugte, mit `@SpringBootTest` annotierte Testklasse, die wir für unsere Zwecke nutzen können. Wir deklarieren in der Testklasse eine Methode `testCreateUser()`, wie die folgende Abbildung zeigt.

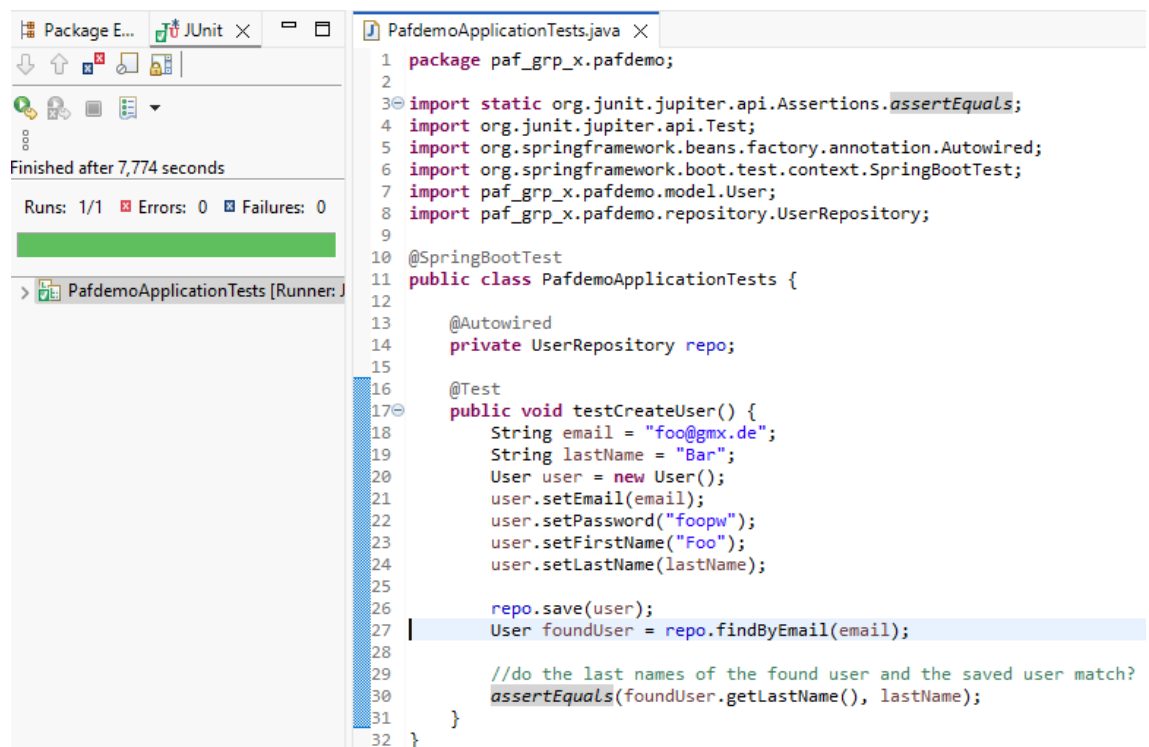


Abb. 3.3: Testklasse PfademoApplicationTests

In der Testklasse begegnet uns die Annotation `@Autowired`, die typisch für Spring Boot ist und die wir im weiteren Projektverlauf immer wieder verwenden werden. `@Autowired` legt fest, dass das folgende Attribut (kann auch z. B. ein Konstruktor oder ein Setter sein) automatisch der Dependency Injection von Spring unterliegen soll. Konkret bedeutet das hier: Das Instanzattribut `repo` wird automatisch vom Framework instanziiert. Wir müssen uns also nicht selbst darum kümmern, ein `UserRepository`-Objekt zu erzeugen und dem Attribut `repo` zuzuweisen.

Der Test läuft bei Ausführung erfolgreich durch und legt den Testuser in der Datenbank an, was wir z. B. mit der MySQLWorkbench überprüfen können:

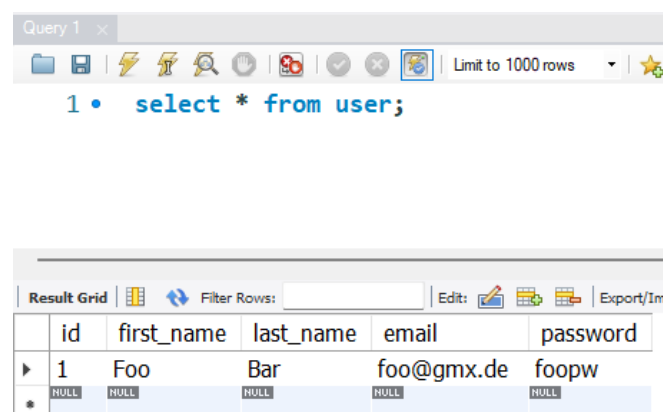


Abb. 3.4: Datenbanktabelle user

Wie wir außerdem sehen, sind die Tabelle und ihre Spalten exakt so benannt, wie wir sie mit den Annotationen in der User-Klasse spezifiziert haben. Des weiteren können wir in der IDE-Konsole einsehen, welche SQL-Befehle im Hintergrund vom Framework und ohne unser Zutun generiert und ausgeführt wurden.

Derzeit wird das User-Passwort noch im Klartext gespeichert - diesen Missstand werden wir in Kürze beheben.

Damit die Datenbanktabellen nicht bei nächsten Serverstart gelöscht und neu (leer) erzeugt werden, ändern wir in application.properties die Zeile



Code

```
spring.jpa.hibernate.ddl-auto=create
```

in



Code

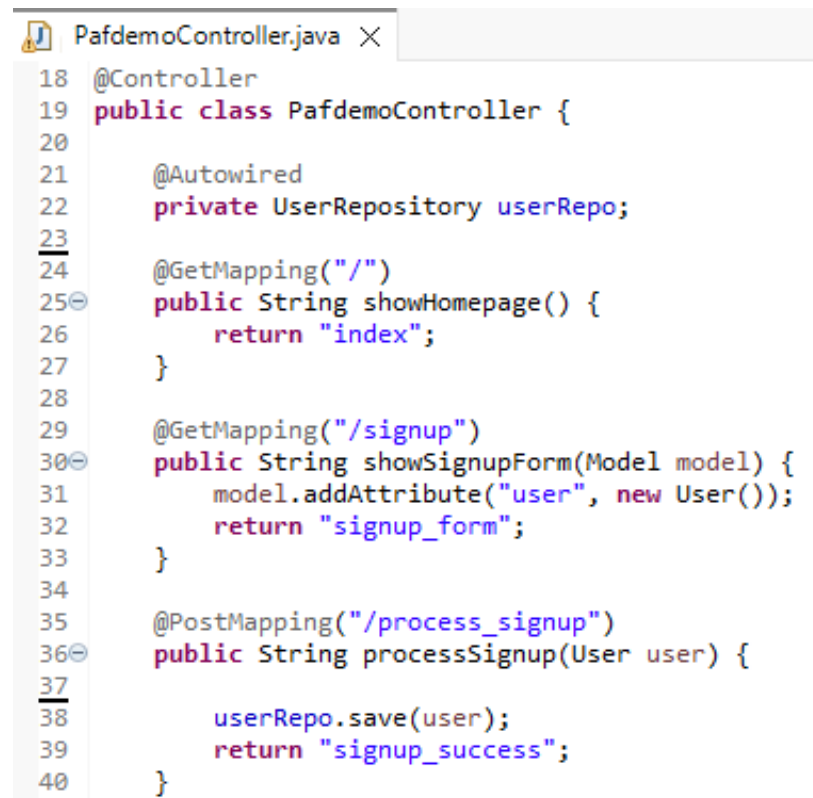
```
spring.jpa.hibernate.ddl-auto=none
```

Auf diese Weise werden Schemaänderungen (inkl. Schemalöschungen) unterbunden. Die angelegten Datenbanktabellen inkl. Inhalt bleiben nun auch nach einem Neustart des Servers erhalten.

3.2 REST-API für die User-Verwaltung definieren

Registrieren (Sign-Up)

Wir werden nun den PdfdemoController um weitere REST-Endpunkte ergänzen, über die sich ein neuer User registrieren kann. Dabei gehen wir minimalistisch vor und nehmen ausschließlich die E-Mail-Adresse und das Passwort auf. Vor- und Nachname, die ja ebenfalls in unserer User-Klasse spezifiziert waren, bleiben in dieser Demo leer. (In Ihrem Projekt sollten Sie solche "Leerstellen" hingegen vermeiden.)



```
18 @Controller
19 public class PfademoController {
20
21     @Autowired
22     private UserRepository userRepo;
23
24     @GetMapping("/")
25     public String showHomepage() {
26         return "index";
27     }
28
29     @GetMapping("/signup")
30     public String showSignupForm(Model model) {
31         model.addAttribute("user", new User());
32         return "signup_form";
33     }
34
35     @PostMapping("/process_signup")
36     public String processSignup(User user) {
37
38         userRepo.save(user);
39         return "signup_success";
40     }
}
```

Abb. 3.5: Controller, erweitert um Endpunkte zur Registrierung

Die Abbildung zeigt drei weitere Endpunkte: "/" wird zur Startseite weiterleiten, auf der wir ein einfaches Menü zur Verfügung stellen werden. "/signup" ruft ein leeres Registrierungsformular auf, und "/process_signup" verarbeitet die Daten des ausgefüllten Registrierungsformulars.

Da im letztgenannten Fall Daten vom Client an den Server übertragen werden, um ein neues User-Objekt zu persistieren, wurde hier ein `@PostMapping` gewählt. Zum Persistieren des User-Objekts wird - wie wir es bereits in der Testklasse kennengelernt haben - die `save()`-Methode unseres `UserRepository` genutzt. Das Passwort wird aktuell noch im Klartext gespeichert - das werden wir in Kürze ändern.

Erwähnenswert sind noch die Parameter der Methoden: Sowohl `model` als auch `user` werden von Spring zur Laufzeit per Dependency Injection bereitgestellt. `model` dient dazu, in einem HTML-Template (siehe unten) auf Objekte aus dem Model (hier: auf User-Objekte) zugreifen zu können. Konkret wird hier ein neues User-Objekt erzeugt und mit `addAttribute()` dem Model hinzugefügt. Im HTML-Template kann dann auf dieses User-Objekt zugegriffen werden - insbesondere können clientseitig die Werte des Objekts gesetzt werden, z. B. anhand von Formulareingaben. Das mit Werten versehene User-Objekt wird dann der Methode `processSignup()` übergeben, damit diese es weiterverarbeiten kann.



Sollten Sie beim Zugriff auf den Endpunkt `/signup` auf die Default-Login-Seite von Spring Security umgeleitet werden, liegt das daran, dass wir Spring Security erst im folgenden Abschnitt passgenau konfigurieren werden und bis dahin alle Endpunkte per Default zugriffsgeschützt sind.

Als Workaround können Sie die Dependencies mit den artifactIds *spring-boot-starter-security*, *spring-security-test* und *thymeleaf-extras-springsecurity6* temporär in *pom.xml* auskommentieren und den Server danach neu starten.

Bitte denken Sie aber daran, die Dependencies vor dem Durcharbeiten des folgenden Abschnitts wieder einzukommentieren.

Die drei neuen Endpunkte sind - anders als der `/hello`-Endpunkt - *nicht* mit `@ResponseBody` annotiert. Der Grund hierfür ist, dass die Rückgabewerte der Methoden hier nicht über den Response-Body zum Client übertragen werden sollen, sondern Namen von HTML-Templates darstellen, die jeweils aufzurufen sind.

Das erste dieser Templates ist `index.html`. Es bietet ein einfaches Menü mit drei Links. Bemerkenswert ist der `th`-Namespace, der die Verwendung der Template Engine Thymeleaf anzeigt. An dieser Stelle verwenden wir Thymeleaf nur zur Verlinkung zu den jeweiligen REST-Endpunkten (von denen wir bisher nur `/signup` deklariert haben). In den nachfolgenden Templates wird der Mehrwert von Thymeleaf deutlicher werden.

```
1 <!DOCTYPE html>
2 <html xmlns:th="http://www.thymeleaf.org">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>PaF Demo</title>
7   <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel=
8   <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js
9 </head>
10 <body>
11   <div class="container text-center" style="max-width: 400px;">
12     <h1 class="mb-4">PaF Demo</h1>
13     <a th:href="@{/signup}" class="btn btn-primary btn-lg d-block w-100 mb-3">Sign Up</a>
14     <a th:href="@{/login}" class="btn btn-success btn-lg d-block w-100 mb-3">Sign In</a>
15     <a th:href="@{/hello}" class="btn btn-info btn-lg d-block w-100">Hello World!</a>
16   </div>
17 </body>
18 </html>
```

Abb. 3.6: `index.html` mit einfachem Menü

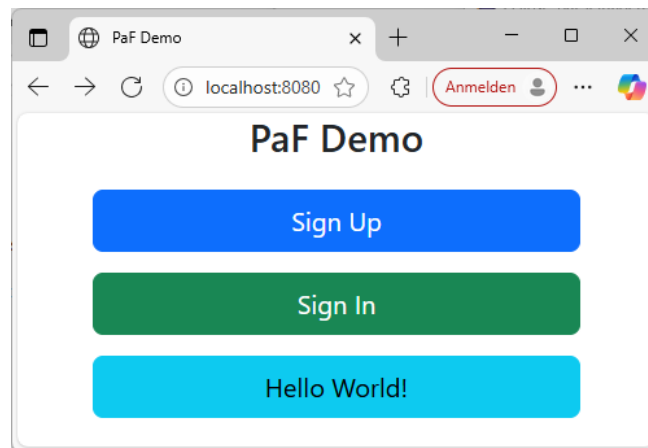


Abb. 3.7: Startseite im Browser

Das Registrierungsformular (Quelltext und Rendering siehe unten) macht wiederum Gebrauch von Thymeleaf: Es greift das im Model (siehe oben) hinterlegte User-Objekt im form-Tag auf (`th:object="${user}"`), so dass die Input-Elemente auf die einzelnen User-Attribute Bezug nehmen können (`th:field="*{email}"`). Dadurch werden die Formularfelder bidirektional an das User-Objekt gebunden. In das Formular eingetragene Werte werden von Thymeleaf in das User-Objekt eingetragen, eventuell bereits vorhandene Werte werden im Formular angezeigt. Notationen wie `*{email}` und `*{password}` suggerieren hier, dass Thymeleaf direkt auf die Attribute eines User-Objekts zugreifen kann. Das ist aber nicht der Fall. Wie Sie dem Quellcode der User-Klasse entnehmen können, sind deren Attribute als `private` deklariert. Thymeleaf muss für den Zugriff also Methoden wie `getEmail()` oder `setPassword()` aufrufen. Diese sind in `User.java` nicht sichtbar, weil sie intern von Lombok erzeugt und der Klasse hinzugefügt werden. Auch hier arbeitet das Framework also wieder mit Konventionen; konkret wird hier der Name der get- bzw. set-Methode aus dem Attributnamen abgeleitet.



Hinweis

Thymeleaf als Template Engine bietet weitaus mehr Möglichkeiten als bisher gezeigt. So werden z. B. auch Kontrollstrukturen wie Schleifen (`th:each`) und Fallunterscheidungen (`th:switch`, `th:case`) unterstützt. Nähere Informationen hierzu finden Sie in der [Thymeleaf-Dokumentation](#).

Bei der Submit-Aktion (`th:action="@{/process_signup}"`) wird das "ausgefüllte" User-Objekt im Post-Request mitgeschickt, so dass es serverseitig verarbeitet werden kann. Bitte beachten Sie, dass das User-Objekt im Formular nur zur Verfügung steht, weil es im Controller dem Model-Objekt hinzugefügt wurde (siehe oben).

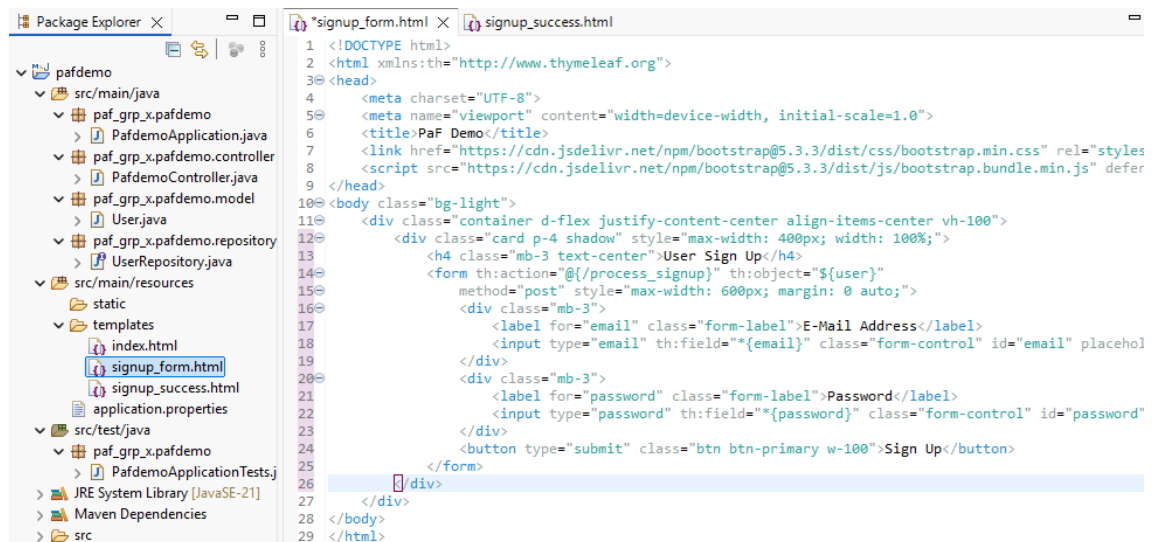


Abb. 3.8: Registrierungsformular in HTML

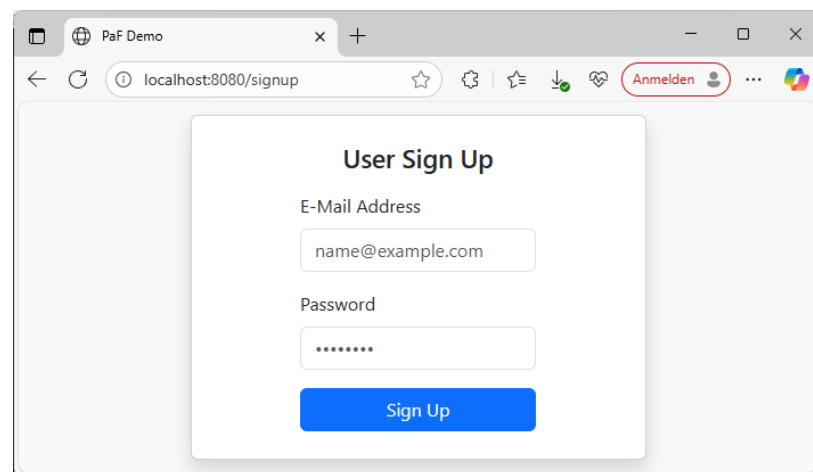


Abb. 3.9: Registrierungsformular im Browser

Nach erfolgreicher Registrierung wird im Browser eine Bestätigung angezeigt, siehe Quellcode und Rendering unten. (Den Fall, dass die Registrierung fehlschlägt, insb. weil es bereits einen User mit derselben E-Mail-Adresse gibt, betrachten wir in diesem Leitfaden nicht. In Ihrem Projekt sollten Sie diesen Fall aber sehr wohl bedenken.) Es besteht nun die Möglichkeit, sich in der Web-Anwendung anzumelden (Sign in).



Abb. 3.10: Registrierungsbestätigung in HTML

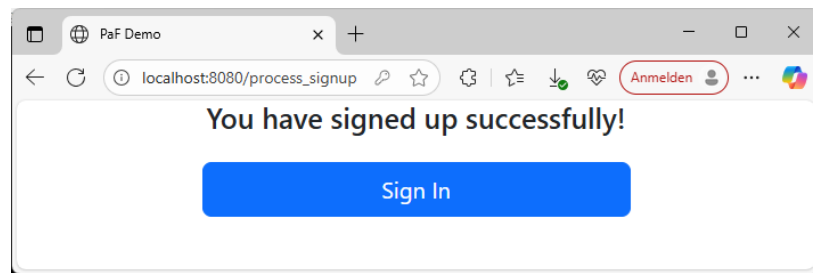


Abb. 3.11: Registrierungsbestätigung im Browser

Einloggen (Sign-In)

Den Endpunkt `/login`, der sowohl aus der Registrierungsbestätigung als auch aus dem Hauptmenü heraus angesprochen werden kann, haben wir in unserem `PaFdemo`-Controller nicht deklariert. Das hat den Hintergrund, dass wir in diesem Fall Default-Funktionalität von Spring Security nutzen. Der Endpunkt `/login` wird also von Spring Security selbst bereitgestellt. Dies schließt auch die Prüfung ein, ob die eingegebenen Zugangsdaten korrekt sind. Natürlich darf nur in diesem Fall der Zugang zu dem geschützten Bereich der Webanwendung gewährt werden. Welche Konfigurationen und "Verdrahtungen" hierfür erforderlich sind, werden wir uns im Abschnitt Mit Spring Security den Zugang zur Web-Anwendung schützen anschauen.

Der Default-Login-Dialog von Spring Security bringt den Nachteil mit sich, dass er nicht an das Design unserer Anwendung angepasst werden kann. Wenn wir eine eigene Login-Seite verwenden wollen, sind dafür Anpassungen nötig, die ebenfalls im Abschnitt Mit Spring Security den Zugang zur Web-Anwendung schützen erläutert werden.

3.3 Mit Spring Security den Zugang zur Web-Anwendung schützen

Damit die Objekte unserer eigenen User-Klasse von Spring Security verarbeitet, also z. B. zur Zugangskontrolle herangezogen werden können, müssen wir eine entsprechende Verknüpfung vornehmen. Dies machen wir anhand dreier Klassen, die in unserem Demo-Projekt `CustomUserDetails`, `CustomUserDetailsService` und `WebSecurityConfig` heißen. Die Klassennamen sind frei wählbar.

Klasse `WebSecurityConfig`

In `WebSecurityConfig` konfigurieren wir Spring Security nach unseren Wünschen. Unter anderem legen wir hier fest, welche Bereiche unserer Anwendung nur angemeldeten Usern zugänglich und welche frei zugänglich sein sollen. Als Beispiel wollen

wir für den Endpunkt `/hello` nur authentifizierten Zugriff erlauben, während alle anderen Endpunkte, insb. `/`, `/signup` und natürlich `/login` allen Client-Requests offen stehen.

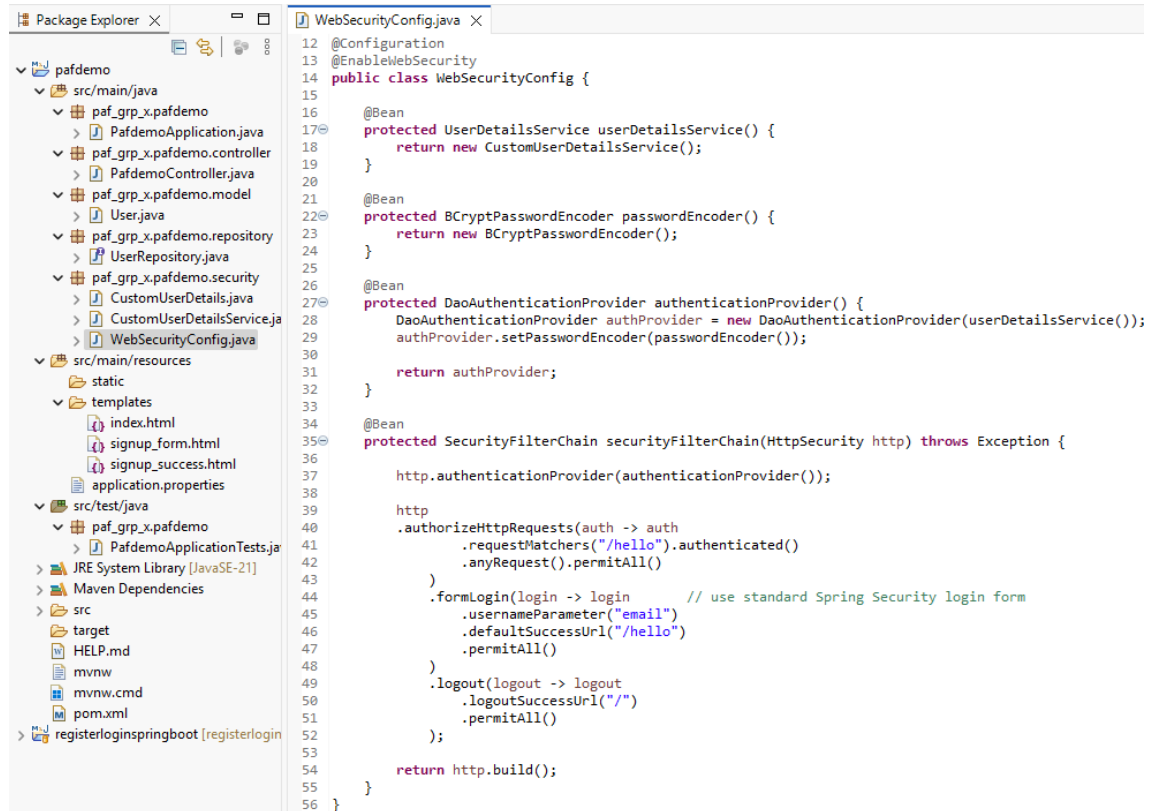


Abb. 3.12: Klasse WebSecurityConfig

Die Annotation `@Configuration` markiert die Klasse als Spring-Konfiguration, d. h. sie wird automatisch vom Spring-Framework als Konfigurationsklasse erkannt und entsprechend behandelt. Die Annotation `@EnableWebSecurity` aktiviert die Spring-Security-Infrastruktur. Die `@Bean`-Annotationen der Methoden legen fest, dass die Rückgabeobjekte jeweils von Spring verwaltet werden sollen (im sog. Application Context).

Zentrale Methode von `WebSecurityConfig` ist `securityFilterChain()`. Sie enthält die HTTP-Sicherheitsregeln, also insbesondere, welche Endpunkte geschützt werden sollen und welche nicht. In unserem Fall soll der Endpunkt `/hello` nur authentifizierten Usern zur Verfügung stehen, während alle anderen Requests für alle erlaubt sind (`anyRequest().permitAll()`). Der Aufruf von `formLogin()` legt fest, dass das Standard-Login-Formular von Spring Security verwendet werden soll, wobei das Feld `email` als Username genutzt wird und nach erfolgreichem Login eine Weiterleitung zum (geschützten) Endpunkt `/hello` erfolgt. Entsprechend wird beim Logout (Button wird im weiteren Projektverlauf ergänzt) zum Endpunkt `/`, also zur Startseite weitergeleitet.



Die Notation der Sicherheitsregeln basiert auf dem sog. Fluent API von Java. Dieses nutzt intensiv die Verkettung von Methodenaufrufen, um eine bessere Lesbarkeit des Quellcode im Sinne einer domänenspezifischen Sprache (domain specific language, DSL) zu erreichen. Technisch wird die Verkettung von Aufrufen dadurch möglich, dass jede Methode das Objekt zurückgibt, zu dem es gehört (also "this"). Auf diese Weise können beliebig viele Aufrufe kaskadiert werden.

`http.build()` erstellt aus den zuvor spezifizierten Regeln eine sog. Filterkette (Filter Chain), die von Spring Security zur Zugangsprüfung verwendet wird. Wir werden im weiteren Projektverlauf noch mehrmals auf die Filterkette zurückkommen und sie an unsere Bedürfnisse anpassen. Nähere Informationen zum Konzept der Filterkette finden Sie in der Spring-Dokumentation.

Neben diesen Sicherheitsregeln muss noch spezifiziert werden, wie die Authentifizierung erfolgen soll. Dies wird in der Methode `authenticationProvider()` festgelegt. Hier erfolgt - indirekt über `userService()` - eine Verknüpfung zu unserem selbst-definierten `CustomUserService` (siehe unten). Durch die Annotation `@Bean` der Methode `userService()` wird erreicht, dass das hier erzeugte Objekt der (Singleton-)Klasse `CustomUserService` im Application Context von Spring Boot abgelegt wird und somit an anderen Stellen zur Dependency Injection bereitsteht.

Auf analoge Weise wird in `passwordEncoder()` festgeschrieben, dass der `BCryptPasswordEncoder` zum Hashen von eingegebenen Passwörtern verwendet werden soll. Wiederum stellt `@Bean` sicher, dass ein Objekt der (Singleton-)Klasse `BCryptPasswordEncoder` im Application Context abgelegt wird.

Injizierung des `BCryptPasswordEncoder`-Objekts in den `PafdemoController`

Dies ermöglicht nun die Injizierung dieses Objekts u. A. in der Klasse `PafdemoController`, wie in der folgenden Abbildung gezeigt. `PasswordEncoder` selbst ist nur ein Interface. Spring Boot sucht daher im Application Context nach einer konkreten Implementierung dieses Interface. Dort findet es nun ein Objekt der Klasse `BCryptPasswordEncoder` (siehe oben), das hier injiziert und zum Hashen von Klartextpassworten verwendet wird.

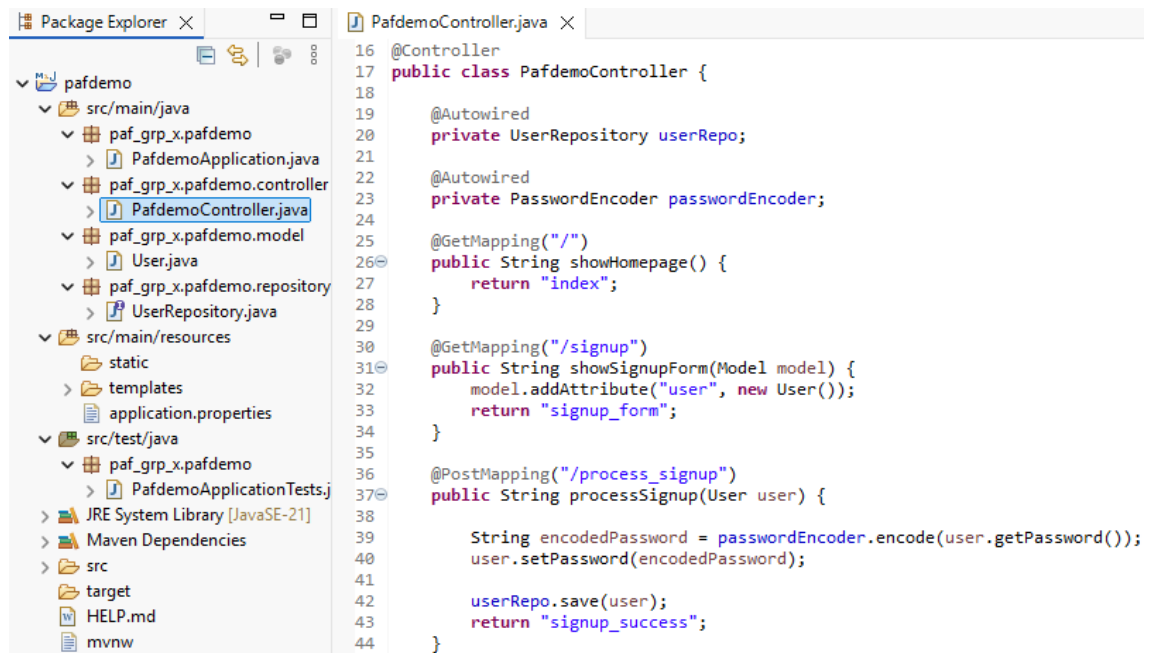


Abb. 3.13: Controller mit injiziertem PasswordEncoder

Durch diese "automatische Verdrahtung" (@Autowired) ist sichergestellt, dass derselbe Encoder, den wir zum Hashen von Passwörtern im Zuge einer Authentifizierung heranziehen (siehe oben), auch im Rahmen der Registrierung verwendet werden kann - siehe Methode processSignup(). Das im Registrierungsformular eingegebene Klartextpasswort wird mit dem daraus berechneten Hash-Wert überschrieben, bevor es mit repo.save() in die Datenbank geschrieben wird.

Noch besser wäre es natürlich, das Passwort bereits clientseitig zu schützen - dies werden wir im weiteren Projektverlauf umsetzen.

Klasse CustomUserDetailsService

Nun zum CustomUserDetailsService: Diese Klasse implementiert das Spring-Interface UserDetailsService, insbesondere die Methode loadUserByUsername(). Wie der Methodenname schon sagt, müssen wir in dieser Methode festlegen, wie anhand eines Usernamens das entsprechende User-Objekt (hier vom Spring-Typ UserDetails) ermittelt werden soll. In unserem Fall fungiert die E-Mail-Adresse als Username, und das User-Objekt holen wir mit der Methode findByEmail() aus unserem UserRepository.

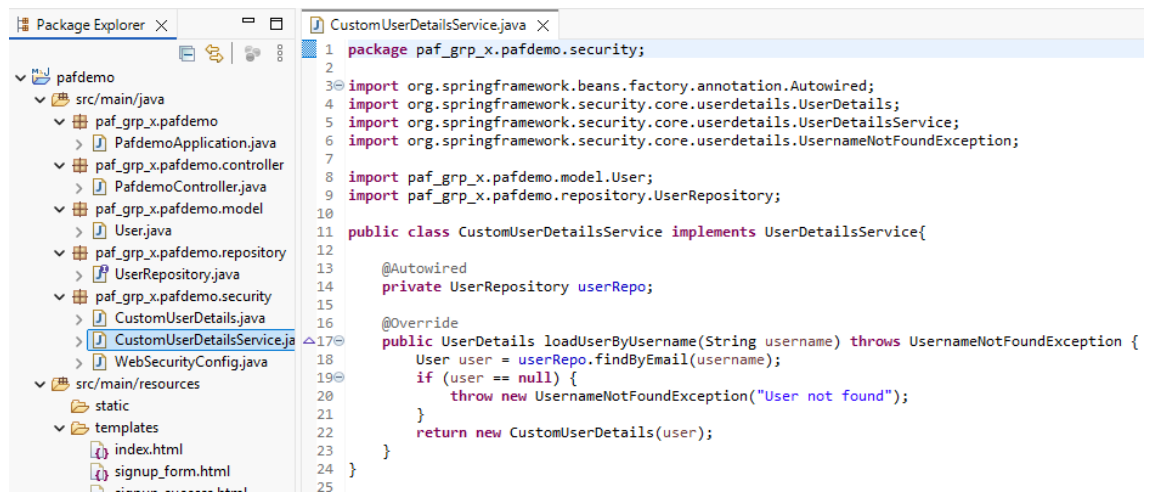
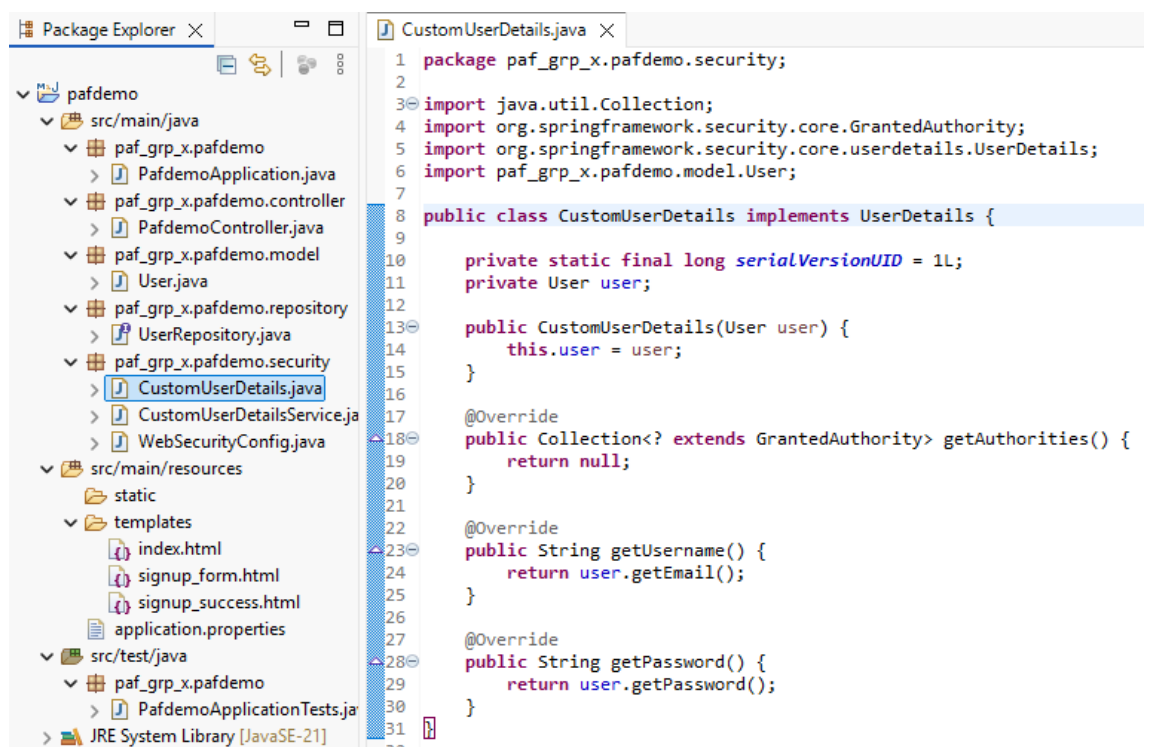


Abb. 3.14: Klasse CustomUserDetailsService

Klasse CustomUserDetails

Damit das Framework die User-Daten verarbeiten kann, müssen wir diese mittels einer Klasse bereitstellen, die das Interface UserDetails implementiert. Diese Klasse nennen wir CustomUserDetails (Quellcode siehe unten). Über den Konstruktor speisen wir ein Objekt unserer User-Klasse (Rückgabotyp von findByEmail()) ein. Indem wir die Methoden getUsername() und getPassword() von UserDetails überschreiben, ermöglichen wir dem Framework den Zugriff auf unsere Userdaten. Die Methode getAuthorities() müssen wir ebenfalls überschreiben - hiermit könnten wir Rollen und Rechte ausliefern, die wir unseren Usern gegeben haben. Da unsere Demo-Anwendung kein Rechtemanagement beinhaltet, lassen wir die Methode einfach null zurückgeben.





Hinweis

Abb. 3.15: Klasse CustomUserDetails

Die das Spring-Interface UserDetails implementierende Klasse CustomerUserDetails stellt eine Anwendung des Entwurfsmusters "Adapter" dar, vgl. Skript "Patterns and Frameworks" -> Entwurfsmuster -> Strukturmuster -> Adapter. Als Adapterklasse hilft sie, Methoden aus einer externen Komponente (hier aus Spring-Sicht: unsere eigene User-Klasse) aufzurufen und diese in das eigene System (hier: Spring Security) zu integrieren.

Unsere Anwendung verhält sich nun wie gewünscht: Die Endpunkte "/", "/signup" und "/login" sind frei zugänglich. Bei "/login" erscheint das Default-Anmeldeformular von Spring Boot. Der Endpunkt "/hello" ist geschützt. Ein Klick auf den "Hello World!"-Button leitet entsprechend weiter zum Anmeldeformular. Bei fehlerhaften Zugangsdaten erscheint eine Fehlermeldung. Bei korrekten Zugangsdaten erfolgt eine Weiterleitung zu "/hello". Da serverseitig nun eine User-Session gehalten wird, können wir nun auch von der Startseite aus auf "Hello World!" klicken und gelangen ohne erneuten Anmeldevorgang zu "/hello".

Custom Login

Wie bereits am Ende des Abschnitts REST-API für die User-Verwaltung definieren angedeutet, wollen wir nun das Default-Anmeldeformular von Spring Boot durch ein selbstgestaltetes Formular ersetzen. Im PdfdemoController ergänzen wir hierzu einen Endpunkt "/login", der im bereits angemeldeten Zustand direkt auf die Startseite weiterleitet und ansonsten das Thymeleaf-Template login.html aufruft.



Code

```
@GetMapping("/login")
public String login() {
    //check if user is already logged in
    Authentication authentication = SecurityContextHolder.getContext().getAuthentication();

    //if user is already logged in (not "anonymousUser"), redirect to menu
    if (authentication != null && authentication.isAuthenticated()
        && !(authentication instanceof AnonymousAuthenticationToken) ) {
        return "redirect:/?alreadyLoggedIn";
    }
    return "login";    //if not authenticated, show custom login page
}
```



Hinweis

Die Bedingung `!(authentication instanceof AnonymousAuthenticationToken)` ist erforderlich, da `authentication.isAuthenticated()` - wenig intuitiv - auch dann true liefert, wenn man nur als anonymer User auf die Seite gekommen, also noch nicht eingeloggt ist.

In der Klasse `WebSecurityConfig` machen wir der `SecurityFilterChain` unseren neuen Login-Endpoint bekannt und ersetzen damit den bisherigen Aufruf von `formLogin()`:



Code

```
.formLogin(login -> login
    .loginPage("/login").permitAll()
)
```

Anstelle des Default-Login-Diologs von Spring Security stellen wir nun das folgende Thymeleaf-Template bereit, das von unserem Controller retourniert wird (siehe oben). Beachten Sie hierbei bitte insbesondere die Formular-Aktion, also den POST-Request auf den von Spring Security bereitgestellten Endpoint `/login`.

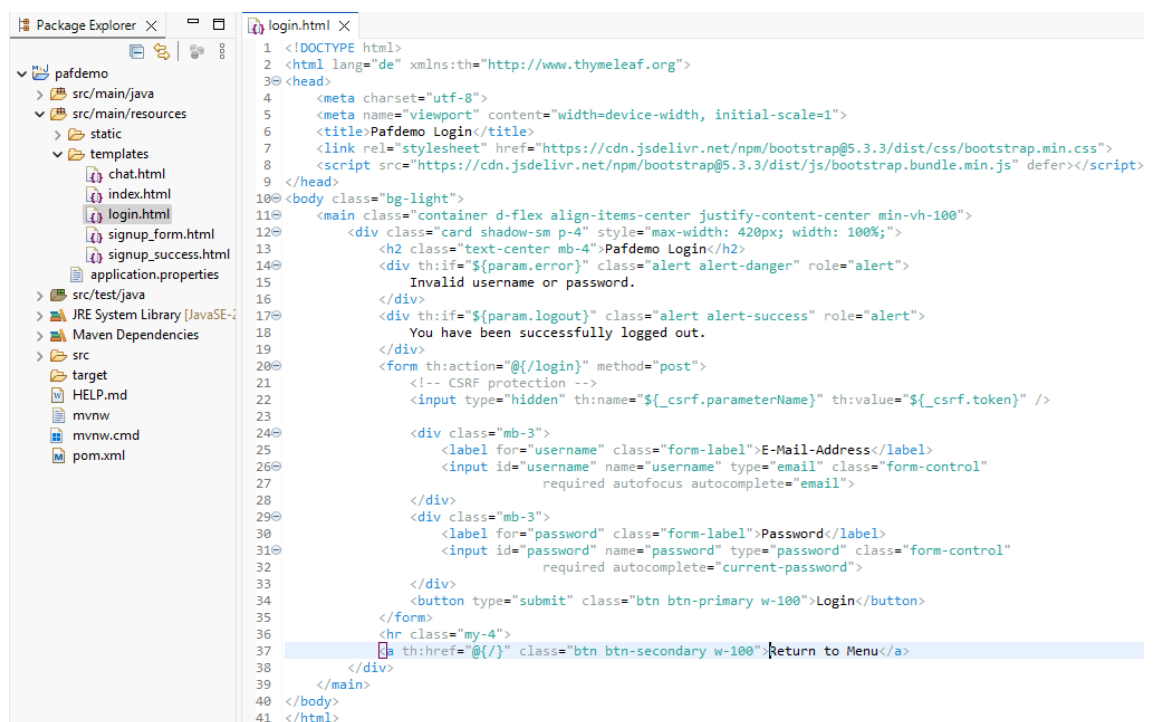


Abb. 3.16: Login-Dialog in HTML

Logout

Um auch ein explizites Logout zu spezifizieren, ergänzen wir in der `SecurityFilterChain` außerdem folgenden Aufruf:



Code

```
.logout(logout -> logout
    .logoutUrl("/logout")
    .logoutSuccessUrl("/login?logout") //redirect to login form
    .invalidateHttpSession(true)
    .clearAuthentication(true)
    .deleteCookies("JSESSIONID")
);
```

Der von Spring Security vordefinierte Endpunkt `/logout` sollte hierbei per POST-Request aufgerufen werden. Einen Logout-Button mit dieser Aktion können wir z. B. in unsere Startseite einbauen:



Code

```
<form th:action="@{/logout}" method="post">
  <input type="hidden" th:name="${_csrf.parameterName}" th:value="${_csrf.token}">
  <button type="submit" class="btn btn-outline-secondary btn-lg d-block w-100">Logout</button>
</form>
```

Erläuterungen zum `_csrf`-Objekt finden Sie im Abschnitt [Schutz vor CSRF implementieren](#).

4 WebSockets zur bidirektionalen Client-Server-Kommunikation

4 WebSockets zur bidirektionalen Client-Server-Kommunikation

4.1 Serverseitig WebSockets bereitstellen

4.2 Clientseitig mit WebSockets arbeiten

Während der Server bei einem REST-API lediglich Anfragen von Clients verarbeitet und beantwortet, kann er bei einem WebSocket-API auch aktiv Nachrichten an Clients senden. Das bedeutet also, Clients bekommen u. U. Nachrichten vom Server, ohne zuvor eine Anfrage gestellt zu haben. Dieser Mechanismus ist in unserem Projektkontext an mehreren Stellen sehr hilfreich. So kann z. B. ein zum Spielen bereiter User darüber informiert werden, dass sich ein Mitspieler gefunden hat. Während des Spielablaufs können WebSocket-Nachrichten dazu genutzt werden, Clients darüber zu informieren, dass sich der Spielzustand geändert hat, weil ein Spieler eine spielspezifische Aktion ausgeführt hat (Spielfigur bewegt, Karte ausgespielt usw.).

Wenn WebSockets im Spring-Kontext eingesetzt werden, geschieht dies oft mittels STOMP (Simple Text Oriented Messaging Protocol. STOMP setzt auf einem bidirektionalen Netzwerkprotokoll wie WebSocket auf und vereinfacht dessen Handhabung durch einfache Befehle wie subscribe, publish, (de-)activate usw. Für nähere Informationen sei auf die [Spring-Dokumentation zu WebSockets](#) und [zu STOMP](#) verwiesen.

4.1 Serverseitig WebSockets bereitstellen

Analog zur Klasse WebSecurityConfig für Spring Security benötigen wir auch für WebSockets eine Klasse, mit der wir die Details des geplanten Nachrichtenaustauschs vorkonfigurieren. Diese von uns WebSocketConfig genannte Klasse ist wiederum mit @Configuration annotiert, um sie für Spring Boot als Konfigurationsklasse zu markieren. Zusätzlich ist die Klasse mit @EnableWebSocketMessageBroker annotiert, um die WebSocket-Infrastruktur von Spring Boot zu aktivieren. Die Klasse implementiert zwei Methoden des Interface WebSocketMessageBrokerConfigurer.

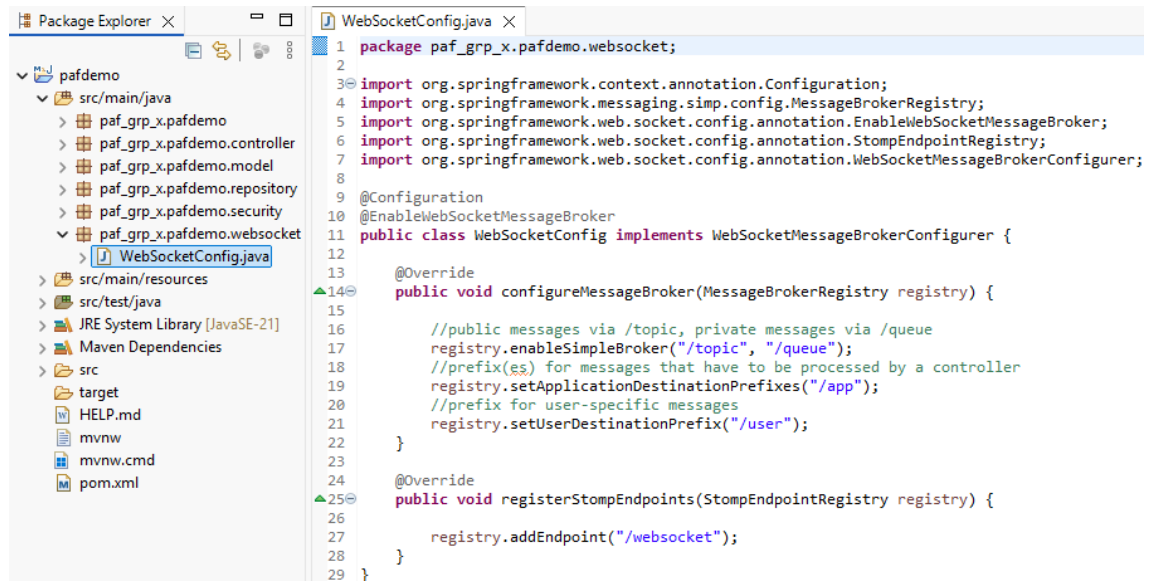


Abb. 4.1: Klasse WebSocketConfig

Die Methode `configureMessageBroker()` konfiguriert eine Komponente (Broker), die Nachrichten von einem oder mehreren Publishern entgegennimmt und diese an einen oder mehrere Subscriber weiterleitet. Als Publisher und Subscriber können die Clients oder auch der Application Server selbst auftreten. Dabei aktiviert die Anweisung `configureMessageBroker("/topic", "/queue")` einen Nachrichtenbroker, der empfangene Nachrichten an Ziele mit dem Präfix `"/topic"` bzw. `"/queue"` weiterleitet. Per Konvention werden Broadcasts über `"/topic"` versendet und private Nachrichten an einzelne User über `"/queue"`.

Ein solcher Nachrichten-Broker wird in der Regel für die Publish-Subscribe-Kommunikation verwendet, bei der mehrere Clients über ein gemeinsames Thema (Topic) informiert werden sollen. Beispiel: Wenn ein Client eine Nachricht an `"/topic/messages"` sendet, werden alle Clients, die dieses Thema abonniert haben, diese Nachricht erhalten.

Die Anweisung `config.setApplicationDestinationPrefixes("/app")` legt fest, dass alle Nachrichten, die vom Client an den Server gesendet werden und die Verarbeitung durch einen Controller erfordern, mit dem Präfix `"/app"` beginnen müssen. Wenn ein Client also z. B. eine Nachricht an `"/app/message"` sendet, wird diese Nachricht zur weiteren Verarbeitung an einen entsprechenden Controller-Handler auf Serverseite weitergeleitet.

Über den mit `setUserDestinationPrefix()` festgelegten Präfix (hier `"/user"`) können User individuelle, auf ihre eigene Session bezogene Nachrichtenkanäle abonnieren.

Durch die verschiedenen Präfix-Typen können Nachrichten, die an den Broker gehen (`"/topic"`, `"/queue"`), von Nachrichten, die vom Server verarbeitet werden sollen (`"/app"`), unterschieden werden.

Die zweite Methode, die die Klasse `WebSocketConfig` überschreibt, ist `registerStompEndpoints()`. Sie macht dem Spring-Framework einen Endpunkt (hier: `"/websocket"`) bekannt, über den Clients eine STOMP-Verbindung zum Server herstellen können, um darüber Nachrichten zu senden und zu empfangen. Beispiel: Wenn der Server auf `http://localhost:8080` läuft, können sich STOMP-Clients über `ws://localhost:8080/websocket` mit dem Server verbinden. Um Verbindungen von anderen Domains zu erlauben, z. B. bei Verwendung eines Frontend-Framework wie `Vue.js`, können die zugelassenen Domains spezifiziert werden. Optional kann der Endpunkt die `SockJS`-Bibliothek verwenden, um `WebSocket`-Kommunikation auch dann zu unterstützen, wenn `WebSockets` nicht nativ verfügbar sind, z. B. bei älteren Browsern oder bestimmten Netzwerkumgebungen, die `WebSockets` blockieren:



Code

```
registry.addEndpoint("/websocket")
    .setAllowedOrigins("http://localhost:8081")
    .withSockJS();
```

Neben der oben beschriebenen `WebSocket`-Konfiguration müssen wir serverseitig noch den oben bereits erwähnten Controller bereitstellen, der an den Endpunkt `"/app"` gerichtete Nachrichten empfängt und verarbeitet. Da wir in unserem Demo-Projekt eine minimalistische Chat-Applikation realisieren wollen, nennen wir die Klasse `ChatController`. Wie am folgenden Quellcode zu erkennen ist, ähnelt der `WebSocketController` einem `REST-Controller`. Der wesentliche Unterschied sind die abweichenden Annotationen. Mit `@MessageMapping("/chat")` legen wir fest, dass der folgende Endpunkt für Nachrichten an `"/app/chat"` gelten soll. Die zugehörige Methode `sendChatMessage()` nimmt die empfangene Nachricht vom Typ `ChatMessage` (siehe unten) entgegen, liest aus dem Objekt den Wert des `content`-Attributs aus, stellt den `Username` (hier: die E-Mail-Adresse) des sog. `Principal` (= User, der die Nachricht gesendet hat) vor den bisherigen `content` und gibt die modifizierte `ChatMessage` zurück. Diese wird an alle `Subscriber` von `"/topic/messages"` gesendet, wie die vor der Methode stehende Annotation `@SendTo("/topic/messages")` spezifiziert.

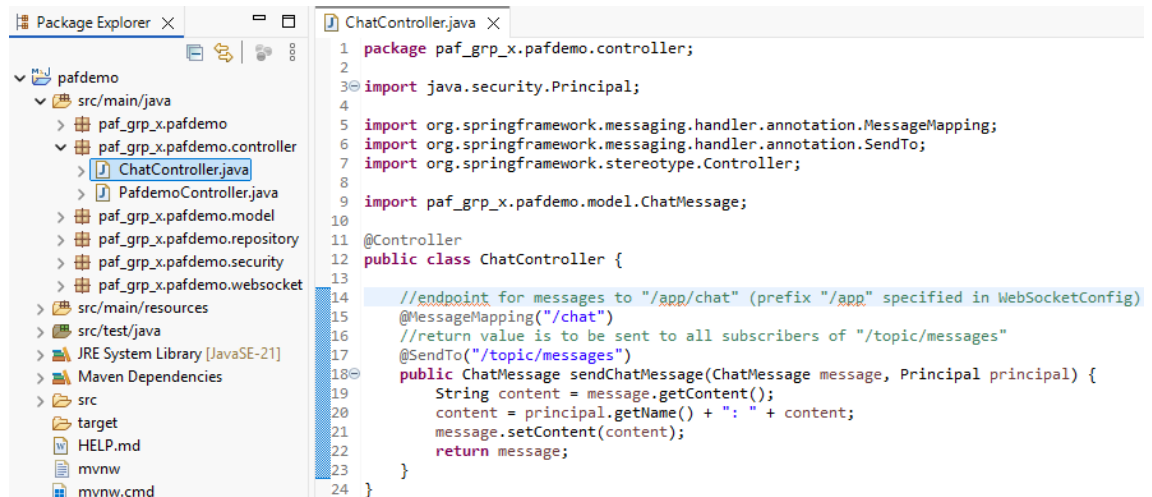


Abb. 4.2: Klasse ChatController



Hinweis

Das Application Destination Prefix `"/app"` (siehe Konfiguration oben) gilt global für alle `@MessageMapping`-Handler, egal in welcher Controller-Klasse sie stehen.



Hinweis

In unserer Demo-Anwendung haben wir es nur mit statischen WebSocket-Endpunkten wie `/chat` zu tun. In anderen Anwendungen mag es aber auch Endpunkte mit dynamischen Anteilen geben, z. B. einer User-ID oder einer Game-ID. Mit einer Game-ID im Endpunkt-Namen könnte man zum Beispiel sicherstellen, dass ausschließlich diejenigen User, die an dem Spiel mit dieser Game-ID beteiligt sind, eine Nachricht erhalten - sofern sie den Game-spezifischen Kanal zuvor abonniert haben.

Auf solche dynamischen Anteile können wir über die Annotation `@DestinationVariable` Bezug nehmen, wie in folgendem Beispiel gezeigt. Bitte beachten Sie, dass `@MessageMapping` den Platzhalter `"{gameId}"` enthält. Dieser wird über `@DestinationVariable` in die Handler-Methode `makeMove()` eingespeist und kann somit innerhalb der Methode verwendet werden. `@SendTo` macht ebenfalls Gebrauch von dem Platzhalter `"{gameId}"`, um sicherzustellen, dass nur die Clients, die auf genau diese Game-ID "lauschen", über den Spielzug benachrichtigt werden.

```
@MessageMapping("/{gameId}/move")
@SendTo("/topic/game-progress/{gameId}") //send response to clients that subscribed this topic
public GameDto makeMove(@Payload MoveDto moveDto, @DestinationVariable("gameId")
Long gameId) {
    //...
}
```

Dabei ist es wichtig, dass der Variablenname des Platzhalters als Parameter von `@DestinationVariable` explizit benannt wird:

```
@DestinationVariable("gameId") Long gameId
```

Ohne diesen Parameter wird der Name der Destination Variable per Default als `arg0` angenommen, was zu einer Fehlermeldung der Art *"Name for argument of type [java.lang.Long] not specified"* führt. Es reicht nicht aus, dass der Long-Parameter von `makeMove()` den Bezeichner *gameId* hat. Theoretisch können Destination Variable und Methodenparameter unterschiedlich benannt werden; eine Namensgleichheit stellt keinen semantischen Zusammenhang her.

Verwendung von `SimpMessagingTemplate`

REST und WebSockets können auch kombiniert werden, z. B. wie folgt: Ein Request geht auf einem REST-Endpunkt ein. Der zugehörige Request Handler beantwortet den Request, sendet aber zuvor noch eine oder mehrere WebSocket-Nachrichten an alle Clients, die bestimmte Kanäle abonniert haben. Bei der Implementierung kann man sich das `SimpMessagingTemplate` aus dem Spring-Framework zunutze machen:

Per Auto-Injektion wird das entsprechende Objekt initialisiert:



```
@Autowired
private final SimpMessagingTemplate simpMessagingTemplate;
```

Später kann dann dessen Methode `convertAndSend()` aufgerufen werden:



```
simpMessagingTemplate.convertAndSend("/topic/mychannel/", mymessage);
```

Mittels `convertAndSendToUser()` können auch gezielt Nachrichten an einzelne Clients gesendet werden:



```
simpMessagingTemplate.convertAndSendToUser(user, "/queue/mychannel", mymessage);
```

Die Verwendung von `simpMessagingTemplate` bietet sich auch dann an, wenn ein WebSocket-Endpoint *mehr als einen* Nachrichtenversand auslösen soll. (Die Annotation `@SendTo` kann nur einmal pro Endpunkt spezifiziert werden, da sie mit dem return-Wert der Handler-Methode verknüpft ist.)

Der Vollständigkeit halber hier nun noch der aufgrund der Lombok-Nutzung sehr kompakte Quellcode der Klasse `ChatMessage` aus dem `model`-Package:

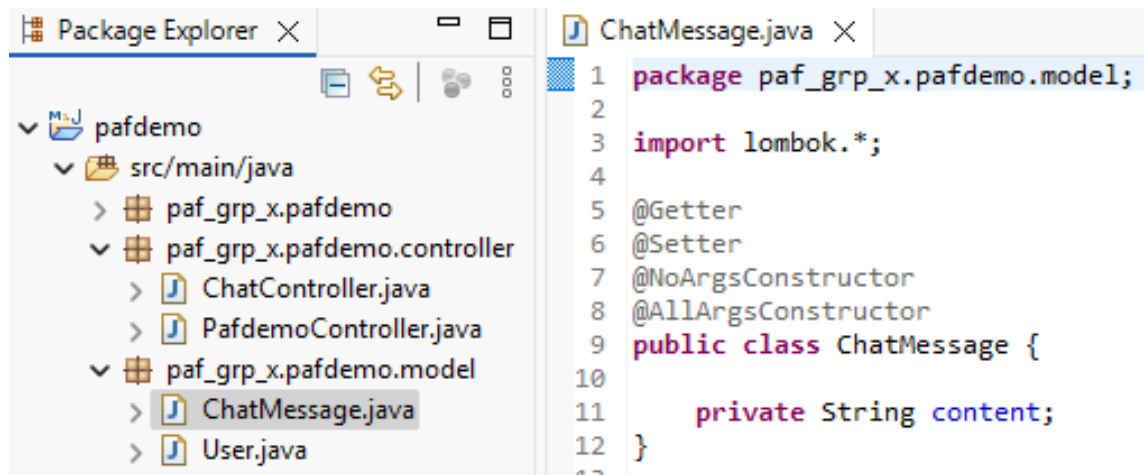


Abb. 4.3: Klasse ChatMessage

Sofern gewünscht (im Demo-Projekt nicht erforderlich), können wir serverseitig auf einen Verbindungsaufbau durch einen Client reagieren, indem wir einen entsprechenden Event-Listener definieren, zum Beispiel:



Code

```
@EventListener
public void handleWebSocketConnect(SessionConnectedEvent event) {
    //perform actions or log when a client connects via WebSocket
    //...
}
```

Wenn Sie sich zu diesem Zeitpunkt noch nicht so richtig vorstellen können, wie die WebSocket-Kommunikation letztlich funktioniert, warten Sie bitte die Erläuterungen zur Client-Seite ab - dann wird das Zusammenspiel der Komponenten hoffentlich deutlicher.

4.2 Clientseitig mit WebSockets arbeiten

Um unsere auf WebSockets basierende Chat-Funktion aus dem Browser heraus aufrufen zu können, erweitern wir unser im Thymeleaf-Template index.html enthaltenes Menü um einen weiteren Eintrag:



Code

```
<a th:href="@{/chat}" class="btn btn-info btn-lg d-block w-100">Chat</a>
```

Da die Chat-Funktion nur authentifizierten Personen zugänglich sein soll, nehmen wir `/chat` in die Liste der geschützten Endpunkte auf (Klasse `WebSecurityConfig`, Methode `securityFilterChain`):



Code

```
http
    .authorizeHttpRequests(auth -> auth
        .requestMatchers("/hello", "/chat").authenticated()
        .anyRequest().permitAll()
    )
```

Im PafdemoController ergänzen wir den REST-Endpoint `/chat`:



```
@GetMapping("/chat")
public String chat() {
    return "chat";
}
```

Den wesentlichen Teil des hier retournierten Thymeleaf-Template zeigt die folgende Abbildung. Neben Bootstrap (im Screenshot nicht zu sehen) werden außerdem die JavaScript-Module StompJS und chat.js eingebunden. Letzteres implementiert unseren Chat-Client auf Basis von StompJS und wird weiter unten auf dieser Seite erläutert. Der main-Bereich des Template enthält Abschnitte für den Verbindungsauf- und -abbau, für die Eingabe von Chat-Nachrichten sowie für die Anzeige des Nachrichtenverlaufs.

```

19<script
20  src="https://cdn.jsdelivr.net/npm/@stomp/stompjs@7.1.1/bundles/stomp.umd.min.js"
21  defer
22></script>
23<script src="/chat.js" type="module" defer></script>
24</head>
25<body>
26  <main id="main-content" class="container py-4">
27    <h1 class="h3 mb-4">PaF Demo Chat</h1>
28    <div class="row g-3 align-items-end">
29      <!-- connection -->
30      <section class="col-md-6">
31        <div class="d-flex gap-2 align-items-center">
32          <label class="form-label mb-0" for="connectBtn">WebSocket:</label>
33          <button id="connectBtn" type="button" class="btn btn-primary">Connect</button>
34          <button id="disconnectBtn" type="button" class="btn btn-outline-secondary">Disconnect</button>
35        </div>
36      </section>
37
38      <!-- send message -->
39      <section class="col-md-6">
40        <div class="input-group">
41          <label class="input-group-text" for="message">Message</label>
42          <input
43            id="messageInput"
44            type="text"
45            class="form-control"
46            placeholder="Your message here..."
47            autocomplete="off"
48            enterkeyhint="send"
49          />
50          <button id="sendBtn" type="button" class="btn btn-success" disabled>Send</button>
51        </div>
52      </section>
53    </div>
54
55    <!-- messages list -->
56    <section class="row mt-4">
57      <div class="col-12">
58        <div id="conversation" class="card" hidden>
59          <div class="card-header">Messages</div>
60          <div class="card-body p-0">
61            <ul
62              id="messagesList"
63              class="list-group list-group-flush overflow-auto"
64              style="max-height: 50vh;"
65              role="log"
66            >
67            </ul>
68          </div>
69        </div>
70      </section>
71    </main>
72  </body>
73</html>

```

Abb. 4.4: Thymeleaf-Template chat.html

In dem eingebundenen Modul chat.js wird zunächst ein StompJS-Client erzeugt und mit der URL `ws://localhost:8080/websocket` verknüpft (vgl. `WebSocketConfig.registerStompEndpoints()`). Anschließend wird ein `onConnect`-Handler spezifiziert, der u. A. den Nachrichtenkanal `"/topic/messages"` abonniert (vgl. `ChatController.sendChatMessage()`). Der zu `subscribe()` gehörige Event Handler ruft bei Eingang einer Nachricht auf dem abonnierten Kanal `showMessage()` auf, was die empfangene Nachricht als neues Listenelement der bisherigen Nachrichtenliste hinzufügt.



Hinweis

Subscriptions müssen nicht zwingend bereits beim Verbindungsaufbau vorgenommen werden. Je nach Spielkonzept mag es sinnvoll sein, bei Bedarf im weiteren Programmverlauf gezielt `subscribe()` oder `unsubscribe()` für bestimmte Nachrichtenkanäle aufzurufen.

Zwei weitere Event-Handler übernehmen die Fehlerbehandlung. `setConnected` (aufgerufen vom `onConnect`-Handler und von `disconnect()`) steuert die (In-)Aktivität von Buttons bzw. (Un-)Sichtbarkeit der Nachrichtenliste abhängig vom Verbindungszustand.

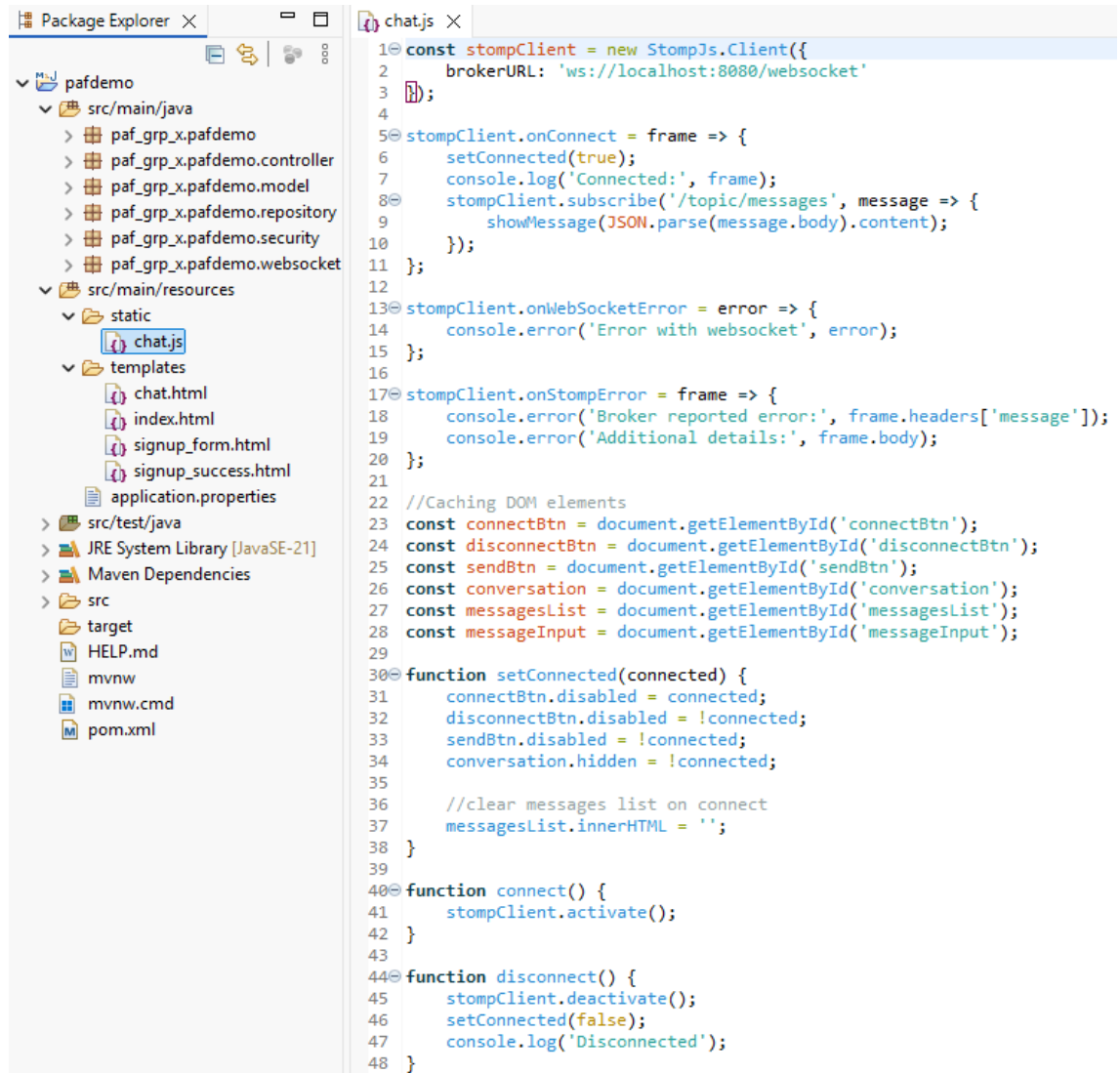


Abb. 4.5: Modul chat.js, Teil 1

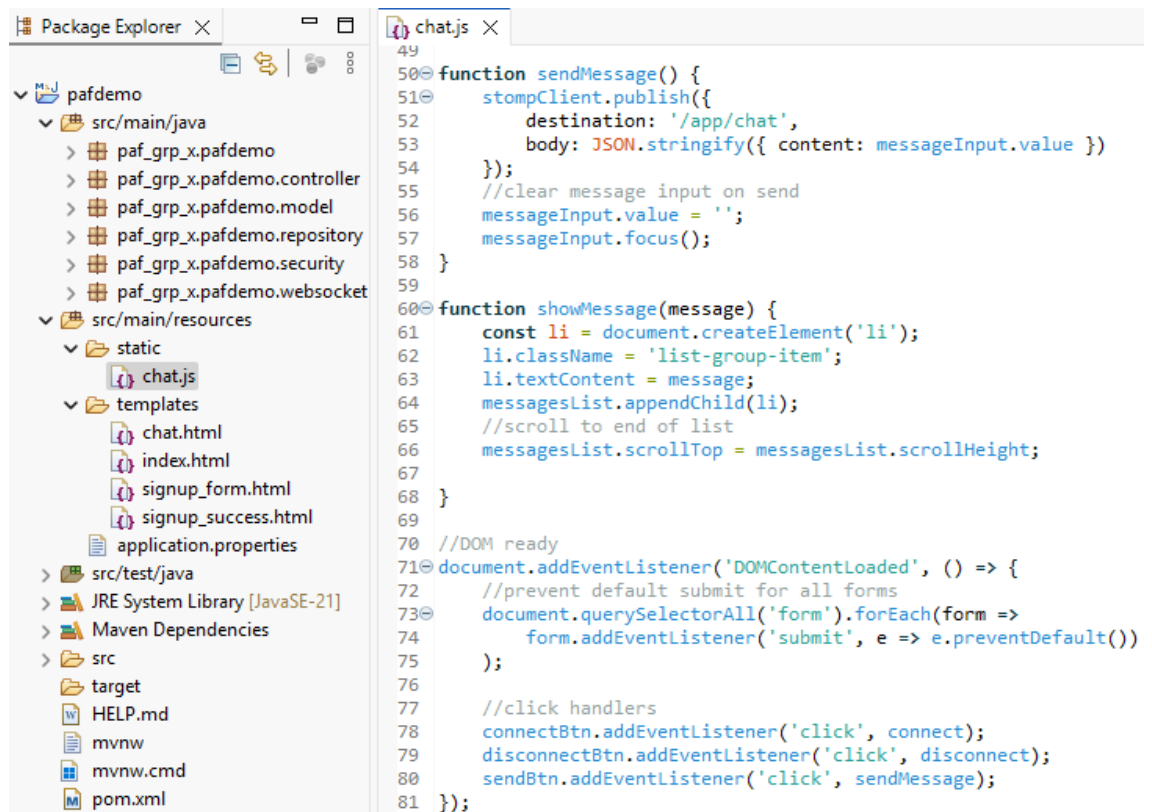


Abb. 4.6: Modul chat.js, Teil 2

`connect()`, `disconnect()` und `sendMessage()` implementieren die jeweiligen Button-Aktionen. Bei `sendMessage` beinhaltet diese insb. das Publizieren der eingegebenen Nachricht mit der Destination `"/app/chat"`. Diese korrespondiert mit dem Endpunkt, der im `ChatController` mit `@MessageMapping("/chat")` annotiert wurde.

Am Ende des Moduls wird ein Listener für das Ereignis `"DOMContentLoaded"` spezifiziert. Dieser wird also unmittelbar nach Abschluss des Ladevorgangs der Seite ausgeführt. Dies ist erforderlich, um die Submit-Aktionen aller enthaltenen form-Elemente - abweichend vom Default-Verhalten - so zu modifizieren, dass die bereits beschriebenen Funktionen `connect()`, `disconnect()` bzw. `sendMessage()` aufgerufen werden.



Hinweis

In Ihrem Projekt mag es sinnvoll sein, auf einen Connect-Button zu verzichten und stattdessen den Aufruf `stompClient.activate()` direkt in den Event-Listener für `"DOMContentLoaded"` zu verlagern. Damit erreichen Sie, dass die Client-Anwendung automatisch nach dem Laden der Seite eine StompJS-Verbindung zum Server aufbaut, anstatt dass der User den Verbindungsaufbau explizit über einen Button-Click veranlassen muss.

Insgesamt stellt sich unsere Chat-Anwendung im Browser so dar wie im folgenden Screenshot gezeigt. Bemerkenswert ist, dass selbst der Client, der eine Nachricht gesendet hat, die eigene Nachricht über die Subscription empfängt und sie erst dadurch in der eigenen Nachrichtenliste erscheint.

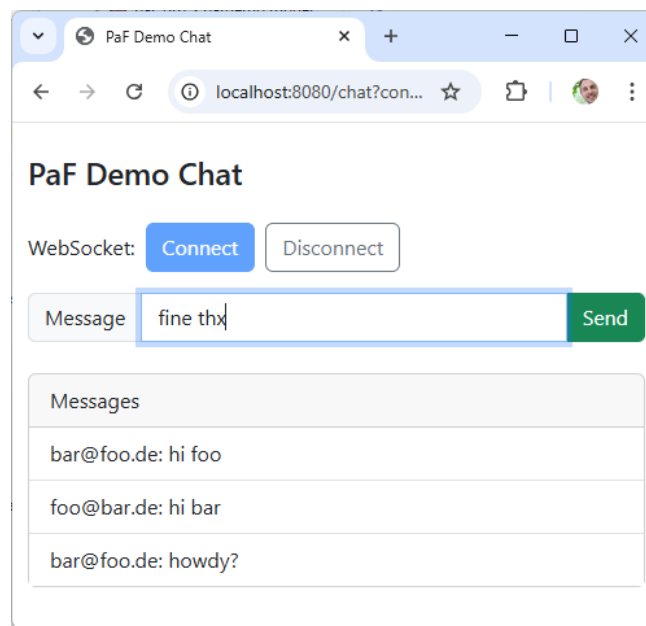


Abb. 4.7: Chat im Browser



5 JWT verwenden

5 JWT verwenden

5.1 Serverseitig JWT implementieren

5.2 Clientseitig JWT implementieren

5.3 Fortgeschrittenes JWT-Handling

Alternativ zur sessionbasierten Authentifizierung, die wir in einem vorigen Abschnitt kennengelernt haben, lässt sich auch eine zustandslose (stateless) Authentifizierung realisieren. Diese arbeitet mit sog. JSON Web Tokens (JWT). Das zugrundeliegende Konzept besteht darin, dass der Server, nachdem der User seine korrekten Credentials eingegeben hat, ein (temporär gültiges) JWT erzeugt und an den Client sendet. Dieser speichert das JWT lokal und sendet es mit jedem weiteren Request mit. Der Server kann erkennen, ob er selbst das JWT erzeugt hat und ob es noch gültig ist. Falls dies der Fall ist, wird der Request akzeptiert, ansonsten abgewiesen. Eine serverseitige Verwaltung von Sessions entfällt somit. Dieser Ansatz hat insbesondere Vorteile bei horizontaler Skalierung auf Serverseite.

Um unsere Anwendung JWT-fähig zu machen, müssen wir an verschiedenen Stellen, sowohl server- als auch clientseitig, Anpassungen vornehmen. Zu Demonstrationszwecken werden wir die bisherige sessionbasierte Authentifizierung beibehalten und die JWT-Lösung parallel dazu installieren. In der Praxis (und in Ihren Projekten) würde man sich natürlich exklusiv für einen der beiden Ansätze entscheiden.

5.1 Serverseitig JWT implementieren

Beginnen wir also mit den serverseitigen Anpassungen für eine JWT-Authentifizierung.

Dependencies für JWT in pom.xml ergänzen

Voraussetzung für den serverseitigen Einsatz von JWT ist, dass wir die Bibliothek JJWT (mit Group-ID io.jsonwebtoken) in unser Maven-Projekt einbinden. Hierzu ergänzen wir in der pom.xml die Abhängigkeiten zu den drei Artefakten jjwt-api, jjwt-impl und jjwt-jackson:



Code

```
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-api</artifactId>
  <version>0.13.0</version>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-impl</artifactId>
```

```
<version>0.13.0</version>
<scope>runtime</scope>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-jackson</artifactId>
  <version>0.13.0</version>
  <scope>runtime</scope>
</dependency>
```

Wichtig ist dabei, dass die drei dependency-Elemente *innerhalb* des dependencies-Elements eingefügt werden, zum Beispiel unmittelbar vor dem schließenden Tag </dependencies>. Die Versionsnummern können Sie anpassen, sofern es im Maven Repository inzwischen aktuellere Versionen gibt.

Klasse JwtTokenService

Um für einen User, der sich zuvor mit gültigen Credentials authentifiziert hat, ein JWT erzeugen zu können, benötigen wir eine Service-Klasse, die wir JwtTokenService nennen. Weitere Aufgaben dieser Klasse sind das Prüfen eines gegebenen JWT auf Gültigkeit und die Extraktion verschiedener Informationen (z. B. Username) aus einem JWT. Der Einfachheit halber unterstützt unser JwtTokenService keine Rollen (Authorities), dies könnte bei Bedarf aber nachgerüstet werden.

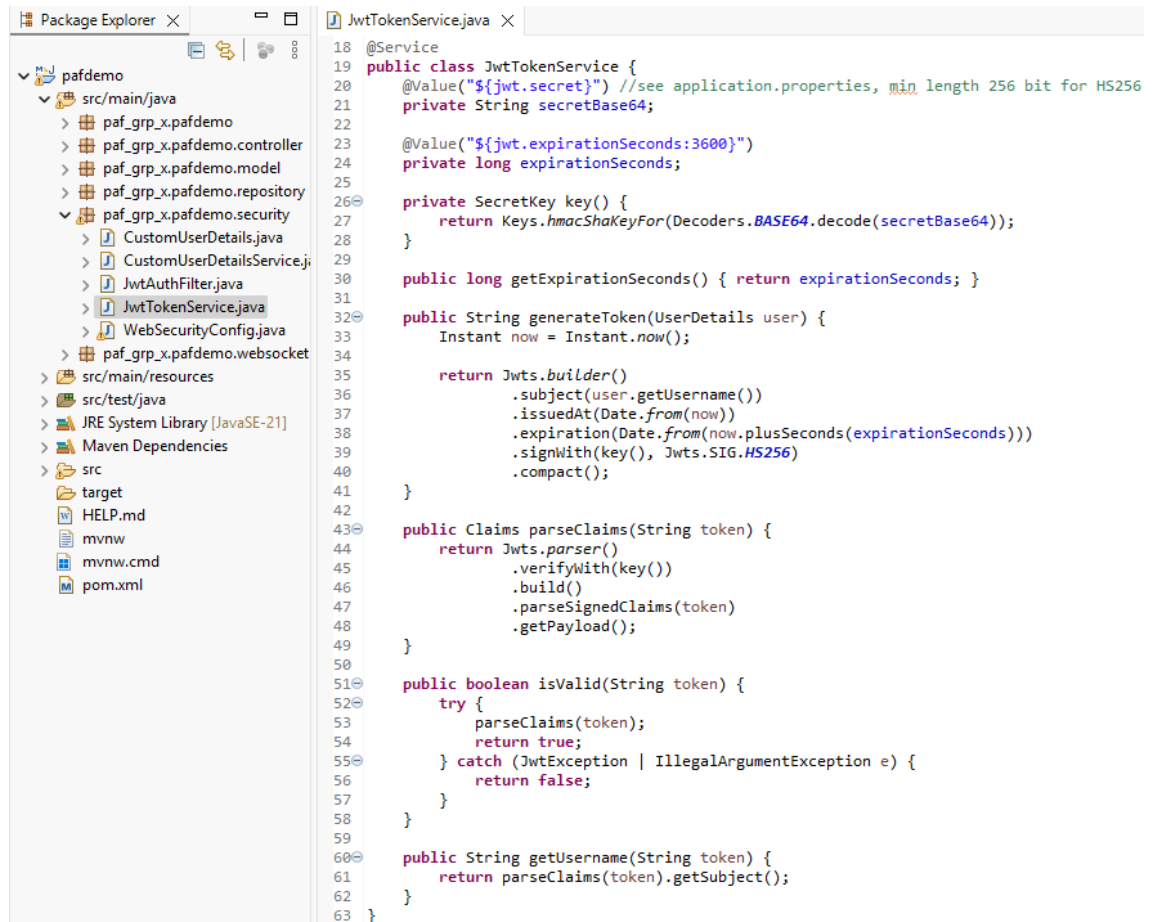


Abb. 5.1: Klasse JwtTokenService

In jedem generierten JWT wird ein serverseitiges Geheimnis verbaut, das wir in unserer Demo-Umgebung in `application.properties` spezifizieren und über eine `@Value`-Annotation dem Token-Service bekannt machen. Gleiches gilt auch für die Gültigkeitsdauer eines JWT (hier 3.600s = 1h).



Code

```

jwt.secret=myLongSigningSecretOfAtLeast32Base64Characters
jwt.expirationSeconds=3600

```

Klasse JwtAuthController

Als Alternative zum bisherigen Formular-Login erstellen wir nun einen API-Controller mit einem POST-Endpunkt `/api/auth/login`. JWT-Login-Requests werden auf diesen Endpunkt geführt, der zunächst eine Authentifizierung mittels E-Mail-Adresse und Passwort versucht. Im Erfolgsfall wird anhand der User-Details ein JWT erzeugt und (hier zusammen mit der Gültigkeitsdauer) in den Response-Body gelegt.

In den Konstruktor `JwtAuthController()` werden Objekte der Klassen `AuthenticationManager` und `JwtTokenService` injiziert, da diese im Login-Prozess zu den o. g. Zwecken benötigt werden.

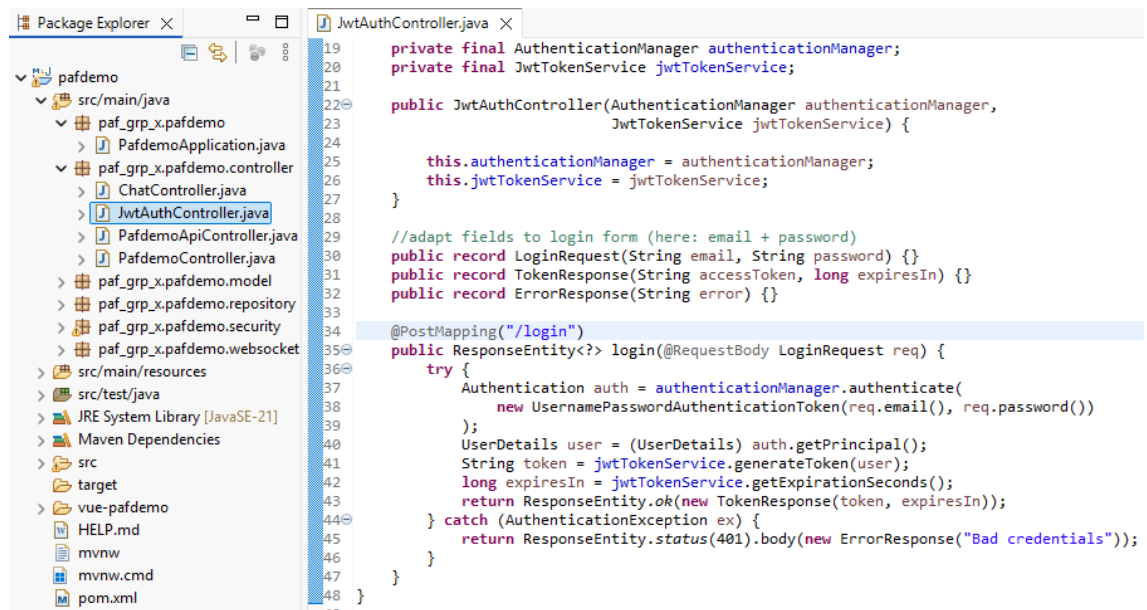


Abb. 5.2: Klasse JwtAuthController

Klasse JwtAuthFilter

Nach der Authentifizierung per Credentials und der folgenden Erzeugung und Auslieferung des JWT muss der Client bei jedem folgenden Request, der auf eine geschützte Ressource abzielt, den JWT mitsenden. Der Server muss also den Request vor der Beantwortung auf Vorhandensein und Gültigkeit des JWT untersuchen. Dies implementieren wir mit einer Filterklasse, die die Prüfung übernimmt und im Erfolgsfall die Authentifizierung im sog. Security Context setzt. JwtAuthFilter wurde hierfür von der Spring-Klasse OncePerRequest-Filter abgeleitet und überschreibt deren Methode `doFilterInternal()`. Das JWT wird aus dem Authorization-Header des Request gelesen. Nachdem der Standard-Präfix "Bearer " entfernt wurde, wird das Token geparkt, der Username extrahiert und der Authentifizierungsversuch unternommen. Für den Fall, dass das Token ungültig oder abgelaufen ist, wird der catch-Block angesprungen, d. h. es wird keine Authentifizierung gesetzt, was in der Folge eine Abweisung des Requests führen wird, sofern die angefragte Ressource geschützt ist.

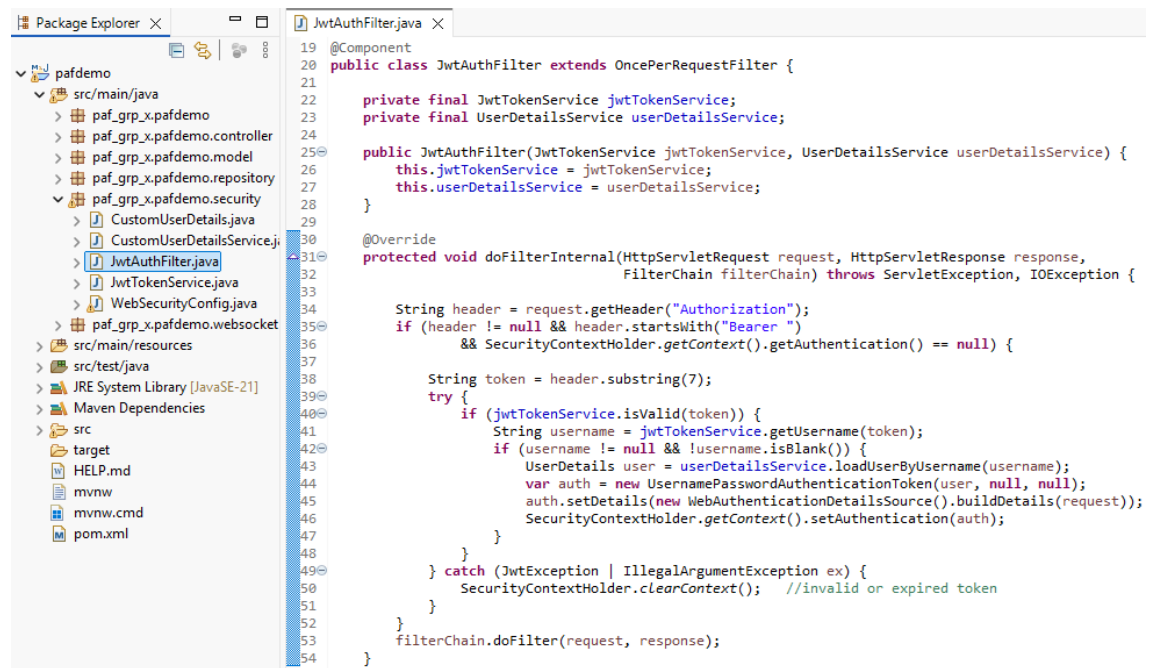


Abb. 5.3: Klasse JwtauthFilter

Anpassung der Klasse WebSecurityConfig

Unsere Security Configuration müssen wir für den JWT-Support in verschiedener Hinsicht anpassen:

Die sog. Session Creation Policy stellen wir auf "stateless" um, d. h. der Server wird für einen authentifizierten User keine Session mehr anlegen und insofern zustandslos arbeiten. Der Server wird also nicht mehr "wissen", welche User gerade authentifiziert sind. Jeder eingehende Request wird auf die Gültigkeit des mitgesendeten JWT geprüft und daraufhin authentifiziert oder nicht.

Den oben definierten JwtauthFilter binden wir mittels `addFilterBefore()` in die Filterkette ein. Der CSRF-Schutz deaktivieren wir, da er für JWT-basierte Zugriffe nicht relevant ist. Das Formular-Login und das Default-Logout deaktivieren wir, da beides bei JWT clientseitig gehandhabt wird. Weil wir im JwtauthController ein Objekt vom Typ `AuthenticationManager` benötigen, definieren wir in `WebSecurityConfig` eine entsprechende Bean-Methode.

Da wir, wie oben beschrieben, die bisherige, sessionbasierte Authentifizierungsvariante zu Demozwecken beibehalten wollen, differenzieren wir die beiden Filterketten mittels der `@Order`-Notation. Die mit `@Order(1)` annotierte `apiChain()` gilt für alle Zugriffe auf `/api/**`, also insb. auf die vom JwtauthController (siehe oben) verarbeiteten Zugriffe. Alle anderen Zugriffe werden von der mit `@Order(2)` annotierten `webChain()` verarbeitet.

Für detailliertere Informationen zu Filtern sei auf die [Spring-Dokumentation](#) verwiesen.



Abb. 5.4: Klasse WebSecurityConfig mit zweiter Filterkette

Für die geschützten Endpunkte `/hello` und `/chat` unserer Anwendung stellen wir in Ergänzung zu unserem `PafdemoController` einen separaten `PafdemoApiController` bereit, der als `@RequestMapping "/api"` hinterlegt hat und somit auf JWT-basierte Requests reagiert.

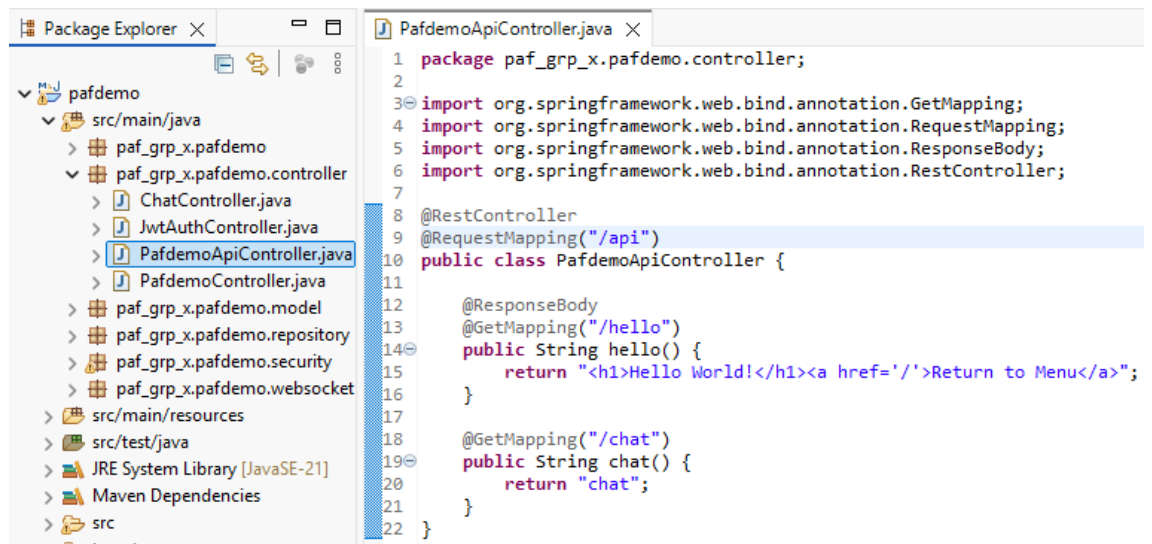


Abb. 5.5: Klasse PafdemoApiController

Klasse JwtStompChannelInterceptor

Da unsere Chat-Funktion, die über den `/chat`-Endpunkt aufgerufen wird, mit WebSockets bzw. STOMP arbeitet, wollen wir auch die STOMP-Nachricht zum Verbindungsaufbau (CONNECT) mit dem JWT anreichern (siehe [clientseitige JWT-Implementie-](#)

rung). Analog zum `JwtAuthFilter` greifen wir also in die Verarbeitung des STOMP-Handshake ein, um den JWT auf Vorhandensein und Gültigkeit zu prüfen. Hierzu implementieren wir die Klasse `JwtStompChannelInterceptor`, die das Spring-Interface `ChannelInterceptor` implementiert. Wir injizieren unseren `JwtTokenService` sowie den `UserDetailsService` über den Konstruktor und überschreiben die Methode `preSend()`. Diese prüft, ob es sich um eine CONNECT-Nachricht handelt, liest im Positivfall das JWT aus und prüft es auf Gültigkeit. In der Folge darf die Nachricht entweder passieren oder wird durch eine Exception geblockt.



Abb. 5.6: Klasse `JwtStompChannelInterceptor`

Den `JwtStompChannelInterceptor` injizieren wir per Konstruktor in unsere Klasse `WebSocketConfig`. Anschließend registrieren wir ihn in der Methode `configureClientInboundChannel()`. Dies entspricht in analoger Weise der Einbindung des `JwtAuthFilter` in die Security Chain (siehe oben) - dort für REST-Requests, hier für STOMP-Nachrichten.



Code

```
private final JwtStompChannelInterceptor jwtInterceptor;

public WebSocketConfig(JwtStompChannelInterceptor jwtInterceptor) {
    this.jwtInterceptor = jwtInterceptor;
}

@Override
public void configureClientInboundChannel(ChannelRegistration registration) {
    registration.interceptors(jwtInterceptor); //JWT for STOMP
}
```

5.2 Clientseitig JWT implementieren

Auch clientseitig müssen wir eine Reihe von Anpassungen vornehmen, um mit JWT arbeiten zu können. Dies betrifft zum einen das Login-Formular selbst, zum anderen aber auch die Aufrufe der geschützten Endpunkte und das Logout.

Login-Formular

Da wir zu Demozwecken beide Login-Varianten parallel betreiben wollen, erweitern wir unser bisheriges Login-Formular so, dass wir in einem Tab-Reiter das sessionbasierte Login vornehmen können und in einem separaten Tab-Reiter das JWT- bzw. API-basierte Login. Aufgrund der Länge des HTML-Quellcodes zeigen wir diesen nicht in voller Länge als Screenshot, sondern beschränken uns auf den inline ergänzten JavaScript-Code.

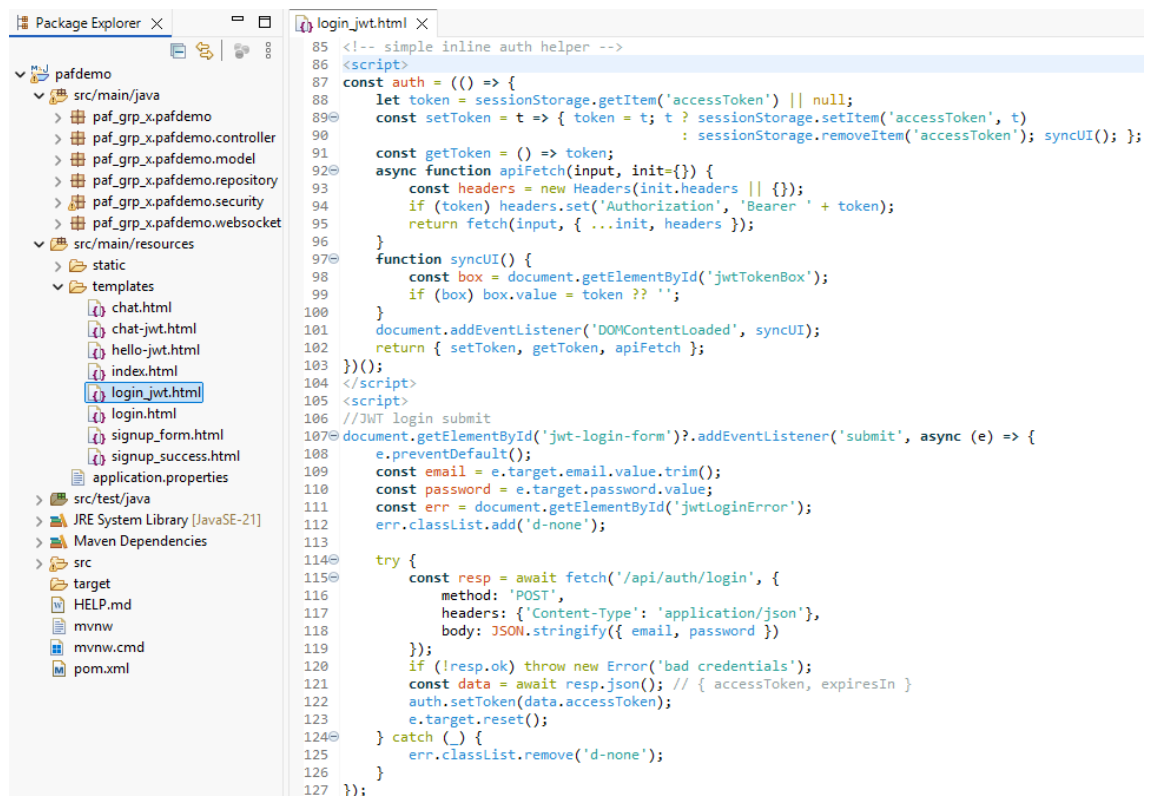


Abb. 5.7: login_jwt.html (Auszug)

Hier erkennen wir, dass in der submit-Aktion des Login-Formulars das Default-Verhalten unterdrückt und durch einen fetch-Aufruf an unseren Endpunkt `"/api/auth/login"` (vgl. Klasse `JwtAuthController`) ersetzt wird. Das im OK-Fall erhaltene Token wird aus der Response ausgelesen und per `auth.setToken()` im Session Storage gesetzt. Dies ist eine einfache, aber nicht die optimale Lösung, da sie z. B. nicht Browser-Tab-übergreifend funktioniert. Alternativen wären eine Speicherung im Local Storage (komfortabler, aber aufgrund der persistenten Speicherung riskanter im Hinblick auf Cross-Site-Scrip-

ting) oder eine Verwendung von HttpOnly-Cookies. In unserem Demo-Kontext bleiben wir beim Session Storage.

Im PafDemoController liefern wir nun für das GET-Mapping auf `"/login"` nicht mehr das bisherige Thymeleaf-Template `"login.html"` zurück, sondern das erweiterte Template `"login_jwt.html"`:



```
@GetMapping("/login")
public String login() {
    //...
    //return "login";    //if not authenticated, show custom login page
    return "login_jwt"; //now use new login page that also supports JWT
}
```

Das Login-Formular mit den beiden Varianten zur Anmeldung stellt sich nun wie folgt dar:

Abb. 5.8: Login-Formular mit JWT-Unterstützung

hello-jwt.html

Unseren geschützten Endpunkt `"/api/hello"` können wir - anders als `"/hello"` bei der Session-Variante - nicht einfach als Get-Request aufrufen, da wir das JWT im Authorization Header mitsenden müssen. Daher stellen wir eine HTML-Datei bereit, die eine anonyme Funktion spezifiziert, die bei Ausführung den Endpunkt `"/api/hello"`

inkl. Token im Header anspricht und im OK-Fall das Ergebnis ("Hello World!") in das div-Element mit der ID "result" einfügt.

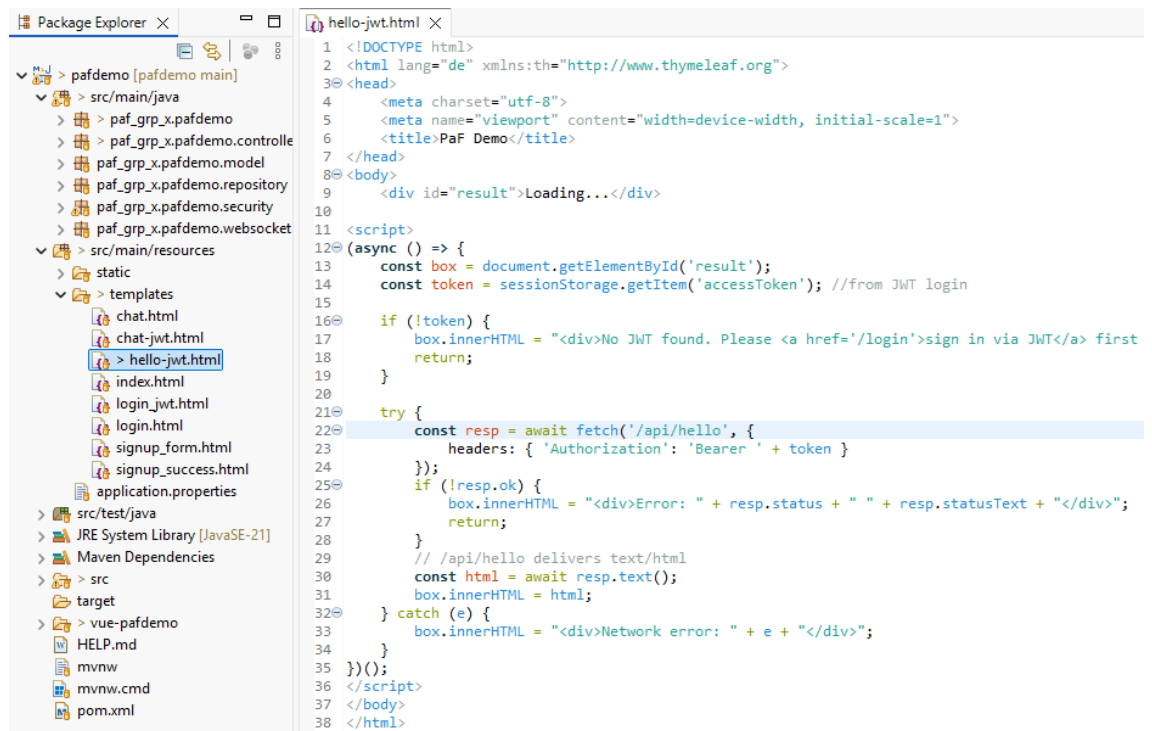


Abb. 5.9: hello-jwt.html

chat-jwt.html und chat-jwt.js

Die HTML-Datei für unsere JWT-basierte Chat-Funktion ist genauso aufgebaut wie unsere Variante für die Session-Authentifizierung (chat.html), abgesehen von einem Hinweisbereich für ein fehlendes JWT sowie die Einbindung eines Moduls chat-jwt.js, das die Nachricht zum STOMP-Verbindungsaufbau um das JWT im Authorization Header anreichert:



Code

```
<head>

  <script src="/chat-jwt.js" type="module" defer></script>
</head>
<body>

  <div id="jwtNotice" class="alert alert-warning d-none" role="alert">
    Kein JWT gefunden. Bitte zuerst über „Login (JWT)“ anmelden.
  </div>

</body>
```

Auch chat-jwt.js ähnelt sehr unserem Modul chat.js, nur dass - wie bereits erläutert - beim Verbindungsaufbau das JWT mitgeschickt wird und an einigen Stellen geprüft wird, ob das JWT vorhanden ist:



Code

```
//read JWT
const jwt = sessionStorage.getItem('accessToken');

//STOMP client with JWT header
const stompClient = new StompJs.Client({
  brokerURL: 'ws://localhost:8080/websocket',
  //set authorization header only if token is available
  connectHeaders: jwt ? { Authorization: 'Bearer ' + jwt } : {}
});

//...

function connect() {
  if (!jwt) {
    console.warn('No JWT found. Please login (JWT) first.');
```

```
const notice = document.getElementById('jwtNotice');
if (notice) notice.classList.remove('d-none');
return;
  }
  stompClient.activate();
}
```

Den Event Listener für DOMContentLoaded erweitern wir so, dass der Connect-Button deaktiviert wird, wenn kein JWT vorhanden ist. So kann man zwar ohne JWT-Authentifizierung die Chat-Seite betreten, aber nicht chatten.



Code

```
if (!jwt) {
  connectBtn.disabled = true;
}
```

Die neuen Thymeleaf-Templates "hello-jwt.html" und "chat-jwt.html" lassen wir von unserem PdfdemoController bei GET-Requests an "/hello-jwt" bzw. "/chat-jwt" ausliefern:



Code

```
@GetMapping("/hello-jwt")
public String helloJwt() {
  return "hello-jwt";
}

@GetMapping("/chat-jwt")
public String chatJwt() {
  return "chat-jwt";
}
```

Die in die Templates eingebundenen fetch-Aufrufe zielen dann, wie beschrieben, auf die Endpunkte `/hello` und `/chat`, die vom `PaFdemoApiController` behandelt werden. Schließlich ist noch zu beachten, dass die o. g. GET-Endpunkte `/hello-jwt` und `/chat-jwt` in der `SecurityFilterChain` (hier `webChain`, nicht `apiChain`!) freigeschaltet werden müssen:



Code

```
http
    // (keep CSRF protection enabled for Thymeleaf forms)
    .authorizeHttpRequests(auth -> auth
        // freely accessible pages
        .requestMatchers("/", "/login", "/signup", "/process_signup",
            "/hello-jwt", "/chat-jwt").permitAll()
        // static files, also freely accessible
        .requestMatchers("/css/**", "/js/**", "/images/**", "/webjars/**",
            "/favicon.ico", "/chat-jwt.js").permitAll()
        // WebSocket handshake endpoint
        .requestMatchers("/websocket/**").permitAll()
        // other pages only within session
        .requestMatchers("/hello", "/chat").authenticated()
        .anyRequest().authenticated()
    )
```

Startseite und Logout

Das Hauptmenü auf der Startseite erweitern wir um Links zu den o. g. GET-Endpunkten `/hello-jwt` und `/chat-jwt` sowie um einen zusätzlichen Logout-Button für die JWT-Authentifizierung:

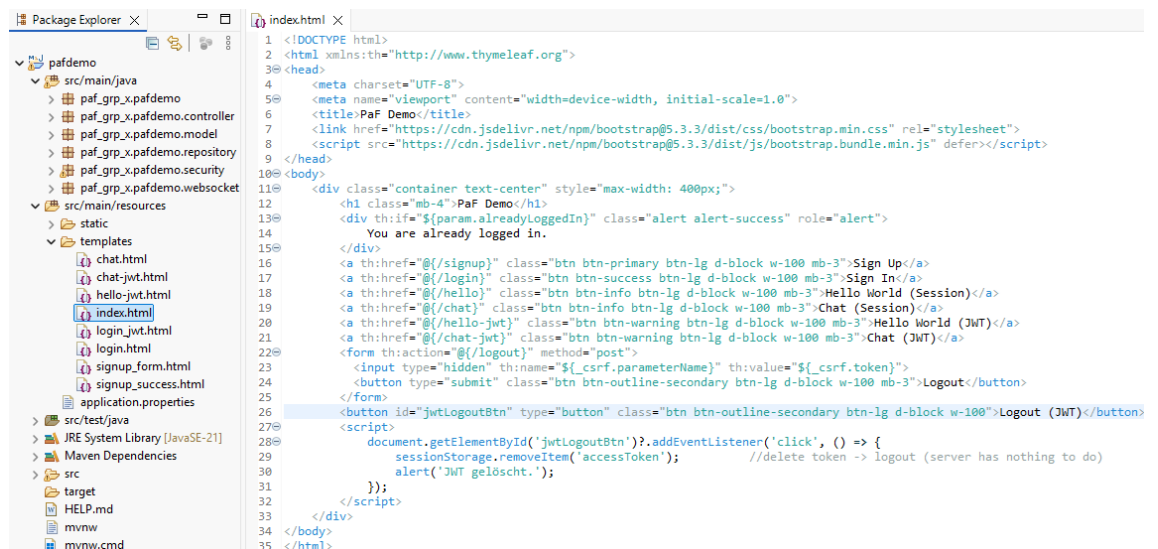


Abb. 5.10: index.html mit JWT-bezogenen Menüpunkten

Der Logout-Button entfernt insbesondere das JWT aus dem Session Storage. Damit kann es bei keinen nachfolgenden Requests mitgesendet werden, was faktisch einem Logout entspricht. Der Server ist also in diesem Fall nicht aktiv an dem Ausloggen beteiligt.

Insgesamt stellt sich unsere Startseite mit dem Menü nun wie folgt dar. Das Sign-In differenziert nach Session-Login und JWT-Login. Je nach gewählter Authentifizierung sind dann entweder die mit "(Session)" oder die mit "(JWT)" gekennzeichneten Menüpunkte zugreifbar.

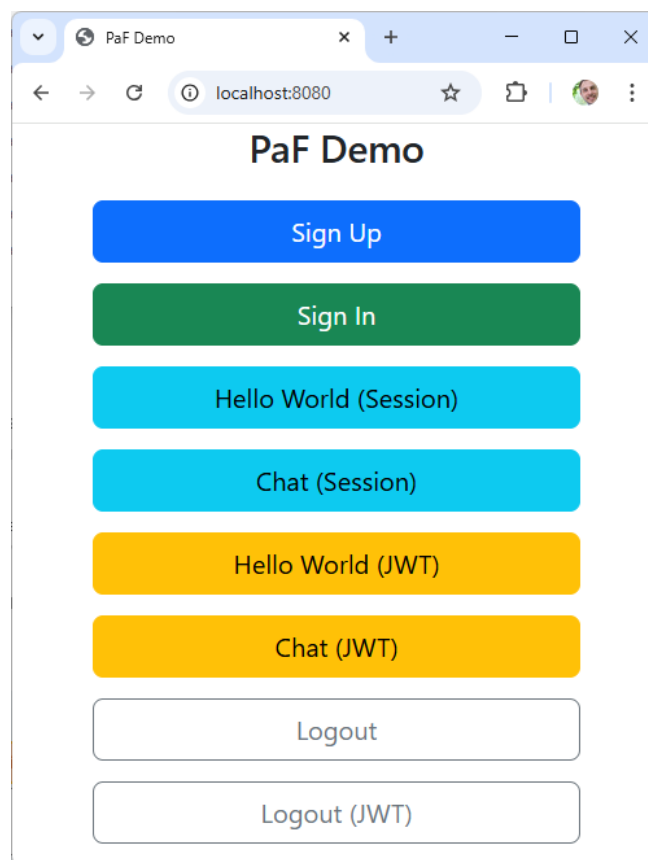


Abb. 5.11: Startseite mit Logout- und JWT-Support

5.3 Fortgeschrittenes JWT-Handling

Insbesondere in produktiv eingesetzten Anwendungen empfiehlt es sich, das JWT-Handling etwas ausgefeilter zu gestalten als bisher gezeigt, vor allem im Hinblick auf Usability und Sicherheit. Solche Erweiterungen könnten sein:

- **JWT Blacklist:** Ein Logout besteht nicht mehr nur aus dem clientseitigen Löschen des JWT, sondern wird explizit als Request an den Server implementiert. Dieser trägt das JWT in eine sog. Blacklist ein. Bei jedem Request wird das enthaltene JWT mit dieser Blacklist abgeglichen - sollte es dort enthalten sein, wird der

Request abgewiesen. Damit wird die Anwendung gegen Requests von Clients abgesichert, die in den Besitz von zwar noch nicht zeitlich abgelaufenen, aber per Logout (oder Refresh - siehe unten) invalidierten Tokens gekommen sind.

- **JWT Refresh:** Kurz vor Ablauf der Gültigkeit eines JWT fordert der Client beim Server ein neues Token an. Das Token-Refresh kann über einen clientseitigen Timer gesteuert werden und erspart dem Client eine neue Authentifizierung mit Credentials nach dem Ablauf der Gültigkeit des alten JWT. Insbesondere ein durch JWT-Ablauf erzwungener Logout mitten im Spiel wäre ja äußerst ungünstig. Das ersetzte Token wird der serverseitigen Blacklist hinzugefügt.
- **Inaktivitätsprüfung:** Das o. g. Token Refresh sollte nur erfolgen, wenn der Client tatsächlich in der Anwendung aktiv ist. Ansonsten wäre ein Logout bei Token-Ablauf die bessere Wahl.

Im folgenden implementieren wir zunächst die Token Blacklist und das Token Refresh auf der Serverseite unserer Demo-Anwendung. Die clientseitigen Anpassungen (Refresh Timer und Inaktivitätsprüfung) werden wir später in unserer Single Page Application vornehmen.

JWT Blacklist

Die Blacklist-Funktionalität bündeln wir in der Service-Klasse `JwtBlacklistService`, wie üblich mit Annotation `@Service`. Die Klasse arbeitet strenggenommen nicht mit einer Liste, sondern mit einer Menge (Set), die ihrerseits auf einer thread-sicheren `HashMap` (Klasse `ConcurrentHashMap`) basiert. Sie verfügt über einfache Methoden, um ein Token zu "blacklisten", es aus der Blacklist zu entfernen bzw. zu ermitteln, ob ein gegebenes Token auf der Blacklist steht.

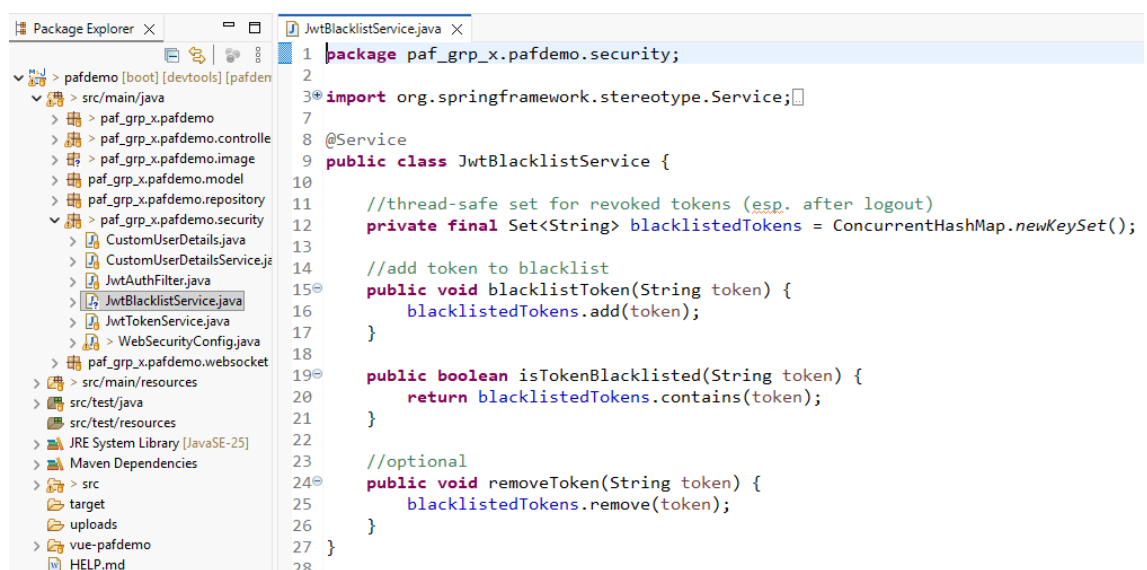


Abb. 5.12: Klasse JwtBlacklistService

Unsere neue Service-Klasse verwenden wir im JwtAuthController (vgl. Serverseitig JWT implementieren). Zunächst erweitern wir die Dependency Injection um den neuen JwtBlacklistService sowie vorsorglich schon mal um den aus einem vorigen Kapitel bekannten UserDetailsService. Letzteren benötigen wir später für den Token Refresh - siehe unten.



Abb. 5.13: JwtAuthController, erweiterte Dependency Injection

Wir fügen der Klasse JwtAuthController nun eine explizite logout-Methode zu, die das JWT aus dem Authorization Header extrahiert und mit Hilfe des JwtBlacklistService auf die Blacklist setzt. Über die Annotation @RequestHeader können wir den Header direkt als Parameter an die Methode übergeben. Da das Logout nicht fehlschlagen kann, geben wir immer den HTTP-Status 200 OK zurück.

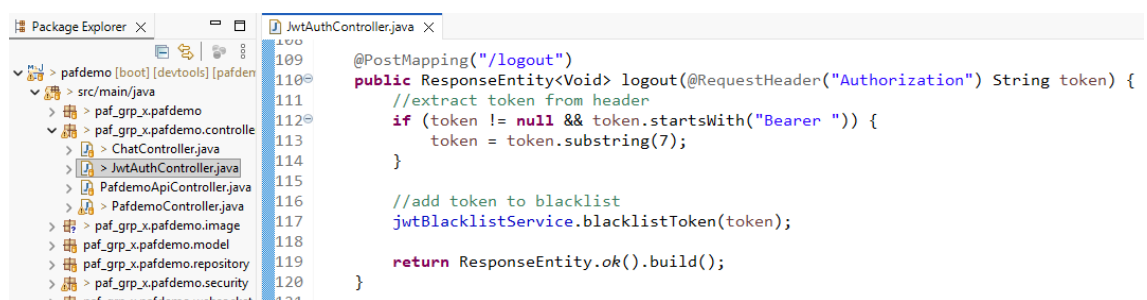


Abb. 5.14: JwtAuthController, logout-Methode

JWT Refresh (serverseitig)

Um ein Token Refresh zu ermöglichen, erweitern wir den JwtAuthController um eine weitere Methode, die wir refreshToken() nennen. Diese ermittelt den Usernamen aus dem übergebenen (alten) JWT, holt sich über den UserDetailsService ein UserDetails-Objekt, um aus diesem mittels JwtTokenService ein neues JWT generieren zu lassen

und dieses als Response zurückzugeben. Das alte Token wird - analog zum Logout (siehe oben) - auf die Blacklist genommen.

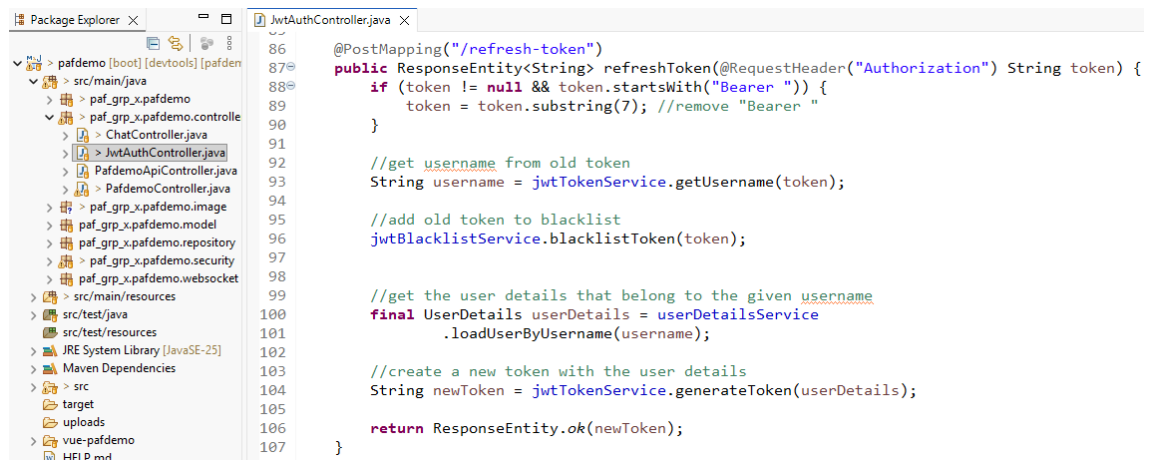


Abb. 5.15: JwtauthController, refresh-Methode

Abgleich von Tokens mit der Blacklist

Bisher haben wir zwar Tokens auf die Blacklist gesetzt, aber bei eingehenden Requests noch nicht geprüft, ob diese eventuell auf der Blacklist stehen. Das holen wir jetzt nach.

Im Kapitel Serverseitig JWT implementieren hatten wir die Klasse JwtauthFilter entworfen, deren Methode doFilterInternal() jeden eingehenden Request daraufhin überprüft, ob er einen Authorization Header mit einem JWT enthält und ob dieses valide ist. Nur in diesem Fall darf der Request den Filter passieren.

In diese Methode können wir sehr einfach unsere JWT-Blacklist integrieren, indem wir im Falle eines enthaltenen JWT prüfen, ob dieses auf der Blacklist steht, und falls ja, eine Exception werfen, so dass der Request den Filter nicht mehr passieren kann. Wenn das JWT nicht auf der Blacklist steht, wird die Filterlogik fortgesetzt. Um den Abgleich mit der Blacklist durchführen zu können, müssen wir den JwtauthFilter bekannt machen.

Der folgende Quellcode zeigt die beschriebenen Sachverhalte.



Code

```
private final JwtauthBlacklistService jwtBlacklistService;

public JwtauthFilter(..., JwtauthBlacklistService jwtBlacklistService) {
    //...
    this.jwtBlacklistService = jwtBlacklistService;
}

@Override
protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
```

```
FilterChain filterChain) throws ServletException,
IOException {

    String header = request.getHeader("Authorization");
    if (header != null && header.startsWith("Bearer ")
        && SecurityContextHolder.getContext().getAuthentication() == null) {

        String token = header.substring(7);

        if (jwtBlacklistService.isTokenBlacklisted(token)) {
            throw new IllegalArgumentException("Unauthorized: Token has been revo
ked.");
        }
        //...
    }
}
```



Gliederung

6 Eine Single-Page-Application entwickeln

6 Eine Single-Page-Application entwickeln

6.1 Serverseitige SPA-Anpassungen vornehmen

6.2 Clientseitig die SPA umsetzen

6.3 Clientseitige JWT-Erweiterungen

Bisher haben wir das Frontend unserer Anwendung über Thymeleaf-Templates realisiert, die dynamisch vom Server mit Daten angereichert und als einzelne HTML-Seiten an anfragende Clients ausgeliefert werden. Dies entspricht der klassischen Multi-Page-Architektur vieler Websites. Eine bessere User Experience lässt sich über eine sog. Single-Page-Application (SPA) erreichen. Hier wird der DOM-Baum einer Anwendung nur einmal initial geladen. Im weiteren Verlauf der Anwendungsnutzung wird die DOM-Struktur gezielt verändert, so dass dem User z. B. eine andere View angezeigt wird, nachdem er auf einen Button geklickt hat. Ein vollständiges DOM-Reload entfällt jedoch. Somit bietet die Anwendung eine User Experience, die man von Desktop-Applikationen oder nativen Apps kennt. Eine SPA bietet sich insbesondere an, wenn es um die Bereitstellung von zusammenhängenden Funktionalitäten geht und weniger um die reine Bereitstellung von Informationen (auf einzelnen Seiten).

Technisch lassen sich SPAs mit Frontend-Frameworks wie z. B. Vue.js (kurz Vue), Angular und React realisieren.



Anmerkung

React wird inzwischen offiziell nicht mehr als Framework, sondern als Library beschrieben, was aber aus verschiedenen Gründen diskussionswürdig ist, u. A. weil React den "Lebenszyklus" von Views aktiv steuert, was weit über das hinausgeht, was Libraries typischerweise tun.

In diesem Leitfaden demonstrieren wir die Erstellung einer SPA anhand von Vue in Verbindung mit Vite, einem Build Tool und Dev Server für Vue-Projekte. Die gezeigten Konzepte finden sich in ähnlicher Form auch in anderen Frontend-Frameworks wieder. Im Abschnitt Fortgeschrittenes JWT-Handling waren wir noch die clientseitigen Erweiterungen schuldig geblieben, konkret das Erneuern von JWTs kurz vor Ablauf (Token Refresh) und das Löschen von JWTs nach clientseitiger Inaktivität. Diese Features werden wir am Ende dieses Kapitels in unsere SPA einbauen.

6.1 Serverseitige SPA-Anpassungen vornehmen

Serverseitig nutzen wir die in den vorigen Abschnitten dieses Leitfadens entwickelte SpringBoot-Anwendung. An dieser müssen wir nur kleinere Anpassungen vornehmen, um sie - zusätzlich zum bisherigen Thymeleaf-Frontend - auch für einen SPA-Client nutzbar zu machen.

Zunächst müssen wir in unserer Web-Chain den Pfad `/app/**` ohne Authentifizierung zulassen, da wir über diesen die SPA-Requests empfangen wollen:



Code

```
http.  
    //...  
    //static files (incl. SPA assets), also freely accessible  
    .requestMatchers("/app/**", "/css/**", "/js/**", "/images/**", "/webjars/**",  
        "/favicon.ico", "/chat-jwt.js").permitAll()
```

Cross-Origin-Resource-Sharing (CORS)

Des weiteren müssen wir uns dem Thema Cross-Origin-Resource-Sharing (CORS) widmen: Unsere SPA wird auf einem anderen Port laufen als unsere Server-Anwendung. (Wichtig: Bei unserem Thymeleaf-Frontend war dies nicht der Fall!) Solche Cross-Origin-Zugriffe müssen serverseitig explizit erlaubt werden; ansonsten werden sie mit dem HTTP-Fehler 403 (forbidden) abgewiesen. Die Erlaubnis für den Zugriff von bestimmten Origins (hier: unserer Vue/Vite-SPA auf `http://localhost:5173`) erteilen wir über eine `CorsConfiguration` in unserer `WebSecurityConfig`-Klasse. Hierbei können wir u. A. im Detail eingrenzen, welche HTTP-Methoden und -Header wir zulassen wollen:



Code

```
//added for SPA support (Vue.js), especially if no client side proxy is used  
@Bean  
CorsConfigurationSource corsConfigurationSource() {  
    CorsConfiguration cors = new CorsConfiguration();  
    cors.setAllowedOrigins(List.of("http://localhost:5173"));  
    cors.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS"));  
    cors.setAllowedHeaders(List.of("Authorization", "Content-Type"));  
    cors.setAllowCredentials(true);  
    UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();  
    source.registerCorsConfiguration("/api/**", cors);  
    return source;  
}
```

Eine Alternative zur serverseitigen CORS-Konfiguration stellt die clientseitige Konfiguration eines Reverse-Proxy dar, der Anfragen zum Backend so tunnelt, dass anstelle von CORS die Same Origin Policy greift und die Anfragen folglich akzeptiert werden. Einen solchen Proxy können wir in der Datei `vite.config.js` spezifizieren:



```
vite.config.js X
vite.config.js > ...
1 import { defineConfig } from 'vite'
2 import vue from '@vitejs/plugin-vue'
3
4 export default defineConfig({
5   plugins: [vue()],
6   server: {
7     port: 5173,
8     //to avoid CORS trouble...
9     proxy: {
10       '/api': { target: 'http://localhost:8080', changeOrigin: true },
11       '/websocket': { target: 'ws://localhost:8080', ws: true, changeOrigin: true },
12     }
13   }
14 })
```

Abb. 6.1: vite.config.js

Eigentlich sollte einer der beiden Ansätze - entweder serverseitige CORS-Konfiguration oder clientseitiger Reverse Proxy - ausreichen, um CORS-Probleme zu vermeiden. Da es aber immer wieder CORS-Komplikationen in PaF-Projekten gibt, schadet es auch nicht, beide Ansätze parallel zu fahren. ;-)

Neuer REST-Endpunkt für das Sign-Up

Für das Registrieren eines neuen Users hatten wir bereits einen Endpunkt `@PostMapping("/process_signup")` im `PaFdemoController` eingerichtet. Dieser bekommt die User-Daten aus dem Thymeleaf-Formular übergeben, legt den User-Datensatz an und liefert ein Thymeleaf-Template mit einer Erfolgsmeldung und weiterführenden Links zurück.

Für die SPA benötigen wir jedoch einen Endpunkt, der die User-Daten im Request-Body bekommt und das Ergebnis der Registrierung (Erfolg oder Fehlermeldung) als Response-Entity zurückliefert. Diesen Endpunkt ergänzen wir im `JwtAuthController`:

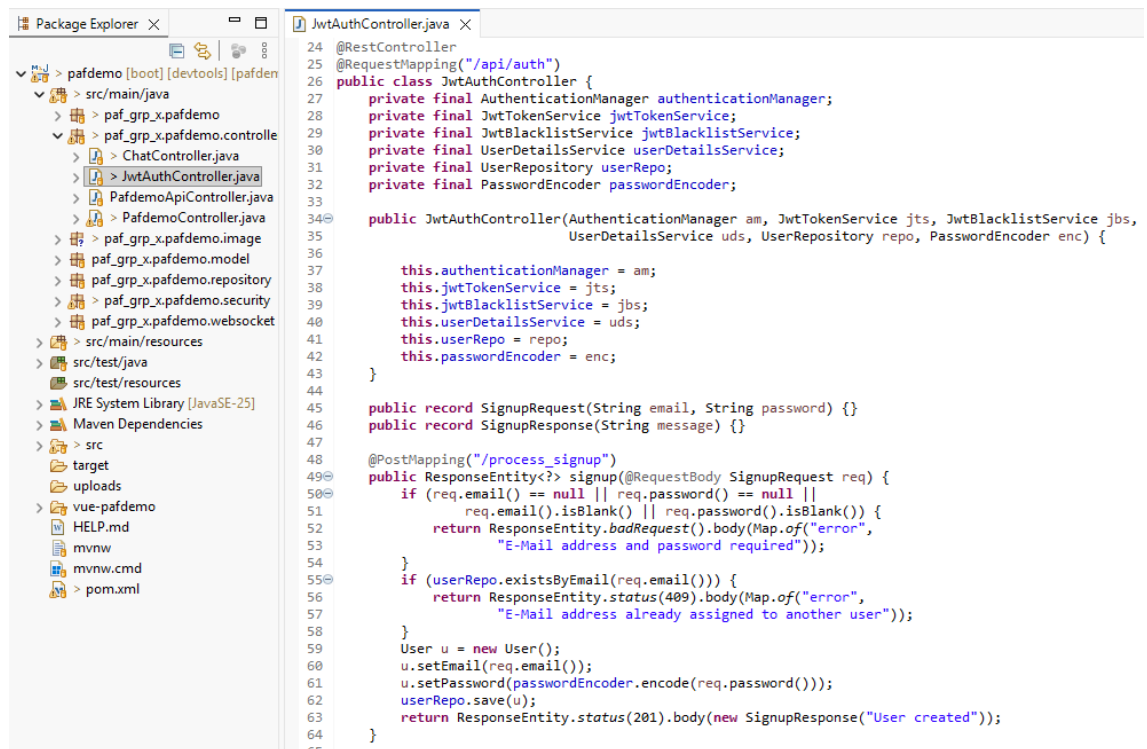


Abb. 6.2: Neuer Endpunkt im JwtauthController

Die in `signup()` aufgerufene Methode `existsByEmail` ergänzen wir im `UserRepository`:



Code

```
boolean existsByEmail(String email);
```

In der `apiChain` von `WebSecurityConfig` müssen wir den neuen Endpunkt nun noch freischalten, da zum Zeitpunkt der Registrierung noch kein JWT mitgesendet werden kann:



Code

```
//(1) apiChain for JWT handling: /api/** is stateless and uses JWT
@Bean @Order(1)
public SecurityFilterChain apiChain(HttpSecurity http, JwtAuthFilter jwtFilter)
throws Exception {
    http
        //...
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/api/auth/login", "/api/auth/process_signup").permitAll()
            .anyRequest().authenticated()
        )
        //...
```

Die SPA-seitige Umsetzung des Sign-Up wird im folgenden Abschnitt erläutert.

6.2 Clientseitig die SPA umsetzen

Nach den serverseitigen Anpassungen können wir uns nun um den SPA-Client selbst kümmern.

Vue/Vite-Projekt aufsetzen

Ein initiales Vue/Vite-Projekt können wir mit folgenden Terminal-Anweisungen erzeugen. Voraussetzung dafür ist, dass Node.js und npm auf Ihrem Rechner installiert sind. (Zu npm gibt es auch Alternativen wie yarn, die an dieser Stelle nicht betrachtet werden.)



Code

```
npm create vue@latest  
#hier Fragen zur Installation beantworten (möglichst Vue-Router mitinstallieren)  
cd <projektordner>  
npm install  
npm run dev
```

Notfalls kann der Vue-Router auch nachinstalliert werden:



Code

```
npm i vue-router@4
```

Der Dev-Server läuft standardmäßig auf `http://localhost:5173`. Als IDE für Vue-Projekte wird VS Code in Verbindung mit dem Plugin Volar empfohlen.

JWT-Unterstützung

In unserer SPA soll die Authentifizierung über JWT laufen. Hierbei können wir sämtliche serverseitigen Erweiterungen nutzen, die wir im Abschnitt JWT verwenden vorgenommen haben. Analog zum Thymeleaf-Frontend wollen wir das JWT wieder im Session Storage speichern, was für unsere Demo-Anwendung völlig ausreichend ist.



Hinweis

In komplexeren Anwendungen mag der Einsatz einer Zustandsverwaltung wie - im Vue-Kontext - Pinia sinnvoll sein, um Daten clientseitig zu speichern.

In dem Helfer-Modul `auth.js` stellen wir Funktionen bereit, um ein JWT im Session Storage abzulegen, es von dort zu lesen sowie um es im Authorization Header von `fetch`-Aufrufen zu hinterlegen, wie wir es bereits kennengelernt haben.

```

JS auth.js M ●
src > JS auth.js > ...
1  //in Pafdemo, the JWT is stored in sessionStorage
2  //for production: use HttpOnly cookie or refresh flow
3  export const auth = {
4    get token() {
5      return sessionStorage.getItem('accessToken')
6    },
7    set token(t) {
8      if (t) sessionStorage.setItem('accessToken', t);
9      else sessionStorage.removeItem('accessToken');
10   },
11
12   //include JWT in any api call
13   async apiFetch(input, init = {}) {
14     const headers = new Headers(init.headers || {});
15     const t = auth.token;
16     if (t) headers.set('Authorization', 'Bearer ' + t);
17     return fetch(input, { ...init, headers });
18   }
19 };

```

Abb. 6.3: Helfer-Modul auth.js

Einstiegspunkt und Navigation

Den Einstiegspunkt in unsere SPA definiert die Datei main.js:

```

JS main.js M ●
src > JS main.js
1  import { createApp } from 'vue'
2  import { router } from './router.js'
3  import App from './App.vue'
4
5  createApp(App).use(router).mount('#app')

```

Abb. 6.4: Einstiegspunkt main.js

Sie legt insb. das HTML-Element fest, in das die View-Inhalte zur Laufzeit eingefügt werden sollen. In unserem Fall ist das ein div-Element mit der ID "app", welches in der Datei index.html deklariert ist, die wiederum main.js importiert:

```

index.html ×
1  <!doctype html>
2  <html lang="en">
3  <head>
4    <meta charset="UTF-8" />
5    <link rel="icon" type="image/svg+xml" href="/vite.svg" />
6    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
7    <title>PaF Demo</title>
8    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
9  </head>
10 <body>
11   <div id="app"></div>
12   <script type="module" src="/src/main.js"></script>
13   <script src="https://cdn.jsdelivr.net/npm/@stomp/stompjs@7.1.1/bundles/stomp.umd.min.js"></script>
14 </body>
15 </html>

```

Abb. 6.5: SPA-Startseite index.html

Ausgehend von dieser (zunächst noch leeren) Startseite müssen wir nun festlegen, welche Inhalte initial angezeigt werden sollen und wie man die einzelnen Views der SPA ansteuern kann. Diese Dinge spezifizieren wir in der Datei `router.js`. Der Router wird von `main.js` importiert und der Anwendung zur weiteren Nutzung zur Verfügung gestellt (siehe oben).

```
JS router.js ×
src > JS router.js > ...
1  import { createRouter, createWebHistory } from 'vue-router'
2  import { auth } from './auth.js'
3  import Menu from './views/Menu.vue'
4  import SignUp from './views/SignUp.vue'
5  import Login from './views/Login.vue'
6  import Hello from './views/Hello.vue'
7  import Chat from './views/Chat.vue'
8
9  const routes = [
10   { path: '/', redirect: '/app/menu' },
11   { path: '/app/menu', component: Menu },
12   { path: '/app/signup', component: SignUp },
13   { path: '/app/login', component: Login },
14   { path: '/app/hello', component: Hello, meta: { requiresAuth: true } },
15   { path: '/app/chat', component: Chat, meta: { requiresAuth: true } },
16   { path: '/*', redirect: '/app/menu' } // Fallback
17 ]
18
19 export const router = createRouter({
20   //if the SPA runs under /app (in production), use createWebHistory('/app')
21   history: createWebHistory(),
22   routes
23 })
24
25 router.beforeEach((to) => {
26   if (to.meta.requiresAuth && !auth.token) return '/app/login'
27 });
```

Abb. 6.6: `router.js`

`router.js` importiert die einzelnen Views und definiert Routes innerhalb der Anwendung. Beispielsweise wird `/` zu `/app/menu` umgeleitet, was wiederum auf die View-Komponente "Menu" verweist. Die geschützten Endpunkte sind über ein Meta-Attribut `requiresAuth` gekennzeichnet, so dass die SPA bei einem Zugriffswunsch auf die Login-Seite umleiten kann, sollte kein JWT im Session Storage vorliegen.

Dadurch dass der Router die komplette Navigation übernimmt, kann die Root-Komponente `App.vue` sehr minimalistisch bleiben:



Code

```
<template>
  <router-view/>
</template>
```

Sichten

Die in der SPA definierten Sichten "Menu", "SignUp", "Login", "Hello" und "Chat" sind 1:1 den Thymeleaf-Templates der Multi-Page-Variante nachempfunden, um eine gute Vergleichbarkeit zu ermöglichen. Wir verwenden dabei das sog. Options-API von Vue, das durch eine Strukturierung einer Sicht in die drei Abschnitte (Struktur und Binding in HTML), (Daten und Anwendungslogik in JavaScript, alternativ TypeScript) und optional (viewspezifische Gestaltung in CSS) gekennzeichnet ist. Alternativ hätten wir das sog. Composition-API von Vue nutzen können, das eine andere Struktur aufweist.

SignUp.vue

Die folgende Abbildung zeigt exemplarisch den template-Abschnitt der Sicht SignUp.vue. Man erkennt, dass es sich im Wesentlichen um HTML-Code handelt, der aber angereichert ist mit Vue-Direktiven wie v-model und v-if sowie Notationen wie Elementinhalte in doppelten geschweiften Klammern. Diese regeln das bidirektionale Data Binding sowie die situationsbedingte (Nicht-)Anzeige von Inhalten. Letztlich werden hier ähnliche Möglichkeiten bereitgestellt wie wir sie bereits in Thymeleaf kennengelernt haben.

```

▼ SignUp.vue ×
src > views > ▼ SignUp.vue > {} script setup
1  <template>
2    <div class="container d-flex justify-content-center align-items-center vh-100">
3      <div class="card p-4 shadow" style="max-width: 400px; width: 100%;">
4        <h4 class="mb-3 text-center">User Sign Up (SPA)</h4>
5
6        <form @submit.prevent="signup" novalidate>
7          <div class="mb-3">
8            <label class="form-label" for="email">E-Mail Address</label>
9            <input id="email" v-model.trim="email" type="email" class="form-control"
10              placeholder="name@example.com" required autocomplete="email" />
11          </div>
12          <div class="mb-3">
13            <label class="form-label" for="password">Password</label>
14            <input id="password" v-model="password" type="password" class="form-control"
15              placeholder="....." required autocomplete="new-password" />
16          </div>
17
18          <button class="btn btn-primary w-100 mb-3" :disabled="busy">Sign Up</button>
19          <RouterLink to="/app/menu" class="btn btn-secondary w-100">Return to Menu</RouterLink>
20
21          <div v-if="error" class="alert alert-danger mt-3" role="alert">{{ error }}</div>
22          <div v-if="success" class="alert alert-success mt-3" role="alert">{{ success }}</div>
23        </form>
24      </div>
25    </div>
26  </template>

```

Abb. 6.7: Sicht SignUp.vue, template-Teil

Die Formularaktion `@submit.prevent="signup"` ruft die asynchrone Funktion `signup()` auf, die im `script`-Abschnitt der Sicht deklariert ist. Hier ist insbesondere der `fetch()`-Aufruf zu erkennen, der den POST-Request an unseren eigens ergänzten REST-Endpunkt `/api/auth/process_signup` ausführt (vgl. Serverseitige SPA-Anpassungen vornehmen und dabei die eingegebenen Formulardaten im Body mitsendet. Die Response wird nach Eingang - je nachdem, sie Erfolg oder Misserfolg signalisiert - entsprechend behandelt.



```

28 <script setup>
29 import { ref } from 'vue'
30 import { useRouter } from 'vue-router'
31
32 const email = ref('')
33 const password = ref('')
34 const busy = ref(false)
35 const error = ref('')
36 const success = ref('')
37 const router = useRouter()
38
39 async function signup() {
40   if (busy.value) return
41   busy.value = true; error.value = ''; success.value = ''
42   try {
43     const resp = await fetch('/api/auth/process_signup', {
44       method: 'POST',
45       headers: { 'Content-Type': 'application/json' },
46       body: JSON.stringify({ email: email.value, password: password.value })
47     })
48     if (!resp.ok) {
49       // 409 = e-mail address already in use, 400 = Bad Request
50       let msg = `Fehler: ${resp.status}`
51       try { const j = await resp.json(); msg = j.error || j.message || msg } catch {}
52       error.value = msg
53       return
54     }
55     success.value = 'You signed up successfully.'
56   } catch (e) {
57     error.value = 'Network error'
58   } finally {
59     busy.value = false
60   }
61 }
62 </script>

```

Abb. 6.8: Sicht `SignUp.vue`, `script`-Teil

Login.vue

Der `template`-Abschnitt von `Login.vue` ähnelt sehr dem von `SignUp.vue`. Auch der `script`-Abschnitt ist vergleichbar, nur dass diesmal im Erfolgsfall das JWT aus der Response ausgelesen und im Session Storage abgelegt wird (mithilfe des Moduls

auth.js - siehe oben). Anschließend wird über den Router die Weiterleitung ins Menü veranlasst:



Code

```
const data = await resp.json() // { accessToken, expiresIn }
auth.token = data.accessToken
router.push('/app/menu')
```

Hello.vue

Die Sicht Hello.vue ist sehr an das Thymeleaf-Template hello-jwt.html (vgl. Abschnitt Clientseitig JWT implementieren) angelehnt. Erwähnenswert ist hier, dass load() hier die Funktion apiFetch() aus dem Hilfsmodul auth.js (siehe oben) verwendet, d. h. das JWT wird implizit im Authorization Header mitgesendet, ohne dass wir es an dieser Stelle explizit angeben müssen. Auf diese Weise wird im Erfolgsfall der Zugriff auf die geschützte Ressource /api/hello gewährt.

```
▼ Hello.vue X
src > views > ▼ Hello.vue > ...
1  <template>
2  |  <div v-html="html"></div>
3  </template>
4
5  <script setup lang="ts">
6  import { onMounted, ref } from 'vue'
7  import { auth } from '../auth'
8  import { useRouter } from 'vue-router'
9  const router = useRouter()
10 const html = ref('Loading...')
11
12 async function load() {
13   const r = await auth.apiFetch('/api/hello')
14   if (!r.ok) { html.value = `Fehler: ${r.status} ${r.statusText}`; return }
15   html.value = await r.text() //server endpoint delivers text/html
16 }
17
18 onMounted(load)
19 </script>
```

Abb. 6.9: Sicht Hello.vue

Weiterhin ist der Lifecycle-Hook onMounted() zu nennen. Dieser ist Vue vordefiniert und wird daher aus dem Modul "vue" importiert. Lifecycle-Hooks erlauben es, sich an verschiedenen Stellen des "Lebenszyklus" einer Sicht "einzuhaken" und an diesen Stellen gewünschte Funktionalitäten bereitzustellen. onMounted() wird Vue-seitig ausgeführt, unmittelbar nachdem das initiale Erzeugen und Rendern der DOM-Knoten abgeschlossen wurde. Dieser Hook eignet sich daher besonders für Initialisierungen und Aktionen, die ausgelöst werden sollen, bevor ein User mit der Sicht interagiert

(z. B. Aufbau einer STOMP-Verbindung). In diesem Fall wird die `load()`-Funktion aufgerufen, um sofort das HTML-Snippet mit "Hello World!" vom Server abzurufen. Gegenstück zu `onMounted()` ist der Lifecycle-Hook `onUnmounted()`, der sich gut für "Aufräumarbeiten" eignet, bevor die Sicht final geschlossen wird (z. B. Abbau einer STOMP-Verbindung, Freigabe von reserviertem Speicher usw.) Weitere Infos zu diesen und anderen Lifecycle-Hooks in Vue finden Sie in der [Vue-Dokumentation](#).

Chat.vue

Die Chat-Sicht ist 1:1 der entsprechenden Thymeleaf-Implementierung nachempfunden. Die dort in das Modul `chat-jwt.js` ausgelagerten Funktionalitäten wie `connect()` (inkl. Event-Handlern wie `onConnect` und `onStompError`), `disconnect()` und `send()` finden sich nun im script-Teil der Sicht wieder. Analog zur Thymeleaf-Implementierung wird auch hier beim STOMP-Handshake das JWT zur Authentifizierung mitgesendet.

Da der Quellcode von `Chat.vue` etwas länglich ist und viele Ähnlichkeiten zur Thymeleaf-Variante aufweist, verzichten wir an dieser Stelle auf entsprechende Screenshots.

Menu.vue

Auch die SPA-Startseite mit dem Menü orientiert sich an der Thymeleaf-Darstellung. Die Navigation zu den einzelnen Menüpunkten läuft hier über `RouterLink`-Elemente. Eine weitere Besonderheit ist die Funktion `logoutJwt()`, welche das JWT aus dem Session Storage löscht und ggf. noch offene STOMP-Verbindungen in anderen Tabs schließt.

```
Menu.vue X
src > views > Menu.vue > { } script setup > logoutJwt
1  <template>
2    <div class="container text-center" style="max-width: 400px;">
3      <h1 class="mb-4">PaF Demo (SPA, JWT)</h1>
4
5      <!-- optional note -->
6      <div v-if="alreadyLoggedIn" class="alert alert-success" role="alert">
7        | You are already logged in.
8      </div>
9
10     <RouterLink to="/app/signup" class="btn btn-primary btn-lg d-block w-100 mb-3">Sign Up</RouterLink>
11     <RouterLink to="/app/login" class="btn btn-success btn-lg d-block w-100 mb-3">Sign In</RouterLink>
12     <RouterLink to="/app/hello" class="btn btn-warning btn-lg d-block w-100 mb-3">Hello World</RouterLink>
13     <RouterLink to="/app/chat" class="btn btn-warning btn-lg d-block w-100 mb-3">Chat</RouterLink>
14
15     <button type="button" class="btn btn-outline-secondary btn-lg d-block w-100"
16       | @click="logoutJwt">
17       | Logout
18     </button>
19   </div>
20 </template>
21
22 <script setup>
23 import { computed } from 'vue'
24 import { useRoute } from 'vue-router'
25 import { auth } from '../auth.js'
26
27 const route = useRoute()
28 const alreadyLoggedIn = computed(() => route.query.alreadyLoggedIn !== undefined)
29
30 function logoutJwt() {
31   //close STOMP connection properly (if open in any tab)
32   try { window.stompJwtDeactivate?.() } catch {}
33
34   //delete token
35   auth.token = null;
36
37   alert('JWT deleted.');
```

Abb. 6.10: Sicht Menu.vue

Damit haben wir die Funktionalität unserer Thymleaf-Anwendung vollständig als SPA nachgebaut (mit Ausnahme der sessionbasierten Authentifizierung). Die SPA präsentiert sich im selben Look-and-Feel und arbeitet mit derselben Server-Implementierung zusammen.

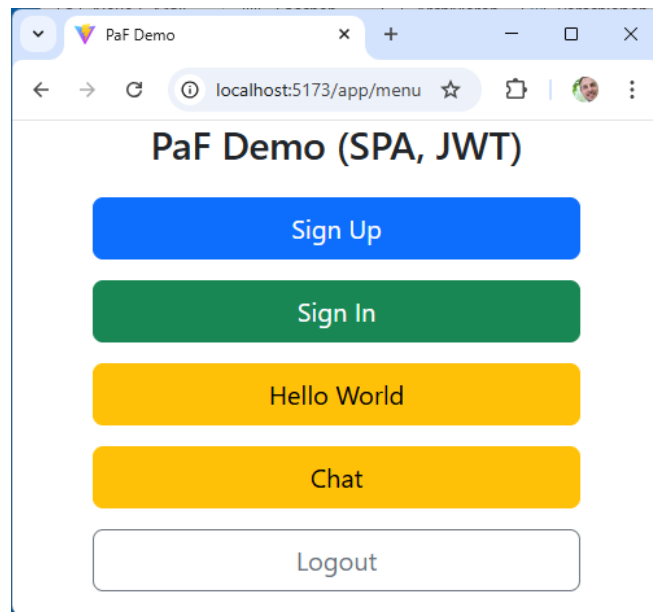


Abb. 6.11: Menü der SPA

Da sowohl die SPA als auch die Thymeleaf-Anwendung mit demselben Server kommunizieren, kann z. B. ein SPA-Client problemlos mit einem Thymeleaf-Client chatten.

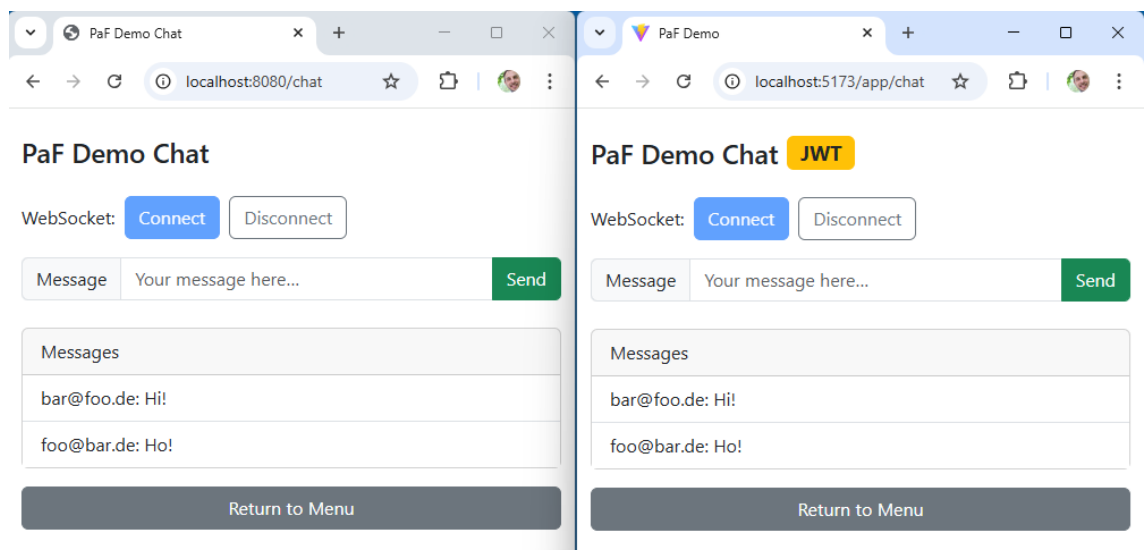
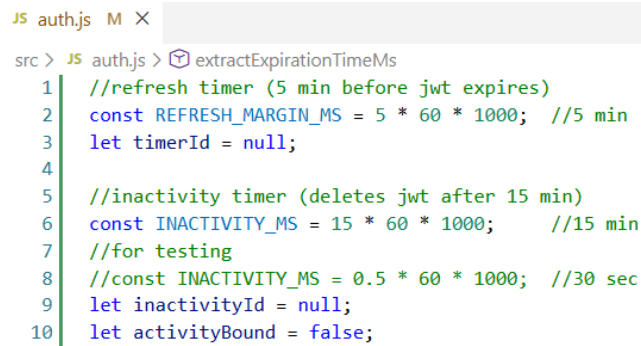


Abb. 6.12: Chat-Funktion der SPA

6.3 Clientseitige JWT-Erweiterungen

Im Abschnitt Fortgeschrittenes JWT-Handling haben wir Konzepte wie Blacklisting und Refresh kennengelernt. Die Umsetzung der clientseitigen Anpassungen für das JWT Refresh steht aber noch aus - diese werden wir nun anhand unserer SPA vornehmen. Darüber hinaus werden wir ein automatisches Logout bei anhaltender Inaktivität implementieren.

Zunächst deklarieren wir in der Datei `auth.js` einige Konstanten und Variablen, die wir später verwenden werden. Ein JWT soll 5 min vor Ablauf erneuert werden. Eine Inaktivität von 15 min soll zum Löschen des JWT und damit zum Logout führen.



```
JS auth.js M X
src > JS auth.js > extractExpirationTimeMs
1 //refresh timer (5 min before jwt expires)
2 const REFRESH_MARGIN_MS = 5 * 60 * 1000; //5 min
3 let timerId = null;
4
5 //inactivity timer (deletes jwt after 15 min)
6 const INACTIVITY_MS = 15 * 60 * 1000; //15 min
7 //for testing
8 //const INACTIVITY_MS = 0.5 * 60 * 1000; //30 sec
9 let inactivityId = null;
10 let activityBound = false;
```

Abb. 6.13: Konstanten und Variablen für Timer-Funktionalität im Helfer-Modul `auth.js`

JWT Refresh (clientseitig)

Die automatische Anforderung eines neuen JWT kurz vor Ablauf des alten realisieren wir über einen Timer im Vue-Client. Die Funktion `schedule()` beendet zunächst einen alten Timer - sofern vorhanden - und bestimmt dann mittels der Funktion `extractExpirationTimeMs()` die Ablaufzeit des übergebenen JWT in Millisekunden seit dem 01.01.1970. Anschließend wird berechnet, in wie vielen Millisekunden der Timer ablaufen soll (Delay). Es wird ein neuer Timer gesetzt, der nach Ablauf des Delay die Funktion `refresh()` aufruft.

```

JS auth.js M X
src > JS auth.js > refresh
12
13 function extractExpirationTimeMs(jwt) {
14   try {
15     //extract exp from middle part of token
16     const b64 = jwt.split('.')[1].replace(/-/g, '+').replace(/_/g, '/');
17     const payload = JSON.parse(atob(b64));
18     return (payload && typeof payload.exp === 'number') ? payload.exp * 1000 : null;
19   } catch {
20     return null;
21   }
22 }
23
24 //use browser API functions clearTimeout and setTimeout to schedule refresh
25 function schedule(jwt) {
26   if (timerId) clearTimeout(timerId);
27   if (!jwt) return;
28
29   const expMs = extractExpirationTimeMs(jwt);
30   if (expMs == null) return;
31
32   const delay = Math.max(0, expMs - Date.now() - REFRESH_MARGIN_MS);
33
34   //start new timer, calls refresh function below
35   console.log('[auth] next refresh in', Math.round(delay/1000), 's');
36   timerId = setTimeout(refresh, delay);
37 }

```

Abb. 6.14: Scheduling des JWT Refresh im Helfer-Modul auth.js

`schedule()` soll immer dann aufgerufen werden, wenn ein neues JWT gesetzt wird. Hierzu erweitern wir `set token()` im Helfer-Modul `auth.js` entsprechend. Damit der Timer auch dann gestartet wird, wenn beim Laden bereits ein JWT hinterlegt ist, ergänzen wir darüber hinaus eine Funktion `init()`, die ebenfalls `schedule()` aufruft.

```

JS auth.js M X
src > JS auth.js > auth > apiFetch
100 //in Pafdemo, the JWT is stored in sessionStorage
101 //for production: use HttpOnly cookie or refresh flow
102 export const auth = {
103   get token() {
104     return sessionStorage.getItem('accessToken')
105   },
106   set token(t) {
107     if (t) sessionStorage.setItem('accessToken', t);
108     else sessionStorage.removeItem('accessToken');
109
110     schedule(t); //new timer for token refresh
111   },
112   init() { //called from main.js
113     schedule(this.token);
114   },

```

Abb. 6.15: Aufruf des Scheduling im Helfer-Modul auth.js

`init()` wiederum wird von `main.js` aus aufgerufen:



```
//initialize token refresh
import { auth } from './auth.js'; auth.init();
```

Die bei Timeout aufgerufene Funktion *refresh()* sendet einen POST-Request an den REST-Endpunkt `/api/auth/refresh-token` des Servers (siehe Abschnitt Fortgeschrittenes JWT-Handling) und ersetzt - sofern die Response ein JWT enthält - das alte JWT mit dem neuen.

```
JS auth.js M X
src > JS auth.js > handleInactivity
39
40 //REST call for token refresh
41 async function refresh() {
42   //include old token
43   const headers = new Headers();
44   const t = auth.token;
45   if (t) {
46     console.log('[auth] refreshing..., old token: ', t);
47     headers.set('Authorization', 'Bearer ' + t);
48   }
49
50   const resp = await fetch('/api/auth/refresh-token', { method: 'POST', headers });
51   if (!resp.ok) {
52     auth.token = null; //refresh failed -> no token
53     return;
54   }
55   const newToken = await resp.text(); //server returns plain text
56   auth.token = newToken; //save and schedule
57   console.log('[auth] new token set: ', newToken);
58 }
```

Abb. 6.16: REST Call für JWT Refresh im Helfer-Modul `auth.js`

Um die Funktion zu testen, können wir serverseitig vorübergehend die Gültigkeitsdauer eines JWT (in `application.properties`) reduzieren:



```
#jwt.expirationSeconds=3600
#for testing: 6 min
jwt.expirationSeconds=360
```

Automatisches Logout bei anhaltender Inaktivität

Die Behandlung von Inaktivität wird komplett clientseitig umgesetzt. Wird 15 min lang keine Taste gedrückt und keine Pointerdown-Aktivität (Mouse, Touch oder Pen) ausgeführt, soll das hinterlegte JWT gelöscht werden, was einem Logout gleichkommt. Das geschieht also ggf. vor dem eigentlichen zeitlichen Ablauf des JWT. Jede Keydown- oder Pointerdown-Aktivität startet den Timer neu.

Ablaufzeit und einbezogene Aktivitäten sind natürlich nur beispielhaft und können beliebig angepasst werden. Wenn auch auf Mausbewegungen reagiert werden soll, empfiehlt sich eine Drosselung, um den Inaktivitätstimer nicht im Verlauf einer Mausbewegung ständig neu zu setzen.



```
JS auth.js M X
src > JS auth.js > bindActivityListeners
60
61 //use browser API functions clearTimeout and setTimeout to schedule inactivity
62 function resetInactivityTimer() {
63   console.log('[auth] activity -> reset');
64
65   if (inactivityId) clearTimeout(inactivityId);
66
67   if (!auth.token) return; //only if logged in
68
69   //start new timer, calls refresh function above
70   inactivityId = setTimeout(handleInactivity, INACTIVITY_MS);
71 }
72
73 function handleInactivity() {
74   console.log('[auth] inactive -> logout');
75
76   //close websocket connection, if any
77   try { window.stompJwtDeactivate?(); } catch {}
78
79   auth.token = null; //-> logout
80
81   //redirect to login page
82   try { window.location && window.location.assign('/app/login'); } catch {}
83 }
```

Abb. 6.17: Inaktivitätsbehandlung im Helfer-Modul auth.js

Die Funktion *resetInactivityTimer()* beendet zunächst den bisherigen Timer - wenn vorhanden - und startet anschließend einen neuen, sofern der User noch eingeloggt ist (JWT vorhanden). Bei Ablauf des Timers wird die Funktion *handleInactivity()* aufgerufen, die das bisherige JWT löscht, eine eventuell noch offene Websocket-Verbindung schließt und die Anwendung zur Login-Seite umleitet.

Der Aufruf von *resetInactivityTimer()* wird über mehrere Event Listener gesteuert, die - wie bereits beschrieben - auf die Ereignisse *pointerdown* und *keydown* reagieren. Zusätzlich wird über das Ereignis *visibilitychange* sichergestellt, dass ein Timer Reset auch dann erfolgt, wenn die Applikation wieder sichtbar wird, z. B. nachdem sie durch einen Wechsel zu einem anderen Browser-Tab in den Hintergrund gerückt worden ist.

```

JS auth.js M X
src > JS auth.js > [auth] > [apiFetch]
85
86 function bindActivityListeners() {
87   if (activityBound) return; //bind onky once
88   activityBound = true;
89
90   window.addEventListener('pointerdown',
91     resetInactivityTimer, { passive: true }); //mouse, touch, pen
92   window.addEventListener('keydown',
93     resetInactivityTimer, { passive: true });
94   window.addEventListener('visibilitychange', () => {
95     //reset when app comes back to front
96     if (!document.hidden) resetInactivityTimer();
97   });
98 }

```

Abb. 6.18: Event Listener Binding im Helfer-Modul auth.js

Um das Binding der Event Listener bereits beim Laden der Komponente durchzuführen, ergänzen wir die Funktion `init()` entsprechend. Für den Fall, dass beim Laden bereits ein JWT hinterlegt ist, rufen wir außerdem `resetActivityTimer()` aus `init()` heraus auf:



Code

```

init() {
  schedule(this.token); //called from main.js
  bindActivityListeners();
  resetInactivityTimer(); //reset if token exists on load
},

```

Die Inaktivitätsbehandlung kann getestet werden, indem `INACTIVITY_MS` vorübergehend heruntergesetzt wird, z. B. auf 30 sek.

7 Projektspezifische Aspekte

7 Projektspezifische Aspekte

7.1 Schutz vor CSRF implementieren

7.2 HTTPS-Verbindung einrichten

7.3 Bilder zwischen Client und Server transferieren

7.4 Mit Countdowns arbeiten

7.5 Auf User-seitige Abbrüche reagieren

7.6 Typische Fallstricke im Projektverlauf

In diesem Abschnitt behandeln wir eine Reihe von kleineren Themen, die möglicherweise für Ihr Projekt relevant sind. Dabei geht es zum einen um Sicherheitsaspekte wie CSRF-Schutz und HTTPS-Kommunikation, zum anderen um spezielle Anwendungsfunktionen wie den Transfer von Bildern und das Behandeln von Timeouts, speziell im Spielkontext.

7.1 Schutz vor CSRF implementieren

CSRF steht für Cross-Site Request Forgery und ist eine Angriffstechnik für Webseiten. Das Opfer wird dazu gebracht, in einer Web-Anwendung, in der es aktuell authentifiziert ist, unbeabsichtigte zustandsverändernde Aktionen (z. B. POST-Requests) auszuführen. Dabei könnte es sich z. B. um Passwortänderungen oder Banküberweisungen handeln. Die folgende Abbildung zeigt den Ablauf eines CSRF-Angriffs am Beispiel eines Web-Formulars.

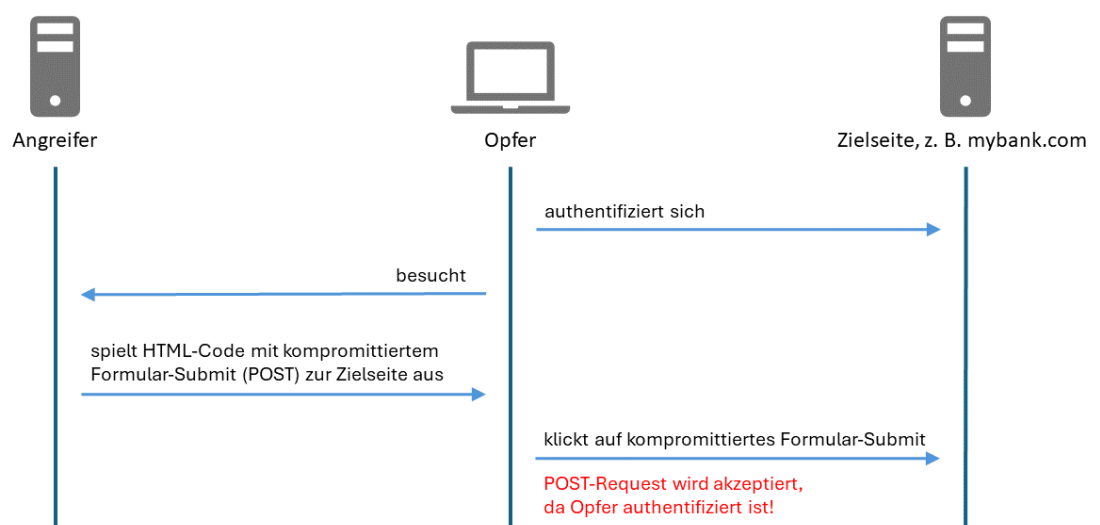


Abb. 7.1: Ablauf eines CSRF-Angriffs am Beispiel eines Web-Formulars

Hierbei authentifiziert sich das Opfer zunächst bei einer legitimen Website, z. B. bei einer Bank, um den Kontostand zu prüfen. Anschließend besucht es eine manipulierte Seite eines Angreifers - diese kann sich z. B. hinter Werbung auf einer anderen legitimen Seite oder einem Phishing-Link in einer E-Mail verbergen. Die Seite des Angreifers enthält ein präpariertes Skript oder Formular, das automatisch bzw. durch Interaktion eine schädliche Aktion auf der legitimen Website auslöst – in unserem Bankbeispiel könnte das eine Überweisung sein. Da der Browser des Opfers das Session-Cookie mitsendet (Opfer ist regulär authentifiziert), erkennt die Zielseite den Request als gültig an, obwohl er vom Angreifer initiiert wurde.

Eine wirksame Schutzmaßnahme gegen diese Angriffstechnik sind sog. *CSRF-Tokens*. Diese helfen dabei sicherzustellen, dass nur solche zustandsändernden Requests akzeptiert werden, die wirklich von der legitimen Anwendung stammen, und nicht solche von fremden Websites. Der Anwendungsserver erzeugt hierzu für jede Benutzersession ein zufälliges, nicht erratbares Token. Dieses Token wird nur der legitimen Web-Anwendung bereitgestellt, z. B. im HTML-Code oder als Cookie. Bei jedem POST/PUT/DELETE-Request muss der Client dieses Token zusätzlich mitschicken. Der Server akzeptiert solche Requests nur, wenn das Token gültig ist und zur aktuellen Session passt.

In SpringBoot ist der CSRF-Schutz defaultmäßig aktiviert, d. h. Sie müssen serverseitig hierfür keine Einstellungen vornehmen. Wie Sie den CSRF-Schutz bei Bedarf ausschalten können, wird weiter unten gezeigt.

CSRF-Schutz in HTML-Formularen

Die folgende Abbildung zeigt den Seiten Quellcode des Registrierungsformulars (Thymeleaf-Variante) aus unserer Demo-Anwendung bei serverseitig aktiviertem CSRF-Schutz.

```

1 <!DOCTYPE html>
2 <html>
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>PaF Demo</title>
7   <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
8   <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js" defer></script>
9 </head>
10 <body class="bg-light">
11   <div class="container d-flex justify-content-center align-items-center vh-100">
12     <div class="card p-4 shadow" style="max-width: 400px; width: 100%;">
13       <h4 class="mb-3 text-center">User Sign Up</h4>
14       <form action="/process_signup" method="post" style="max-width: 600px; margin: 0 auto;">
15
16         <input type="hidden" name="_csrf"
17         value="HD38qNJfH9sMiJt_ycPf_HBAL7prbcq_eyB8dIDamJJh1TSzfQ-dyedoeMhuqMZ--7rzxZ5AoJfC6-SGhZJELC"/>
18
19       <div class="mb-3">
20         <label for="email" class="form-label">E-Mail Address</label>
21         <input type="email" class="form-control" id="email" placeholder="name@example.com"
22         required name="email" value="">
23       </div>
24       <div class="mb-3">
25         <label for="password" class="form-label">Password</label>
26         <input type="password" class="form-control" id="password" placeholder="....."
27         required name="password" value="">
28       </div>
29       <button type="submit" class="btn btn-primary w-100 mb-3">Sign Up</button>
30       <a href="/" class="btn btn-secondary w-100">Return to Menu</a>
31     </form>
32   </div>
33 </div>
34 </body>
35 </html>

```

Abb. 7.2: Seitenquelltext eines Formulars mit CSRF-Token

Wir erkennen im Formular-Code das versteckte Eingabefeld, das das vom Server generierte CSRF-Token enthält. Bei Ausführung der Submit-Aktion (hier: POST-Request an */process_signup*) wird das Token wieder mit an den Server geschickt. Dieser prüft vor der Verarbeitung des Requests, ob das mitgeschickte Token mit dem ursprünglich ausgelieferten übereinstimmt. Falls ja, gilt der Request als zulässig und wird verarbeitet. Falls nein, wird der Request wegen CSRF-Verdacht abgewiesen.

CSRF-Schutz bei JavaScript-Requests

Etwas anders verhält es sich, wenn der Request mittels JavaScript ausgelöst wird - egal ob clientseitig nur mit HTML/JS/CSS gearbeitet wird oder mit einem Web-Framework wie Vue oder Angular. In diesem Fall müssen wir selbst dafür sorgen, dass das vom Server erhaltene CSRF-Token in den nächsten Request eingebaut wird. Hierzu kann man zunächst entsprechende Meta-Attribute setzen, um sich das erhaltene Token zu merken, z. B. in einem Thymeleaf-Template:



Code

```

<meta name="csrfToken" th:content="${_csrf.token}" />
<meta name="csrfHeader" th:content="${_csrf.headerName}" />

```

Auf diese Meta-Attribute können wir dann zugreifen, wenn wir einen Request mit JS bauen:



Code

```
fetch("...", {
  method: ...
  headers: {
    [csrfHeader]: csrfToken,
    ...
  },
  body: ...
})
.then(...)
.catch(...);
```



Beim Austausch von WebSocket-Nachrichten muss man sich nicht um CSRF kümmern, d. h. in Ihrem Projektkontext spielt CSRF nur bei Kommunikation über HTTP eine Rolle, insb. bei HTML-Formularen und REST-APIs.

Deaktivieren des CSRF-Schutzes

Falls Sie trotz dieser Hinweise Probleme mit dem CSRF-Schutz haben, die Sie auch nach Rücksprache mit Ihrer Lehrperson nicht lösen können, bleibt als temporärer Ausweg - natürlich nicht für einen Produktivbetrieb der Anwendung, sondern nur im Entwicklungskontext der Lehrveranstaltung - das explizite Deaktivieren des CSRF-Schutzes.

Hierzu müssen Sie in der Klasse *WebSecurityConfig* (oder wie die entsprechende Konfigurationsklasse in Ihrem Projekt heißt) in der Methode zum Bauen der Filterkette eine Code-Zeile einfügen:



```
http
    .csrf(csrf -> csrf.disable())
    .authorizeHttpRequests(...
```

Nach dem Neustart wird der Server weder CSRF-Tokens erzeugen und ausliefern noch eingehende CSRF-Tokens prüfen. Es sollten demnach keine CSRF-bezogenen Fehlermeldungen mehr auftreten.

7.2 HTTPS-Verbindung einrichten

In unserer Demo-Anwendung wird ein clientseitig eingegebenes Passwort derzeit serverseitig gehashed (siehe Abschnitt Mit Spring Security den Zugang zur Web-Anwendung schützen), d. h. die Übertragung des Passworts vom Client zum Server erfolgt noch im Klartext. Um dies zu vermeiden und allgemein die Kommunikation zwischen Client und Server abzusichern, können wir diese auf HTTPS umstellen, wie im Folgen-

den beschrieben. Für den Produktivbetrieb einer Web-Anwendung ist dies heutzutage ohnehin quasi obligatorisch.

Im Ergebnis sollen unsere URLs wie folgt umgestellt werden:

- Thymeleaf: `http://localhost:8080` -> `https://localhost:8443`
- SPA (Vue): `http://localhost:5173` -> `https://localhost:5173`
- WebSocket: `ws://localhost:8080` -> `wss://localhost:8443`

Schritt 1: Lokales Zertifikat erstellen

Um mit HTTPS arbeiten zu können, müssen wir zunächst ein lokales Zertifikat erstellen. Dieses sollte möglichst vom Browser als vertrauenswürdig eingestuft werden (für lokale Entwicklung), um lästige Browser-Warnungen zu vermeiden. Daher verwenden wir an dieser Stelle `mkcert`. Nach der Installation führen wir im Projektordner unserer Demoanwendung folgende Befehle aus:



Code

```
# install local certification authority (CA)
mkcert -install

# create certificate for localhost
mkdir -p certs
cd certs
mkcert localhost 127.0.0.1 ::1
```

Das erzeugte lokale Zertifikat befindet sich anschließend in einer Datei "localhost+2.pem", der zugehörige Schlüssel in einer Datei "localhost+2-key.pem".

Schritt 2: Spring Boot auf HTTPS umstellen

Die im vorigen Schritt erzeugten .pem-Dateien machen wir Spring Boot nun über `application.properties` bekannt. Außerdem stellen wir den Server-Port auf 8443 um (Standard für HTTPS) und legen fest, dass Session-Cookies nur über HTTPS gesendet werden dürfen.



Code

```
server.port=8443

server.ssl.enabled=true
server.ssl.certificate=file:certs/localhost+2.pem
server.ssl.certificate-private-key=file:certs/localhost+2-key.pem

# Session-Cookie via HTTPS only
server.servlet.session.cookie.secure=true
```

Nach dem Neustart des Servers ist das Thymeleaf-Frontend unserer Anwendung nun über `https://localhost:8443` erreichbar.

Damit auch die Websocket-Kommunikation (im Chat-Modul) weiterhin funktioniert, müssen wir die Broker-URLs unserer STOMP-Clients in den Dateien *chat.js* und *chat-jwt.js* auf das neue Protokoll (wss statt ws) und den neuen Port (8443 statt 8080) umstellen:



Code

```
const stompClient = new StompJs.Client({
  //obsolete: brokerURL: 'ws://localhost:8080/websocket'
  brokerURL: 'wss://localhost:8443/websocket'
  //...
});
```

Schritt 3 (optional): Redirect von HTTP zu HTTPS einrichten

Optional können wir Aufrufe an <http://localhost:8080> an <https://localhost:8443> weiterleiten lassen. Hierzu legen wir eine entsprechende Konfigurationsklasse im security-Package unserer Demo-Anwendung an:

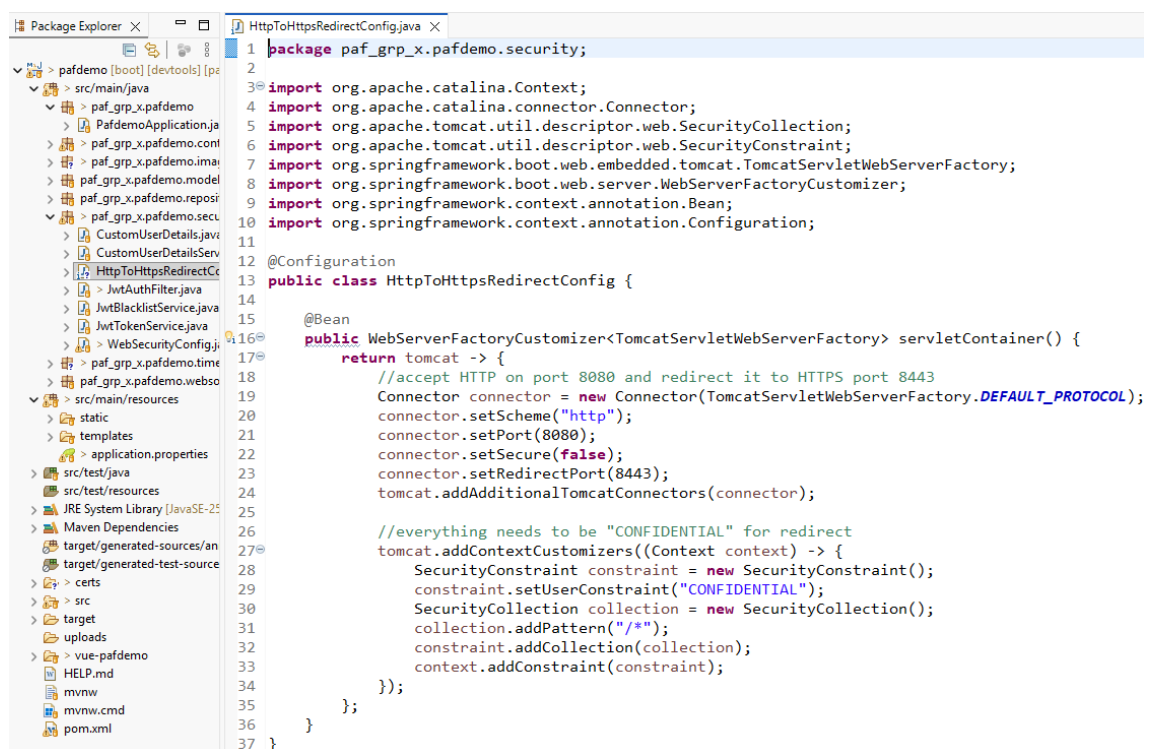


Abb. 7.3: Konfigurationsklasse zum Weiterleiten von HTTP zu HTTPS

Nach einem weiteren Neustart des Servers sollte die Weiterleitung nun funktionieren.

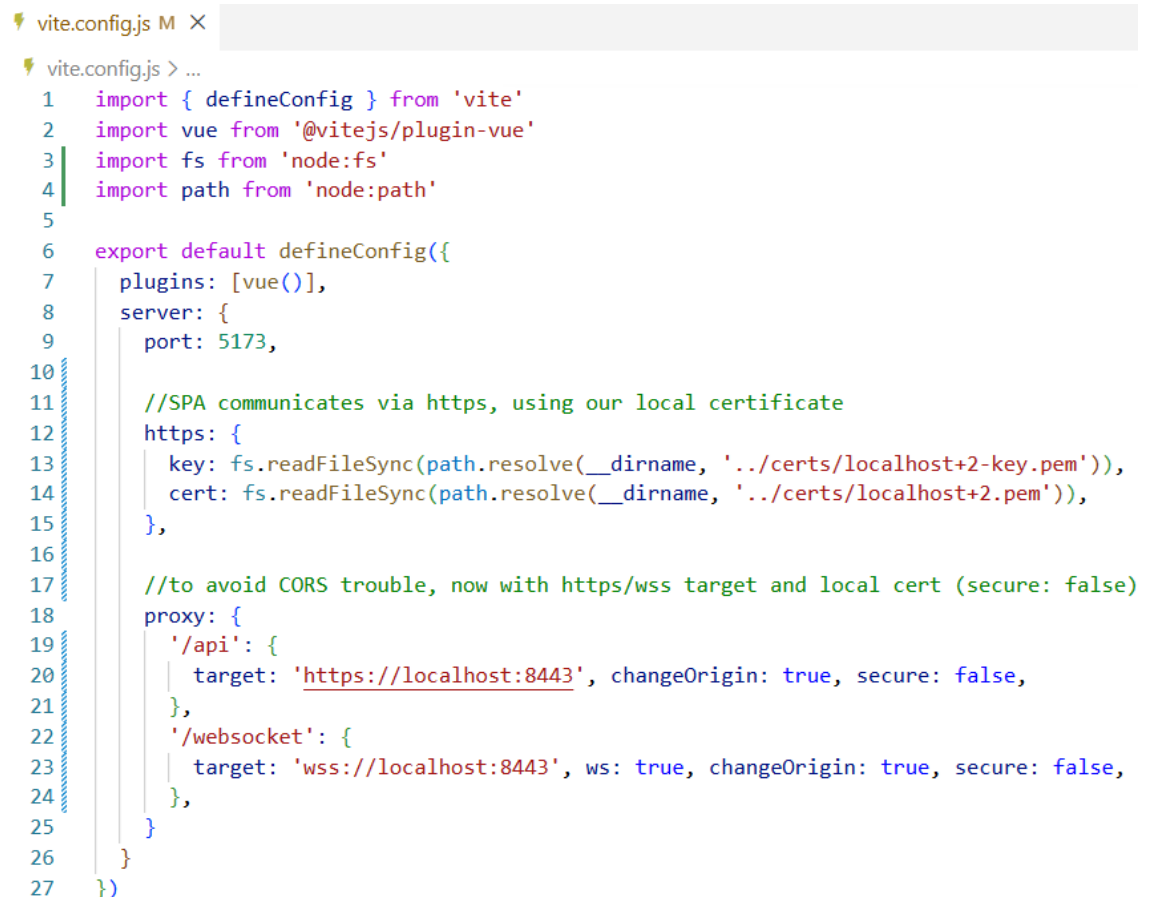
Schritt 4: SPA auf HTTPS umstellen

Da unser Server sowie die Cookies nun HTTPS erwarten, müssen wir auch unsere SPA über HTTPS laufen lassen. Ansonsten würde der (in *application.properties* als "secure" deklarierte) Session-Cookie nicht gesendet. Außerdem würde "Mixed Content" die

REST- und WebSocket-Aufrufe blockieren, da der Zugriff auf HTTP-Ressourcen von einer HTTPS-Seite aus untersagt ist.

Hierzu müssen wir die Datei `vite.config.js` erweitern, wie in der folgenden Abbildung gezeigt. In dieser Datei hatten wir im Abschnitt Serverseitige SPA-Anpassungen vornehmen) bereits einen Proxy angelegt, um SPA-seitig mit relativen URLs wie `/api` und `/websocket` arbeiten zu können und in der Folge CORS-Probleme zu vermeiden. In der Proxy-Spezifikation ändern wir die Targets auf die `https-` bzw. `wss-URLs` und ergänzen jeweils die `secure`-Eigenschaft mit dem Wert `false`, um auch nicht-öffentliche Zertifikate (nur `localhost`) zuzulassen.

Des Weiteren sorgen wir dafür, dass die SPA selbst über HTTPS ausgeliefert wird, indem wir auf unser in Schritt 1 erzeugtes (lokales) Zertifikat und den zugehörigen Schlüssel Bezug nehmen.



```
vite.config.js M X
vite.config.js > ...
1 import { defineConfig } from 'vite'
2 import vue from '@vitejs/plugin-vue'
3 import fs from 'node:fs'
4 import path from 'node:path'
5
6 export default defineConfig({
7   plugins: [vue()],
8   server: {
9     port: 5173,
10
11     //SPA communicates via https, using our local certificate
12     https: {
13       key: fs.readFileSync(path.resolve(__dirname, '../certs/localhost+2-key.pem')),
14       cert: fs.readFileSync(path.resolve(__dirname, '../certs/localhost+2.pem')),
15     },
16
17     //to avoid CORS trouble, now with https/wss target and local cert (secure: false)
18     proxy: {
19       '/api': {
20         target: 'https://localhost:8443', changeOrigin: true, secure: false,
21       },
22       '/websocket': {
23         target: 'wss://localhost:8443', ws: true, changeOrigin: true, secure: false,
24       },
25     }
26   }
27 })
```

Abb. 7.4: `vite.config.js`, erweitert um `https/wss`-Spezifikationen

Nun müssen wir noch darauf achten, dass im Quelltext unserer SPA - sofern dort nicht ohnehin schon der oben gezeigte Proxy genutzt wird - Aufrufe von `http/ws-URLs` konsequent auf `https/wss` umgestellt werden.

Schritt 5: CORS-Konfiguration an HTTPS anpassen

Damit der Server Zugriffe von `https://localhost:5173` nicht mit 403 (Forbidden) abweist, müssen wir im letzten Schritt unsere CORS-Konfiguration (Klasse `WebSecurityConfig`, Methode `corsConfigurationSource()`) anpassen bzw. erweitern:



Code

```
//obsolete: cors.setAllowedOrigins(List.of("http://localhost:5173"));  
cors.setAllowedOrigins(List.of("http://localhost:5173", "https://localhost:5173"));
```

Damit sollte nun das SPA-Frontend unserer Demo-Anwendung (nach dem üblichen Start per "npm run dev") über `https://localhost:5173` erreichbar sein und wie gewohnt funktionieren, inkl. Websocket-Aufrufen per `wss://`.



Hinweis

Es kann sein, dass Sie bei fetch- oder axios-Aufrufen auf Probleme stoßen, wenn Ihre Anwendung sessionbasiert authentifiziert. Achten Sie bitte in diesem Fall darauf, dass die Aufrufe `credentials: 'include'` (bei fetch) bzw. `withCredentials: true` (bei axios) enthalten.

7.3 Bilder zwischen Client und Server transferieren

Eine der Anforderungen an Ihr Projekt ist es, Bilder zwischen Client(s) und Server zu übertragen (siehe [Einführung und Organisatorisches](#)). Dies können zum Beispiel Profilbilder der User sein, alternativ auch gerne spielbezogene Bilder, zum Beispiel solche für Memory-Karten.

In diesem Kapitel demonstrieren wir anhand der Hello-World-Seite, wie das clientseitige Hochladen und spätere Abfragen von Bildern technisch umgesetzt werden kann.



Hinweis

Die Funktion zum Hochladen und Anzeigen von Bildern bauen wir in unsere SPA sowie in die JWT-basierte Variante unseres Thymeleaf-Clients ein. Die sessionbasierte Hello-World-Seite des Thymeleaf-Clients bleibt ohne Bilder.

Serverseitige Speicherung von Bildern

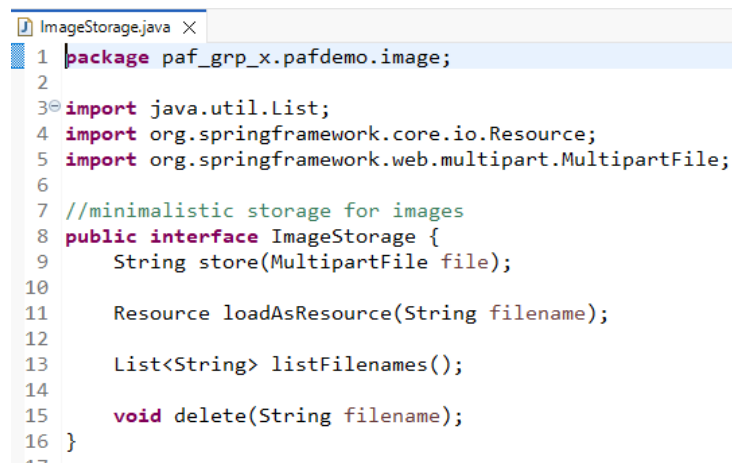
Wir wollen an dieser Stelle sauber zwischen Schnittstelle und Implementierung unterscheiden, da zur Speicherung von Bildern durchaus unterschiedliche Implementierungen denkbar sind. Beispielsweise könnten Bilder direkt in der Datenbank abgelegt und daraus gelesen werden. Im Folgenden werden wir hingegen eine Implementierung zeigen, die die Bilddateien in einem speziellen Serververzeichnis speichert. Der Verzeichnisname sowie die Upload-Limits hinterlegen wir in `application.properties`:



Code

```
#set upload limits
spring.servlet.multipart.max-file-size=5MB
spring.servlet.multipart.max-request-size=10MB
#set update directory
app.upload-dir=uploads
```

Nun spezifizieren wir das Interface, das wir *ImageStorage* nennen (siehe folgende Abbildung). Es stellt Methoden zum Speichern, Laden und Löschen von Bildern bereit. Zusätzlich gibt es eine Methode, die die Namen aller gespeicherten Bilddateien als Liste liefert.



```
ImageStorage.java X
1 package paf_grp_x.pafdemo.image;
2
3 import java.util.List;
4 import org.springframework.core.io.Resource;
5 import org.springframework.web.multipart.MultipartFile;
6
7 //minimalistic storage for images
8 public interface ImageStorage {
9     String store(MultipartFile file);
10
11     Resource loadAsResource(String filename);
12
13     List<String> listFileNames();
14
15     void delete(String filename);
16 }
```

Abb. 7.5: Interface ImageStorage

Das Interface *ImageStorage* wird von der Klasse *FileSystemImageStorage* implementiert. In deren Konstruktor legen wir das Upload-Verzeichnis an, sofern es noch nicht existiert. Den Verzeichnisnamen lesen wir per @Value-Annotation aus *application.properties* (siehe oben).

```
FileSystemImageStorage.java X
1 package paf_grp_x.pafdemo.image;
2
3 import org.springframework.beans.factory.annotation.Value;
16
17 //minimalistic storage for images
18 @Service
19 public class FileSystemImageStorage implements ImageStorage {
20
21     private final Path root;
22     private static final Set<String> ALLOWED = Set.of(
23         "image/jpeg", "image/png", "image/gif", "image/webp");
24
25     //use upload directory specified in application.properties
26 public FileSystemImageStorage(@Value("${app.upload-dir:uploads}") String uploadDir) {
27     this.root = Paths.get(uploadDir).toAbsolutePath().normalize();
28     try {
29         //create directory if it does not already exist
30         Files.createDirectories(root);
31     } catch (IOException e) {
32         throw new RuntimeException(e);
33     }
34 }
```

Abb. 7.6: Klasse FileSystemImageStorage, Teil 1

Als nächstes implementieren wir die Methode *store()*. Sie bekommt ein Datei-Objekt vom Typ *MultipartFile* übergeben. Nach der Sicherstellung, dass die Datei nicht leer ist und einen der erlaubten Typen hat (aktuell: jpeg, png, gif, webp), wird versucht, sie einzulesen. Bei Erfolg wird eine UUID als Dateiname generiert und um die passende Dateiergänzung ergänzt.

```
FileSystemImageStorage.java X
36  @Override
37  public String store(MultipartFile file) {
38      if (file.isEmpty()) {
39          throw new IllegalArgumentException("File is empty.");
40      }
41
42      String contentType = file.getContentType();
43
44      if (contentType == null || !ALLOWED.contains(contentType)) {
45          throw new IllegalArgumentException("Image files only (jpeg, png, gif, webp).");
46      }
47
48      //try to read file as image
49      try {
50          if (ImageIO.read(file.getInputStream()) == null) {
51              throw new IllegalArgumentException("Invalid image file.");
52          }
53      } catch (IOException e) {
54          throw new IllegalArgumentException("Image check failed.");
55      }
56
57      //set file extension
58      String ext = switch (contentType) {
59          case "image/jpeg" -> ".jpg";
60          case "image/png" -> ".png";
61          case "image/gif" -> ".gif";
62          case "image/webp" -> ".webp";
63          default -> "";
64      };
65
66      //copy file to target path
67      String filename = UUID.randomUUID() + ext;
68      Path target = root.resolve(StringUtils.cleanPath(filename));
69      try {
70          Files.copy(file.getInputStream(), target, StandardCopyOption.REPLACE_EXISTING);
71      } catch (IOException e) {
72          throw new RuntimeException("File save failed", e);
73      }
74      return filename; //may be fetched later by /media/{filename}
75  }
```

Abb. 7.7: Klasse FileSystemImageStorage, Teil 2

Die übrigen Methodenimplementierungen fallen vergleichsweise kurz aus. Sie verwenden jeweils die Instanzvariable `_root_`, um die Datei mit dem übergebenen Namen zu lesen oder zu löschen bzw. um eine Liste aller Dateinamen zusammenzustellen.

```

FileSystemImageStorage.java X
77  @Override
78  public Resource loadAsResource(String filename) {
79      Path p = root.resolve(filename).normalize();
80
81      if (!p.startsWith(root)) throw new SecurityException("Path outside upload folder.");
82      return new FileSystemResource(p);
83  }
84
85  @Override
86  public List<String> listFileNames() {
87      try (var stream = Files.list(root)) {
88          return stream
89              .filter(Files::isRegularFile)
90              .map(p -> p.getFileName().toString())
91              .toList();
92      } catch (IOException e) {
93          throw new RuntimeException("Listing failed", e);
94      }
95  }
96
97  @Override
98  public void delete(String filename) {
99      try {
100         Files.deleteIfExists(root.resolve(filename));
101     } catch (IOException ignored) {}
102 }
103 }

```

Abb. 7.8: Klasse FileSystemImageStorage, Teil 3

Damit REST-Requests an die URL bzw. den Endpunkt `/media/**` aus dem Upload-Verzeichnis bedient werden können, fügen wir unserer Anwendung eine weitere Konfigurationsklasse hinzu. Diese heißt (nicht zwingend) *WebConfig* und implementiert das Interface *WebMvcConfigurer*. In der überschriebenen Methode *addResourceHandlers()* wird genau die o. g. Verknüpfung zwischen dem o. g. Endpunkt `/media/**` und dem Speicherort (`uploadDir`) hergestellt.

```

WebConfig.java X
1  package paf_grp_x.pafdemo.image;
2
3  import org.springframework.beans.factory.annotation.Value;
4  import org.springframework.context.annotation.Configuration;
5  import org.springframework.web.servlet.config.annotation.*;
6
7  @Configuration
8  public class WebConfig implements WebMvcConfigurer {
9
10     //get uploadDir from application.properties
11     @Value("${app.upload-dir:uploads}")
12     String uploadDir;
13
14     //any request to URL /media/** shall be served from uploadDir
15     @Override
16     public void addResourceHandlers(ResourceHandlerRegistry registry) {
17         registry
18             .addResourceHandler("/media/**")
19             .addResourceLocations("file:" + uploadDir + "/")
20             .setCachePeriod(3600); //1h cache (using Cache-Control headers)
21     }
22 }

```

Abb. 7.9: Klasse WebConfig

REST-Endpunkte zum Upload und zur Auslieferung von Bildern

Damit der Bildspeicher von Clients genutzt werden kann, erweitern wir den *PafdemoApiController* um je einen POST- und einen GET-Endpunkt `/images`. Der POST-Endpunkt sorgt für die Speicherung der als Parameter übergebenen (hochgeladenen) Datei im oben definierten *ImageStorage* und gibt den neu generierten Dateinamen sowie die relative URL für den lesenden Zugriff zurück.

Der GET-Endpunkt liefert eine Liste aller serverseitig gespeicherten Dateinamen und der zugehörigen relativen URLs. Hierüber können die eigentlichen Bilddateien im Anschluss angefragt werden.

```
PafdemoApiController.java ×
1 package paf_grp_x.pafdemo.controller;
2
3 import java.io.IOException;
17
18 @RestController
19 @RequestMapping("/api")
20 public class PafdemoApiController {
21
22     private final ImageStorage storage;
23
24     public PafdemoApiController(ImageStorage storage) { this.storage = storage; }
25
26     //upload new image
27     @PostMapping(path = "/images", consumes = MediaType.MULTIPART_FORM_DATA_VALUE)
28     public Map<String, String> upload(@RequestParam MultipartFile file) {
29         String name = storage.store(file);
30         return Map.of(
31             "filename", name,
32             "url", "/media/" + name
33         );
34     }
35
36     //get all uploaded images
37     @GetMapping("/images")
38     public List<Map<String, String>> list() throws IOException {
39         return storage.listFiles().stream()
40             .map(name -> Map.of(
41                 "filename", name,
42                 "url", "/media/" + name
43             ))
44             .toList();
45     }
}
```

Abb. 7.10: `/images`-Endpunkte in der Klasse *PafdemoApiController*

Als nächstes passen wir die Hello-World-Views unserer Anwendung derart an, dass Bilder hochgeladen und angezeigt werden können. Wir beginnen mit dem Thymeleaf-Client in der JWT-Variante.

Anpassung der Thymeleaf-View (JWT-Variante)

Unterhalb des *div* mit der id *result*, in das die Hello-World-Nachricht geladen wird, fügen wir ein Formular ein. Dieses besteht aus einem File Chooser für Bilddateien und einem Upload-Button. Unterhalb des Formulars bauen wir ein Grid ein, in das die serverseitig gespeicherten Bilder geladen werden.

```

1 <!DOCTYPE html>
2 <html lang="de" xmlns:th="http://www.thymeleaf.org">
3 <head>
4   <meta charset="utf-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1">
6   <title>PaF Demo</title>
7 </head>
8 <body>
9   <div id="result">Loading...</div>
10  <br/>
11  <form id="uploadForm" enctype="multipart/form-data">
12    <input id="fileInput" type="file" name="file" accept="image/*" required>
13    <button type="submit">Upload image</button>
14  </form>
15  <br/>
16
17  <style>
18    .grid { display:grid;
19            grid-template-columns: repeat(auto-fill, minmax(160px,1fr)); gap:12px; }
20    img { width:100%; height:140px; object-fit:cover; border-radius:8px; }
21  </style>
22
23  <div class="grid" id="grid">
24    <div th:each="src: ${images}">
25      
26    </div>
27  </div>

```

Abb. 7.11: Template hello-jwt.html mit Image-Upload und -Anzeige, Teil 1

Im script-Teil der View sorgen wir mit der Funktion *setUploadEnabled()* dafür, dass die Formular-Buttons aktiv und inaktiv geschaltet werden können. Das Hinzufügen von Bildern zum Grid übernehmen die Hilfsfunktionen *appendMany()* und *prependOne()*. Beide bedienen sich der Funktion *makeImgTile()*, die dem Grid auf DOM-Ebene eine neue Bildkachel hinzufügt, deren *img.src*-Attribut den Wert der übergebenen Bild-URL bekommt.

```

hello-jwt.html X
20
29 <script>
30  (() => {
31    const grid = document.getElementById('grid');
32    const form = document.getElementById('uploadForm');
33    const input = document.getElementById('fileInput');
34    const button = form.querySelector('button[type="submit"]');
35
36    //helpers for image handling
37    const setUploadEnabled = (on) => { //disable buttons if not authenticated
38      input.disabled = !on;
39      button.disabled = !on;
40      form.style.opacity = on ? '1' : '0.5';
41    };
42
43    const makeImgTile = (url) => {
44      const wrap = document.createElement('div');
45      const img = document.createElement('img');
46      img.src = url;
47      img.alt = 'upload';
48      img.style.width = '100%';
49      img.style.height = '140px';
50      img.style.objectFit = 'cover';
51      img.style.borderRadius = '8px';
52      wrap.appendChild(img);
53      return wrap;
54    };
55
56    const appendMany = (items) => {
57      if (!grid || !items?.length) return;
58      const fragment = document.createDocumentFragment();
59      for (const it of items) fragment.appendChild(makeImgTile(it.url));
60      grid.appendChild(fragment);
61    };
62
63    const prependOne = (item) => {
64      if (!grid || !item) return;
65      grid.prepend(makeImgTile(item.url));
66    };

```

Abb. 7.12: Template hello-jwt.html mit Image-Upload und -Anzeige, Teil 2

Beim initialen Laden der View werden zunächst die Formular-Buttons deaktiviert, da zu diesem Zeitpunkt noch nicht sichergestellt ist, dass der User authentifiziert ist. Bei Vorliegen eines JWT im Session Storage wird - wie schon bisher - der REST-Endpoint `/api/hello` abgefragt, um die Hello-World-Nachricht zu laden. Danach fügen wir ein weiteres `fetch()` ein, diesmal an den REST-Endpoint `/api/images`. Die vom *PafdemoApiController* (siehe oben) gelieferte Liste von Bild-URLs wird zwecks Anzeige an `appendMany()` übergeben.

```
hello-jwt.html X
68 //initial load
69 (async () => {
70   setUploadEnabled(false); //disable buttons as long as auth has not been approved
71
72   const box = document.getElementById('result');
73   const token = sessionStorage.getItem('accessToken'); //from JWT login
74
75   if (!token) {
76     box.innerHTML = "<div>No JWT found. Please <a href='/login'>sign in via JWT</a>";
77     return;
78   }
79
80   //hello message
81   try {
82     const resp = await fetch('/api/hello', {
83       headers: { 'Authorization': 'Bearer ' + token }
84     });
85     if (resp.status === 401 || resp.status === 403) {
86       sessionStorage.removeItem('accessToken');
87       box.innerHTML = "<div>Error: " + resp.status + "; your session may have expired";
88       return;
89     }
90     // /api/hello delivers text/html
91     const html = await resp.text();
92     box.innerHTML = html;
93     setUploadEnabled(true); //everything ok -> enable buttons
94
95   } catch (e) {
96     box.innerHTML = "<div>Network error: " + e + "</div>";
97   }
98
99   //images
100   try {
101     const resp = await fetch('/api/images', {
102       headers: { 'Authorization': 'Bearer ' + token }
103     });
104     if (resp.ok) appendMany(await resp.json()); //[{filename, url}, ...]
105   } catch (e) {
106     console.error('load images failed', e);
107   }
108 })();
```

Abb. 7.13: Template hello-jwt.html mit Image-Upload und -Anzeige, Teil 3

Schließlich spezifizieren wir die Submit-Aktion des Upload-Formulars. Nachdem wir wie üblich das Default-Verhalten unterbunden haben, bauen wir einen POST-Request an `/api/images` (natürlich inkl. JWT). In dessen Body schicken wir ein `FormData`-Objekt mit, dem wir zuvor die im File Chooser ausgewählte Bilddatei hinzugefügt haben. In dem Fall, dass der Server unsere Upload-Anfrage mit OK (200) beantwortet, lesen wir die vom Server gelieferte URL aus der Response aus und fügen das zugehörige Bild mittels `prependOne()` ganz vorne in das Grid ein. Ein abschließendes Formular-Reset bereitet den Upload eines weiteren Bilds vor.

```

109
110 //upload submit
111 form.addEventListener('submit', async (e) => {
112   e.preventDefault();
113
114   const token = sessionStorage.getItem('accessToken'); //from JWT login
115   const fd = new FormData();
116   if (input.files?.[0]) fd.append('file', input.files[0]);
117
118   try {
119     const resp = await fetch('/api/images', {
120       method: 'POST',
121       body: fd,
122       headers: { 'Authorization': 'Bearer ' + token }
123       //do not set 'Content-Type' - done by browser when using FormData
124     });
125
126     if (!resp.ok) {
127       const text = await resp.text().catch(() => '');
128       alert(`Upload failed: ${resp.status} ${resp.statusText}\n${text}`);
129       return;
130     }
131
132     const data = await resp.json(); //{ filename, url }
133     prependOne(data); //show uploaded image immediately
134     form.reset();
135   } catch (e) {
136     alert('Network error: ' + e);
137   }
138 });
139 })();
140 </script>
141 </body>
142 </html>

```

Abb. 7.14: Template hello-jwt.html mit Image-Upload und -Anzeige, Teil 4

Anpassung der SPA-View

Die Änderungen, die wir in der Sicht Hello.vue vornehmen müssen, um Bild-Upload und -Anzeige zu ermöglichen, ähneln zwangsläufig denen aus der Thymeleaf-Variante. Aufgrund der reicheren Vue.js-Syntax fällt der zugehörige Quellcode aber kompakter aus.

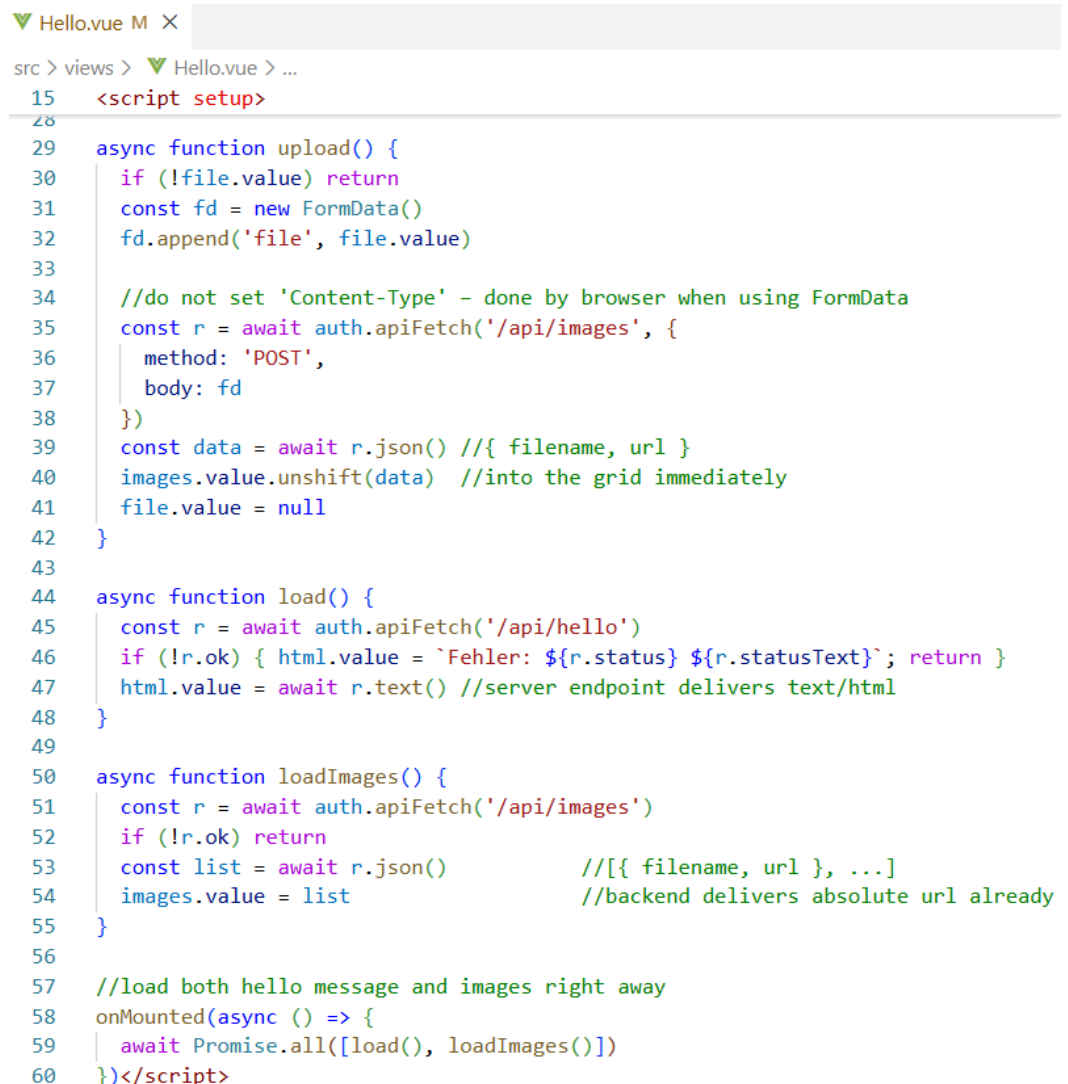
Im template-Teil der Sicht ergänzen wir wieder den File Chooser, den Upload-Button sowie das Bilder-Grid. Die @change-Direktive im File Chooser veranlasst den Aufruf von *onPick()*. Diese Funktion setzt die ausgewählte Bilddatei in die File-Variable, was wiederum den Upload-Button aktiviert.

```
▼ Hello.vue M X
src > views > ▼ Hello.vue > {} script setup > upload
1  <template>
2    <div v-html="html"></div>
3    <br/>
4    <div>
5      <input type="file" @change="onPick" accept="image/*">
6      <button :disabled="!file" @click="upload">Upload image</button>
7      <div v-if="images.length"
8        style="margin-top:1rem; display:grid; grid-template-columns:repeat(auto-fill,minmax(160px,1fr)); gap:12px;"
9        
12    </div>
13  </template>
14
15  <script setup>
16    import { onMounted, ref } from 'vue'
17    import { auth } from '../auth'
18    import { useRouter } from 'vue-router'
19    const router = useRouter()
20    const html = ref('Loading...')
21
22    const file = ref(null)
23    const images = ref([])
24
25    function onPick(e) {
26      | file.value = e.target.files?.[0] ?? null
27    }
```

Abb. 7.15: View Hello.vue mit Image-Upload und -Anzeige, Teil 1

Ein Klick auf den Upload-Button baut - analog zur Thymeleaf-View - den POST-Request mit dem FormData-Objekt im Body. Die Anweisung *images.value.unshift(data)* übernimmt das Einfügen des hochgeladenen Bildes am Anfang des Grid.

Die Dateinamen und URLs der bereits auf dem Server gespeicherten Bilder werden über *loadImages()* geladen. *images.value = list* fügt die zugehörigen Bilder in das Grid ein. Über den Lifecycle-Hook *onMounted()* spezifizieren wir, dass sowohl die Hello-World-Nachricht (über *load()*) als auch die Bilderliste gleich zu Beginn - also unmittelbar nach dem Einfügen der Sicht in den DOM-Baum der Anwendung - geladen werden.



```

15 <script setup>
28
29 async function upload() {
30   if (!file.value) return
31   const fd = new FormData()
32   fd.append('file', file.value)
33
34   //do not set 'Content-Type' - done by browser when using FormData
35   const r = await auth.apiFetch('/api/images', {
36     method: 'POST',
37     body: fd
38   })
39   const data = await r.json() //{ filename, url }
40   images.value.unshift(data) //into the grid immediately
41   file.value = null
42 }
43
44 async function load() {
45   const r = await auth.apiFetch('/api/hello')
46   if (!r.ok) { html.value = `Fehler: ${r.status} ${r.statusText}`; return }
47   html.value = await r.text() //server endpoint delivers text/html
48 }
49
50 async function loadImages() {
51   const r = await auth.apiFetch('/api/images')
52   if (!r.ok) return
53   const list = await r.json()           //{ filename, url }, ...}
54   images.value = list                  //backend delivers absolute url already
55 }
56
57 //load both hello message and images right away
58 onMounted(async () => {
59   await Promise.all([load(), loadImages()])
60 })</script>

```

Abb. 7.16: View Hello.vue mit Image-Upload und -Anzeige, Teil 2

Somit haben wir die Implementierung des Bild-Transfers anhand der relativ sinnfreien Hello-World-Seite unserer Demo-Anwendung abgeschlossen. Es sollte Ihnen nun möglich sein, die gezeigten Konzepte auf sinnvollere Use Cases im Rahmen Ihres Spielprojekts zu übertragen, z. B. auf den Upload eines Profilbilds im Rahmen der Registrierung eines neuen Users bzw. auf das Laden eines Profilbilds nach einer erfolgreichen Authentifizierung.

7.4 Mit Countdowns arbeiten

Mit Countdowns lassen sich Spiele aktionsreicher gestalten - so hat man als Spieler*in nur eine begrenzte Zeit, um z. B. einen Spielzug zu machen (rundenbasierte Brett- oder Kartenspiele), ein Level zu beenden (Action-Spiele) oder eine gegebene Frage zu beantworten (Quizspiele). Außerdem haben Countdowns den angenehmen Nebeneffekt, dass sie implizit eine Auflösung von einseitiger Inaktivität bieten. Kommt z. B. von einem Player binnen der gegebenen Frist kein Spielzug oder keine Antwort,

kann der Server das Spiel beenden und den inaktiven Player zum Verlierer erklären. Somit werden "Hängepartien" vermieden und Frust auf Seiten der aktiv Mitspielenden vermieden.

Da unsere Demo-Anwendung keine Spielfunktionalität enthält, weichen wir auf einen anderen Use Case aus, und zwar sollen Chat-Nachrichten erst nach einem kurzen Countdown versendet und angezeigt werden. Dies ist zwar kein sinnvolles Feature, zeigt aber die Grundprinzipien der technischen Umsetzung von Countdowns.

Die Clients abonnieren dazu einen websocketbasierten Countdown-Kanal des Servers. Sobald beim Server eine neue Chat-Nachricht eintrifft, startet er einen neuen Countdown. Über den Countdown-Kanal sendet der Server sekundlich eine WebSocket-Nachricht an die Clients, damit diese ihre Views synchron aktualisieren können. Erst wenn der Zähler auf 0 geht, zeigen die Clients die Nachricht an. Ob die Nachricht tatsächlich verzögert an die Clients ausgeliefert wird (wie im Folgenden gezeigt) oder ob diese nur deren Anzeige verzögern, kann individuell entschieden werden.



Dieses Kapitel setzt Kenntnisse der nebenläufigen Programmierung voraus, die sich Java-seitig in Schlüsselworten wie *synchronized* und *volatile* sowie in Klassen wie *ScheduledExecutorService*, *ScheduledFuture*, *Thread* und *AtomicInteger* widerspiegeln.



Da es hier nur um eine beispielhafte Implementierung von Countdowns geht, setzen wir diese Funktionalität clientseitig nur in der SPA um. Die in Thymeleaf gebauten Chat-Funktionen (sessionbasiert und JWT-basiert) rufen weiterhin den bisherigen /chat-Endpunkt auf (ohne Countdown).

Countdown-Service (serverseitig)

Die serverseitigen Timer-Jobs realisieren wir in der neuen Service-Klasse *CountdownService* (siehe die beiden folgenden Abbildungen), die wir später in unseren ChatController einbauen werden. Ein CountdownService-Objekt verfügt über einen Scheduler vom Typ *ScheduledExecutorService*. Dieser wiederum spezifiziert einen neuen Thread, der zeitgesteuert ist.

```

1 package paf_grp_x.pafdemo.timer;
2
3 import java.util.concurrent.*;
4
5
6 /** Global countdown for the chat component. With every new chat message,
7  * the old countdown (if any) is stopped and a new one is started.
8  */
9
10 @Service
11 public class CountdownService {
12
13     private final SimpMessagingTemplate messaging;
14     private final ScheduledExecutorService scheduler = //for periodic tasks
15         Executors.newSingleThreadScheduledExecutor(r -> {
16             Thread t = new Thread(r, "countdown-scheduler"); //new named thread
17             t.setDaemon(true); //don't block JVM shutdown
18             return t;
19         });
20
21     private ScheduledFuture<?> running; //references the current timer task
22     private final AtomicInteger remaining = new AtomicInteger(-1); //thread-safe counter
23     private volatile ChatMessage pending; //pending message (visible across threads)
24
25     public CountdownService(SimpMessagingTemplate messaging) {
26         this.messaging = messaging;
27     }
28
29     //starts new countdown for message msg
30     public synchronized void start(int seconds, ChatMessage msg) {
31         cancelLocked(); //clean stop of old countdown, if any
32         pending = msg;
33         remaining.set(seconds);
34         //tick immediately, then every second
35         running = scheduler.scheduleAtFixedRate(() -> tick(), 0, 1, TimeUnit.SECONDS);
36     }
37
38     //...
39 }

```

Abb. 7.17: CountdownService, Teil 1

Ein Aufruf der Methode `start()` (wird später aus dem ChatController heraus aufgerufen) startet über den o. g. Scheduler einen Timer, der sekundlich die Methode `tick()` aufruft. Diese wiederum dekrementiert den Sekundenzähler des Countdowns und sendet den aktuellen Countdown-Wert per `convertAndSend()` an alle Clients, die den Kanal `"/topic/countdown"` abonniert haben. Erreicht der Sekundenzähler den Wert 0, wird auf analoge Weise die eigentliche Nachricht versendet und der Timer gestoppt.

Bitte beachten Sie, dass zu Beginn von `start()` ein eventuell bereits laufender Countdown gestoppt wird. Es läuft also zu jedem Zeitpunkt maximal ein Countdown. Separate Countdowns für eine Abfolge von Nachrichten könnte man über eine Queue realisieren.

```

CountdownService.java X
41 //action to be executed every second (tick)
42 private void tick() {
43     int r = remaining.getAndDecrement(); //returns current value, then -1
44     if (r < 0) return; //already stopped -> do nothing
45     //broadcast tick to clients
46     messaging.convertAndSend("/topic/countdown", new TickPayload(r));
47     if (r == 0) {
48         //release message and stop countdown
49         var msg = pending;
50         pending = null; //avoid repeated broadcasts
51         messaging.convertAndSend("/topic/messages", msg);
52         cancel(); //stop and reset task
53     }
54 }
55
56 public void cancel() {
57     synchronized (this) { cancelLocked(); }
58 }
59
60 private void cancelLocked() {
61     if (running != null) {
62         running.cancel(true); //stop the running task
63         running = null; //reset task
64         remaining.set(-1);
65         pending = null;
66     }
67 }
68
69 public record TickPayload(int remaining) {} //immutable DTO
70 }

```

Abb. 7.18: CountdownService, Teil 2

Anpassungen im ChatController (serverseitig)

Dem ChatController machen wir den CountdownService via Dependency Injection bekannt, wie im folgenden Listing gezeigt. Nun fügen wir einen Websocket-Endpunkt `/chat-delayed` hinzu, der - anders als der Endpunkt `/chat` - nicht mehr die neue Chat-Nachricht per `@SendTo` verschickt, sondern stattdessen über den CountdownService einen neuen 3-sekündigen Countdown startet. Der Rückgabebetyp von `sendDelayedChatMessage()` ist daher `void`. Das eigentliche Versenden der Chat-Nachricht findet nun verzögert über `tick()` statt, wie im vorigen Abschnitt beschrieben.



Code

```

@Controller
public class ChatController {

    private final CountdownService countdown;

    public ChatController(CountdownService countdown) {
        this.countdown = countdown;
    }

    // ... (endpoint /chat)

    //same as /chat, but with delayed delivery after countdown
    @MessageMapping("/chat-delayed")
    public void sendDelayedChatMessage(ChatMessage message, Principal principal)
    {
        String content = message.getContent();
        content = principal.getName() + ": " + content;
        message.setContent(content);
        countdown.start(3, message); //3 sec countdown
    }
}

```

```
}
}
```

Anpassungen in der ChatView (clientseitig)

Die Anzeige eines laufenden Countdown im Vue-Frontend realisieren wir über ein *div* mit Direktive *v-show*, das die verbleibenden Sekunden in einer Meldung anzeigt, sofern gerade ein Countdown aktiv ist.



Code

```
<template>

  <div class="alert alert-info" v-show="countdownActive">
    Message will be published in <strong>{{ countdown }}</strong> seconds.
  </div>

</template>
```

Die Variablen *countdown* und *countdownActive* deklarieren wir im script-Teil der ChatView-Komponente. Des Weiteren sorgen wir dafür, dass beim Websocket-Verbindungsaufbau der neue Nachrichtenkanal *"/topic/countdown"* abonniert wird. Beim Eintreffen einer Nachricht wird diese geparsed und der enthaltene Sekundenwert in die Variable *countdown* geschrieben. Aufgrund des reaktiven Verhaltens der View wird die Anzeige (siehe oben) aktualisiert, sobald sich der Wert von *countdown* ändert.

Schließlich müssen wir noch sicherstellen, dass ein eventuell laufender Countdown beim Schließen der Websocket-Verbindung deaktiviert wird - siehe *client.onWebSocketClose* und *disconnect()* - und dass *sendMessage()* den Endpunkt *"/app/chat-delayed"* anstelle von *"/app/chat"* aufruft.



Code

```
const countdown = ref(-1)    // -1 = inactive, >=0 = active
const countdownActive = computed(() => countdown.value >= 0)
//...

function connect() {
  //...
  client.onConnect = () => {
    //...
    //subscribe to global countdown
    client.subscribe('/topic/countdown', (frame) => {
      try {
        const tick = JSON.parse(frame.body) // { remaining: number }
        countdown.value = tick.remaining
      } catch {
        const n = Number(frame.body)
        countdown.value = isNaN(n) ? -1 : n
      }
    })
  }
  //...
```

```
}  
//...  
client.onWebSocketClose = () => {  
  connected.value = false  
  countdown.value = -1 //-> inactive  
}  
//...  
}  
  
function disconnect() {  
  client?.deactivate()  
  connected.value = false  
  countdown.value = -1 //-> inactive  
}  
  
function sendMessage() {  
  //...  
  client.publish({ destination: '/app/chat-delayed', body: JSON.stringify({ content: t }) })  
  //...  
}
```

Damit ist die Implementierung der Countdown-basierten Chat-Nachrichten abgeschlossen und präsentiert sich im Browser wie unten gezeigt. Trotz des recht sinnfreien Use Case sollte das Grundprinzip klar geworden sein und sich auf sinnvollere (Spiele-)Projekte übertragen lassen.

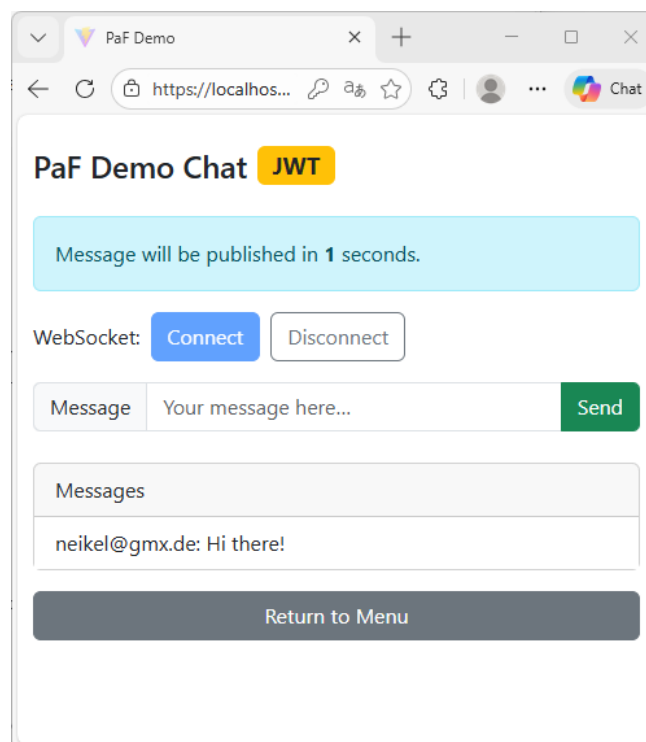


Abb. 7.19: Chat-Funktion mit Countdown in der SPA

7.5 Auf User-seitige Abbrüche reagieren

Zu den Kriterien einer robusten Web-Anwendung gehört es, dass sie auf definierte Weise mit User-seitigen Abbrüchen umgeht. Dazu gehört z. B. das Schließen eines Browser-Tab, das Neuladen einer Seite (in einer SPA) oder das interne Navigieren zu einer anderen View. Je nach Use Case kann es erforderlich sein, auf solche Ereignisse zu reagieren und bestimmte Aktionen durchzuführen. Versucht z. B. ein User, während eines laufenden Spiels den Browser-Tab zu schließen, wäre eine Abfrage sinnvoll, ob die Seite wirklich verlassen werden soll. Sofern der User dies bestätigt und damit das Spiel verlässt, sollten andere Mitspielende darüber informiert werden. Außerdem könnten serverseitige Aktionen notwendig werden, z. B. der Abbruch des Spiels und ggf. das Löschen des laufenden Spiels aus der Datenbank (Rollback). Das Anstoßen solcher serverseitigen Aktionen kann dadurch erschwert werden, dass der Browser insb. beim Schließen oder Reload von Tabs asynchrone Aufrufe nicht mehr sauber zu Ende ausführt. Für solche Fälle sind spezielle Lösungen erforderlich, die wir in diesem Abschnitt beleuchten werden.

Wir behandeln User-seitige Abbrüche am Beispiel der Chat-Funktion unserer Demo-Anwendung, und zwar ausschließlich in der SPA-Variante. In den Thymeleaf-Varianten können die gezeigten Ansätze in ähnlicher Form implementiert werden.

Reaktion auf die Unterbrechung einer WebSocket-Verbindung

Im Folgenden betrachten wir den Fall, dass ein User den Chat verlässt, sei es durch interne Navigation (Rückkehr zum Menü) oder durch Schließen/Neuladen des zugehörigen Browser-Tab. Um die anderen Chat-Teilnehmenden darüber zu informieren, können wir den Umstand ausnutzen, dass beim Verlassen des Chat die WebSocket-Verbindung beendet wird. Dies löst serverseitig das Ereignis *SessionDisconnectionEvent* aus.

Die in der folgenden Abbildung gezeigte Klasse *PresenceEvents* implementiert das Spring-Interface *ApplicationListener* für das *SessionDisconnectionEvent*. Die überschriebene Methode *onApplicationEvent()* informiert alle Clients, die den WebSocket-Kanal */topic/presence* abonniert haben, über den Ausstieg des Users aus dem Chat. Der Username wird dabei dem Event-Objekt entnommen. An dieser Stelle könnte man natürlich auch noch weitere serverseitige Aktionen zum "Aufräumen" ausführen, z. B. Datenbankoperationen (Rollback, Online-Status, ...).

```

PresenceEvents.java X
1 package paf_grp_x.pafdemo.websocket;
2
3 import java.util.Map;
4
5 import org.springframework.context.ApplicationListener;
6 import org.springframework.messaging.simp.SimpMessagingTemplate;
7 import org.springframework.stereotype.Component;
8 import org.springframework.web.socket.messaging.SessionDisconnectEvent;
9
10 //When a client loses its websocket connection, Spring Boot fires a SessionDisconnectEvent.
11 //This component informs subscribers of /topic/presence that the client has left (the chat).
12 @Component
13 public class PresenceEvents implements ApplicationListener<SessionDisconnectEvent> {
14
15     private final SimpMessagingTemplate messaging;
16
17     public PresenceEvents(SimpMessagingTemplate messaging) {
18         this.messaging = messaging;
19     }
20
21     @Override
22     public void onApplicationEvent(SessionDisconnectEvent e) {
23         String user = (e.getUser() != null) ? e.getUser().getName() : "unknown";
24         messaging.convertAndSend("/topic/presence",
25             Map.of("action", "left", "user", user, "at", System.currentTimeMillis()));
26     }
27 }

```

Abb. 7.20: Klasse PresenceEvents

Das Abonnement des Kanals `/topic/presence` nehmen wir in die `onConnect`-Funktion des `StompJs`-Clients auf (Datei `Chat.vue`). Eingehende Nachrichten werden geparkt und in aufbereiteter Form an die Liste der Chat-Nachrichten angehängt: `messages.value.push(...)`



Code

```

client.onConnect = () => {
    //...
    //subscribe to presence events (i.e. other user has left the chat)
    client.subscribe('/topic/presence', (frame) => {
        try {
            const event = JSON.parse(frame.body); // {action:"left", user:"...",
at:...}

            const date = new Date(event.at).toISOString();
            messages.value.push(`${event.user} has left the chat at ${date}.`);
        } catch {
            messages.value.push('A user has left the chat.');
```

Damit haben wir eine leichtgewichtige Lösung für die Reaktion auf userseitige Abbrüche gebaut. Es mag aber Situationen geben, in denen wir keine `WebSocket`-Verbindung haben, auf deren Unterbrechung wir reagieren können. Des Weiteren möchten wir eine Abfrage vorsehen, die ein User bestätigen muss, um wirklich einen Browser-Tab zu schließen bzw. neu zu laden oder eine Seite über die interne Navigation zu verlassen. Diese Aspekte werden im folgenden Abschnitt behandelt.

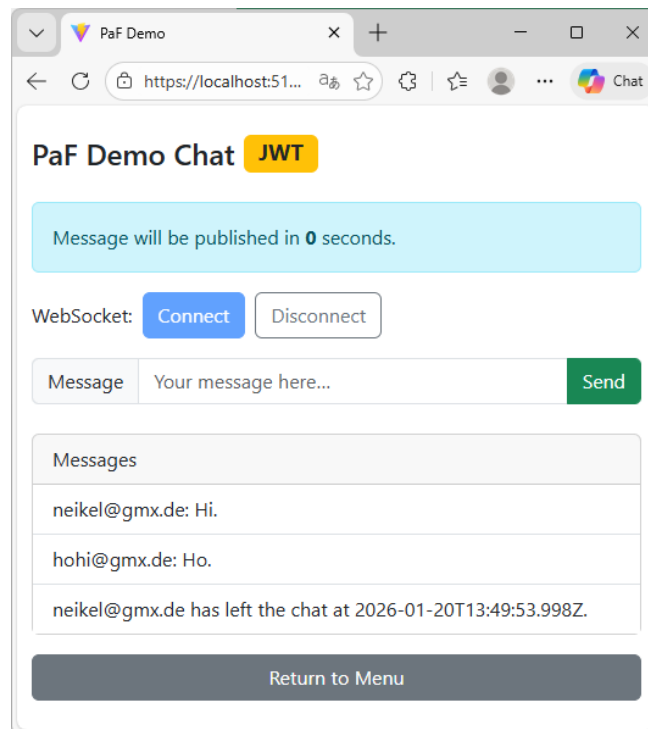
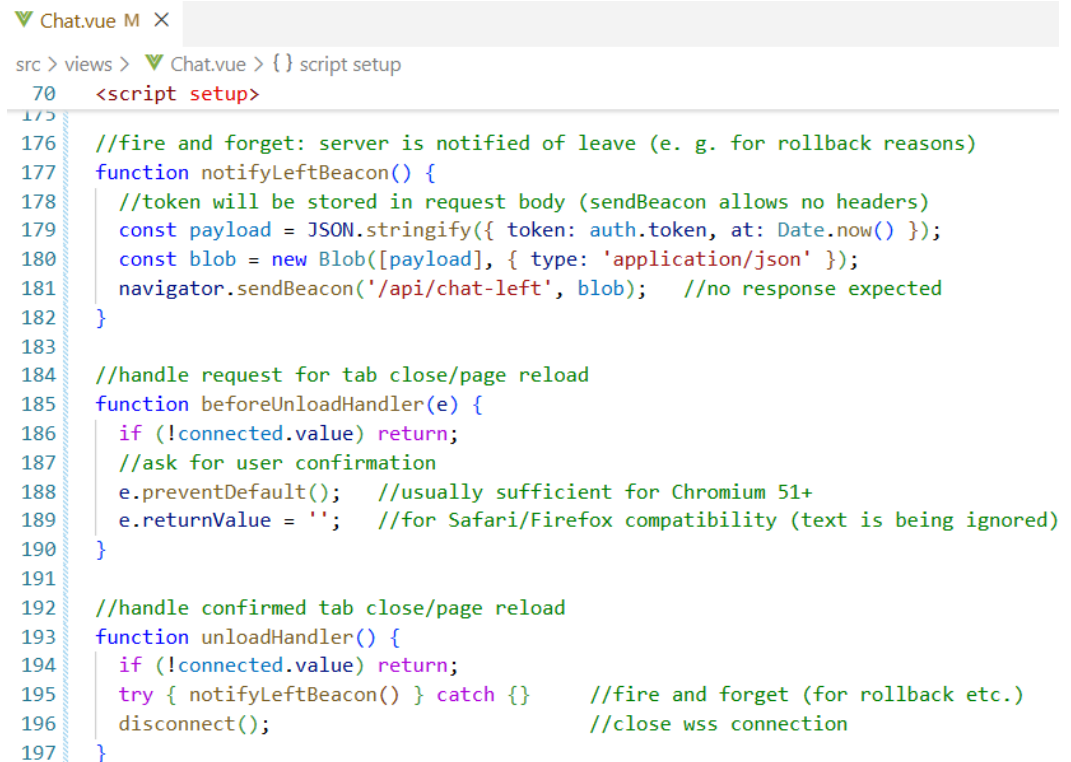


Abb. 7.21: Benachrichtigung, dass ein anderer User den Chat verlassen hat

Reaktion mittels `beforeUnload`, `unload` und `onBeforeRouteLeave()`

Versucht der User, einen Browser-Tab zu schließen oder neu zu laden, wird clientseitig zunächst das Ereignis *beforeunload* ausgelöst. Dies kann dazu genutzt werden, um dem User einen browserseitigen Abfragedialog anzuzeigen, in dem er die Aktion bestätigen oder verwerfen muss. Der in der folgenden Abbildung gezeigte *beforeUnloadHandler* ist für genau diesen Abfragedialog zuständig. Für maximale Browser-Kompatibilität ist die Anweisungssequenz `e.preventDefault(); e.returnValue = ''`; üblich. Der Text des Abfragedialogs lässt sich entwicklerseitig leider nicht beeinflussen.

Bestätigt der User das Verlassen der Seite, wird das Ereignis *unload* ausgelöst, welches wiederum einen *unloadHandler* aufruft. Dieser kann für Aktionen genutzt werden, die nun erforderlich sind, z. B. "Aufräumarbeiten" aller Art. In unserem Beispiel ruft der *unloadHandler* eine Funktion *notifyLeftBeacon()* auf, deren Quellcode ebenfalls in der folgenden Abbildung gezeigt ist.



```

src > views > Chat.vue > {} script setup
70 <script setup>
175
176 //fire and forget: server is notified of leave (e. g. for rollback reasons)
177 function notifyLeftBeacon() {
178   //token will be stored in request body (sendBeacon allows no headers)
179   const payload = JSON.stringify({ token: auth.token, at: Date.now() });
180   const blob = new Blob([payload], { type: 'application/json' });
181   navigator.sendBeacon('/api/chat-left', blob); //no response expected
182 }
183
184 //handle request for tab close/page reload
185 function beforeUnloadHandler(e) {
186   if (!connected.value) return;
187   //ask for user confirmation
188   e.preventDefault(); //usually sufficient for Chromium 51+
189   e.returnValue = ''; //for Safari/Firefox compatibility (text is being ignored)
190 }
191
192 //handle confirmed tab close/page reload
193 function unloadHandler() {
194   if (!connected.value) return;
195   try { notifyLeftBeacon() } catch {} //fire and forget (for rollback etc.)
196   disconnect(); //close wss connection
197 }

```

Abb. 7.22: Abbruchbehandlung mit sendBeacon in Chat.vue, Teil 1

Fire & Forget mit sendBeacon

`notifyLeftBeacon()` baut und versendet einen POST-Request an den Server, allerdings auf eine andere Art als wir es bisher kennengelernt haben, nämlich mittels `navigator.sendBeacon()`. Der Grund hierfür besteht darin, dass asynchrone Aufrufe, wie sie z. B. mit `fetch()` getätigt werden, beim Schließen einer View oft nicht zuverlässig funktionieren. Per `sendBeacon` hingegen kann der Request (meist) zuverlässig gesendet werden, es wird allerdings keine Response abgewartet (reines Fire & Forget). Da `sendBeacon` keine Header zulässt, können wir das JWT nicht wie üblich im Authorization Header mitschicken. Als Notlösung tragen wir es in die Request Payload ein. Diese wird anschließend in ein Blob (binary large object) verpackt und gemeinsam mit der relativen URL des REST-Endpunkts an `navigator.sendBeacon()` übergeben. Diese ungewöhnliche Handhabung hat Auswirkungen auf den Controller auf Serverseite, die weiter unten erläutert werden.

Die oben definierten Funktionen `beforeUnloadHandler()` und `unloadHandler()` müssen wir nun noch mit den jeweiligen Ereignissen verknüpfen. Dies erledigen wir über den Lifecycle Hook `onMounted()` wie in der folgenden Abbildung gezeigt. Damit ist sichergestellt, dass die Event Handler aktiv sind, wenn die View fertig geladen ist. Sobald die View wieder geschlossen wird, werden die Event Handler wieder entfernt, siehe Lifecycle Hook `onBeforeUnmount()`.

```

▼ Chat.vue M X
src > views > ▼ Chat.vue > ...
70  <script setup>
198
199  onMounted(() => {
200    window.addEventListener('beforeunload', beforeUnloadHandler) //triggers confirmation dialog
201    window.addEventListener('unload', unloadHandler)           /// might not work in Firefox ##
202    //window.addEventListener('pagehide', unloadHandler)         /// might not work in Firefox ##
203  })
204
205  onBeforeUnmount(() => {
206    window.removeEventListener('beforeunload', beforeUnloadHandler)
207    window.removeEventListener('unload', unloadHandler)
208    //window.removeEventListener('pagehide', unloadHandler)
209  })
210
211  //handle internal navigation ("back to menu")
212  onBeforeRouteLeave((to, from, next) => {
213    if (!connected.value) return next();
214    const ok = window.confirm('Do you really want to leave the chat?');
215    if (!ok) return next(false) //stay in current view
216
217    //navigation confirmed
218    try { notifyLeftBeacon() } catch {} //fire and forget (for rollback etc.)
219    try { disconnect() } catch {}
220    next()
221  });
222  </script>

```

Abb. 7.23: Abbruchbehandlung mit sendBeacon in Chat.vue, Teil 2

Schließlich müssen wir die verwendeten Lifecycle Hooks noch in die View importieren:



```

import { ref, onMounted, onBeforeUnmount, computed } from 'vue'
import { onBeforeRouteLeave } from 'vue-router'

```



Von der Verwendung des *unload*-Ereignisses wird auf einigen Webseiten abgeraten, z. B. auf [MDN](#). Tatsächlich wird das Ereignis im Firefox nicht zuverlässig gefeuert, so dass der sendBeacon-Request beim Server nicht ankommt. Es wird dazu geraten, anstatt auf *unload* besser auf das *pagehide*-Ereignis zu reagieren.

Nach Erfahrungen des Autors funktioniert dies im Firefox allerdings ebenfalls nicht. In Chromium-basierten Browsern wie Chrome und Edge wird der Event Handler für *unload* hingegen zuverlässig ausgeführt, so dass der Server den sendBeacon-Request empfängt.

Verlassen der View über SPA-interne Navigation

Wird die Chat-View nicht über Tab Close oder Tab Reload verlassen, sondern über den "Return to Menu"-Button, also die interne Navigation, erfolgt in der SPA definitionsgemäß kein *unload*. Daher muss dieser Fall gesondert behandelt werden. Wir tun dies über die Funktion *onBeforeRouteLeave()*, die beim Wechsel zu einer anderen Route der

SPA aufgerufen wird. Sie präsentiert dem User zunächst eine Abfrage, ob die aktuelle View wirklich verlassen werden soll (diesmal mit individuell spezifizierbarem Text). Wenn dies userseitig bestätigt wird, erfolgt der Aufruf von `notifyLeftBeacon()` (siehe oben) sowie das Schließen der WebSocket-Verbindung.

REST-Endpunkt für `sendBeacon` im `PafdemoApiController`

Für clientseitige Fire-&-Forget-Aufrufe mittels `sendBeacon` benötigt die Serverseite einen dedizierten POST-Endpunkt, den wir `/chat-left` nennen. Diesen bauen wir in den `PafdemoApiController` ein. Wie weiter oben erläutert, kann das JWT bei `sendBeacon` nicht im Request Header platziert werden. Alternativ hatten wir es im Request Body untergebracht. Folglich können wir für die Prüfung des JWT auf Gültigkeit nicht wie üblich den `JwtAuthFilter` verwenden (siehe [Serverseitig JWT implementieren](#)), da dieser einen Authorization Header erwartet. Wie in der folgenden Abbildung zu sehen ist, findet die Gültigkeitsprüfung bei `/chat-left` direkt im Controller statt. Die bei erfolgreicher Prüfung ausgegebene Log-Meldung ist als Platzhalter zu verstehen. An dieser Stelle hätte der Controller die Möglichkeit, "Aufräumarbeiten" aller Art durchzuführen. Im Kontext Ihres Spieleprojekts könnte dies z. B. eine Rollback-Operation sein, die ein abgebrochenes Spiel aus der Datenbank entfernt.

```

1 package paf_grp_x.pafdemo.controller;
2
3 //...
4 import org.slf4j.Logger;
5 import org.slf4j.LoggerFactory;
6 import org.springframework.http.HttpStatus;
7 import org.springframework.http.ResponseEntity;
8 import org.springframework.web.bind.annotation.*;
9
10 import paf_grp_x.pafdemo.image.ImageStorage;
11 import paf_grp_x.pafdemo.security.JwtTokenService;
12
13 @RestController
14 @RequestMapping("/api")
15 public class PafdemoApiController {
16
17     private final ImageStorage storage;
18     private final JwtTokenService jwt;
19
20     private static final Logger log = LoggerFactory.getLogger(PafdemoApiController.class);
21
22     public PafdemoApiController(ImageStorage storage, JwtTokenService jwt) {
23         this.storage = storage;
24         this.jwt = jwt;
25     }
26
27     public record LeaveRequest(String token, Long at) {}
28
29     //sendBeacon endpoint: user has left chat
30     @PostMapping("/chat-left")
31     public ResponseEntity<Void> left(@RequestBody(required = false) LeaveRequest body) {
32         String token = null;
33         if (body != null) token = body.token();
34
35         if (token == null || !jwt.isValid(token)) {
36             return ResponseEntity.status(HttpStatus.UNAUTHORIZED).build();
37         }
38
39         log.info(jwt.getUsername(token) + " left the chat, sendBeacon request received.");
40         //do some rollback work here
41
42         return ResponseEntity.accepted().build(); // 202
43     }
44 }

```

Abb. 7.24: REST-Endpunkt für sendBeacon im PafdemoApiController

Wie oben beschrieben kann die Gültigkeit des JWT bei Verwendung von sendBeacon erst im Controller geprüft werden. Der zugehörige REST-Endpunkt /chat-left darf also nicht zugriffsgeschützt sein. Wir ergänzen daher in der *apiChain* unserer *WebSecurity-Config* die Zeile *.requestMatchers("/api/chat-left").permitAll()*.



```

@Bean @Order(1)
public SecurityFilterChain apiChain(
    HttpSecurity http, JwtAuthFilter jwtFilter) throws Exception
{
    http
        .authorizeHttpRequests(auth -> auth
            //...
            .requestMatchers("/api/chat-left").permitAll() //sendBeacon (jwt in
body)
            //...

```

Damit haben wir die Chat-Funktion unserer Demo-Anwendung (SPA-Variante) robust gegen userseitige Abbrüche gemacht. Leider funktioniert dies - wie beschrieben - noch nicht in allen Browsern gleich gut. Hier gilt es also neue Entwicklungen seitens der Browser-Hersteller abzuwarten und zu gegebener Zeit darauf zu reagieren.

7.6 Typische Fallstricke im Projektverlauf

Hier listen wir - ohne jeglichen Anspruch auf Vollständigkeit - einige spezielle Aspekte auf, die bei Nichtbeachtung zu einer aufwändigen Fehlersuche führen könnten und somit erhebliches Frustpotential haben.

Vermeidung von Umlauten in `application.properties`

Wir raten dringend davon ab, in `application.properties` Umlaute zu verwenden, auch in Kommentarzeilen. Diese könnten zu Problemen mit der Zeichencodierung führen, was wiederum merkwürdige Fehlermeldungen an anderen Stellen nach sich ziehen kann, z. B. bei der Datenbank-Konfiguration und in `pom.xml`. In der Folge startet ggf. der Server nicht mehr. In diesem Zusammenhang mag es sinnvoll sein, in `pom.xml` explizit eine Codierung nach UTF-8 festzuschreiben:



Code

```
<properties>
  <java.version>25</java.version>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  <project.reporting.outputEncoding>UTF-8</project.reporting.outputEncoding>
</properties>
```

Datenaustausch über Objekte

Wenn Daten zwischen Client und Spring-Server ausgetauscht werden, ist es wichtig, diese als Objekte zu modellieren. Nur für Objekte funktioniert eine in Spring eingebaute automatische Wandlung von JSON in Objekte und umgekehrt. Selbst wenn nur eine einzelne Zeichenkette übertragen werden soll, sollte dies nicht als String-Wert erfolgen, sondern als Objekt mit einem Attribut vom Typ String. Beispiel:



Code

```
@PostMapping("/move")
@SendTo("/topic/position")
public Position handleMove(MoveDto move) {
    String direction = move.getDirection();
    switch (direction) {
        case "left": posX -=5; break;
        //...
        case "up": posY -=5; break;
        //...
    }
}
```

```
    return Position(posX, posY);  
}
```

Hier hat die Klasse MoveDto ggf. nur ein einzelnes Attribut direction vom Typ String. Allein die Verwendung von "MoveDto move" anstelle von "String direction" als Parameter von handleMove() stellt schon die automatische Wandlung von/nach JSON sicher. Dies gilt in analoger Weise auch für den "Rückweg", hier also die Kapselung von posX und posY in ein Objekt der Klasse Position (siehe auch [diesen inhaltlich verwandten Abschnitt](#)).

Clientseitig muss entsprechenderweise mit JSON.parse() bzw. JSON.stringify() gearbeitet werden, um empfangene Daten zu "entpacken" bzw. zu sendende Daten zu "verpacken".

8 Mit Threads arbeiten

8 Mit Threads arbeiten

8.1 Motivation und Grundlagen von Threads

8.2 Zeitgesteuerte Ausführung von Threads

8.3 Thread Pools

8.4 Synchronisierung von Threads

8.5 Deadlocks und Starvation

§§§ BAUSTELLE - VOM BETRETEN WIRD ABGERATEN ;-)

8.1 Motivation und Grundlagen von Threads

1. Motivation

- MiceHunting: viele Threads (2 pro "Maus") in einer Anwendung, GUI bleibt benutzbar
- ConcurrentGuiService: Fortschrittsbalken in separatem Thread, GUI bleibt benutzbar

Grundlagen:

2. Starten eines Threads, Ausführungsreihenfolge

- MyThread: erweitert Klasse Thread, Thread erzeugen run()-Methode, main-Thread
- MyRunnable; implementiert (funktionales) Interface Runnable, logische Trennung zwischen Thread als Ausführungsumgebung und Aufgabe/Task (hier: MyRunnable), die der Thread bearbeiten soll
- MyLambdaRunnable: elegante (Kurz-)Schreibweise

3. Überlappende Ausführung von Threads

- MyLoopThread (besser Plural): Berechnung überlappend
- MyLoopSleepThread (besser Plural): naiver Versuch einer Synchronisation auf Kosten der Effizienz

8.2 Zeitgesteuerte Ausführung von Threads

4. Abbrechen, zeitgesteuertes Ausführen von Threads

- MyCancelledThread (besser: MyCancelableThread): Abbrechen einer Aktivität nach einer bestimmten Zeit mit Hilfe von (eigener) Variable keepRunning und (eigener) Methode cancel().
- MyInterruptedThread (besser: MyInterruptibleThread): Abbrechen einer Aktivität mit "Bordmitteln", nämlich den Methoden isInterrupted() und interrupt() der Klasse Thread; Vorteil: interrupt() unterbricht auch sleep()-Phasen

- MyPriorityThread (besser: MyPrioritizedThreads): je höher priority-Wert, desto mehr Prozessorzeit, Anomalien aufgrund kurzer Laufzeit
- MyTimerTask: Timer-Objekt bekommt Schedule zur Ausführung -> TimerTask (implementiert Runnable) wird bis zum Abbruch periodisch ausgeführt

8.3 Thread Pools

5. Thread-Pools zur Wiederverwendung und zum Schutz vor Ressourcenüberlastung
- [- MyScheduledExecutorService: sehr ähnlich zu MyTimerTask, nur dass der hier verwendete ScheduledThreadPoolExecutor auch mehrere Threads verwalten kann.]
 - Backsimulation halloweenbakery.ovenpool (Klassen OvenPoolDemo, BakingTask, Bakery) -> ExecutorService verteilt Tasks auf die einzelnen Threads im Pool

8.4 Synchronisierung von Threads

6. Synchronisierung von Threads / kritische Abschnitte

- Motivation: synchronization.LostUpdate -> 100 Threads sollen eine gemeinsame Variable je 100x inkrementieren -> tatsächliches Ergebnis ist oft echt kleiner als das erwartete Ergebnis 10.000.
- Lösung: Semaphore/Wächter für kritische Abschnitte -> Schlüsselwort synchronized für Methode oder einen Block innerhalb der Methode: synchronized(this) { ... } -> am LostUpdate-Beispiel zeigen
- Erzeuger/Verbraucher-Problem am Beispiel der Halloween (Geburtstag??) Simulation
- halloweenbakery.cookiebowl
- CookieBowlDemo: gemeinsame Ressource cookiebowl
- Adult + Kid (erben von Thread) - CookieBowl: Interface mit store() und take() - CookieBowlWithSynchronized: synchronized, wait(), notify() -> benachrichtigt *irgendeinen* wartenden Thread, "unfair" - CookieBowlWithLock: alternativ mit Locks/Conditions statt synchronized, Vorteil: man kann feingranularer informiert werden/reagieren, z. B. store(): notFull.await() -> add -> notEmpty.signal() , take() -> notEmpty.await() -> poll -> notFull.signal() - CookieBowlWithBlockingQueue: spezielle Datenstruktur, die Synchronisierung intern regelt

8.5 Deadlocks und Starvation

7. Deadlocks, Starvation, Dining Philosophers Problem

- halloweenbakery.halloween.live: 5 Kinder am Tisch, dazwischen Handschuhe, Kind braucht 2, um in Keksschale langen zu dürfen
- HalloweenDemo, Tray, Kid, TraySingleGloveTake (die letzten beiden weiter ausprogrammieren)
- > Deadlock provozieren -> besser: beide Handschuhe gleichzeitig nehmen - TrayDoubleGloveTake: kein Deadlock mehr, aber Starvation möglich -> Fairness Policy erforderlich - TrayDoubleGloveTakeFairLock: arbeitet nicht mit synchronized, sondern mit Locks, insb. mit ReentrantLock(true), d. h. fair = true, d. h. intern genutzte Fairness Policy, d. h. alle kommen mal dran (ohne entwicklerseitige Steuerung)

I Abbildungsverzeichnis



Abb. 2.1: Ein initiales SpringBoot-Projekt spezifizieren..... 10

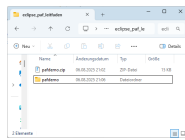


Abb. 2.2: Entpackte zip-Datei aus dem Spring Initializr..... 12

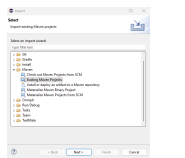


Abb. 2.3: Ein existierendes Maven-Projekt in Eclipse importieren, Teil 1..... 12

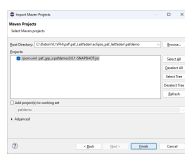


Abb. 2.4: Ein existierendes Maven-Projekt in Eclipse importieren, Teil 2..... 13

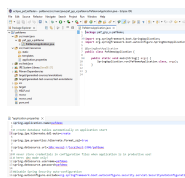


Abb. 2.5: Initiales Maven-Projekt nach dem Import..... 13

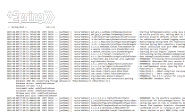


Abb. 2.6: Datenbankkonfiguration in application.properties..... 14

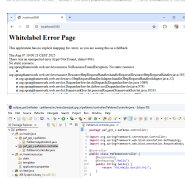


Abb. 2.7: Ausgaben in der Serverkonsole nach Start der Anwendung..... 15

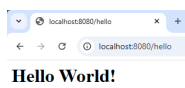


Abb. 2.8: Whitelabel Error Page..... 16

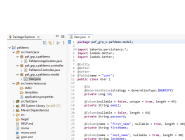


Abb. 2.9: Klasse PafdemoController..... 16



Abb. 2.10: Hello-World-Seite im Browser..... 17

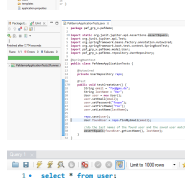


Abb. 3.1: Einfache User-Klasse mit JPA- und Lombok-Annotationen..... 21

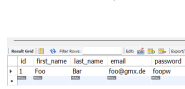


Abb. 3.2: Interface UserRepository..... 23



Abb. 3.3: Testklasse PafdemoApplicationTests..... 24



Abb. 3.4: Datenbanktabelle user..... 24

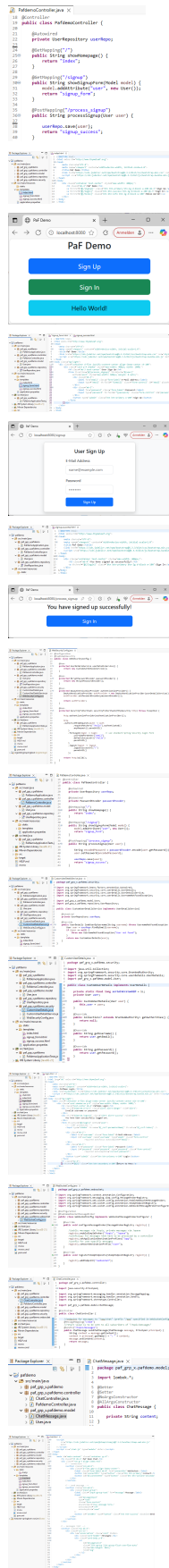


Abb. 3.5: Controller, erweitert um Endpunkte zur Registrierung.....	25
Abb. 3.6: index.html mit einfachem Menü.....	27
Abb. 3.7: Startseite im Browser.....	27
Abb. 3.8: Registrierungsformular in HTML.....	28
Abb. 3.9: Registrierungsformular im Browser.....	29
Abb. 3.10: Registrierungsbestätigung in HTML.....	29
Abb. 3.11: Registrierungsbestätigung im Browser.....	30
Abb. 3.12: Klasse WebSecurityConfig.....	31
Abb. 3.13: Controller mit injiziertem PasswordEncoder.....	32
Abb. 3.14: Klasse CustomUserDetailsService.....	33
Abb. 3.15: Klasse CustomUserDetails.....	34
Abb. 3.16: Login-Dialog in HTML.....	36
Abb. 4.1: Klasse WebSocketConfig.....	38
Abb. 4.2: Klasse ChatController.....	40
Abb. 4.3: Klasse ChatMessage.....	42
Abb. 4.4: Thymeleaf-Template chat.html.....	44



Abb. 4.5: Modul chat.js, Teil 1..... 46

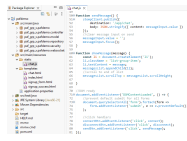


Abb. 4.6: Modul chat.js, Teil 2..... 46

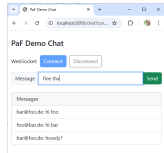


Abb. 4.7: Chat im Browser..... 48

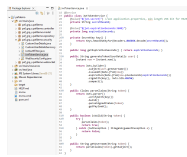


Abb. 5.1: Klasse JwtTokenService..... 50



Abb. 5.2: Klasse JwtAuthController..... 52

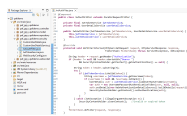


Abb. 5.3: Klasse JwtAuthFilter..... 52



Abb. 5.4: Klasse WebSecurityConfig mit zweiter Filterkette..... 54



Abb. 5.5: Klasse PafdemoApiController..... 54



Abb. 5.6: Klasse JwtStompChannelInterceptor..... 55

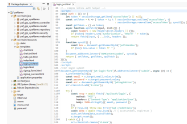


Abb. 5.7: login_jwt.html (Auszug)..... 56

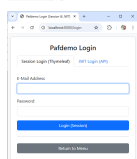


Abb. 5.8: Login-Formular mit JWT-Unterstützung..... 57

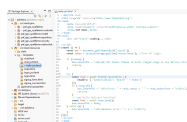


Abb. 5.9: hello-jwt.html..... 58



Abb. 5.10: index.html mit JWT-bezogenen Menüpunkten..... 60



Abb. 5.11: Startseite mit Logout- und JWT-Support..... 61



Abb. 5.12: Klasse JwtBlacklistService.....	62
Abb. 5.13: JwtAuthController, erweiterte Dependency Injection.....	63
Abb. 5.14: JwtAuthController, logout-Methode.....	63
Abb. 5.15: JwtAuthController, refresh-Methode.....	64
Abb. 6.1: vite.config.js.....	68
Abb. 6.2: Neuer Endpunkt im JwtAuthController.....	68
Abb. 6.3: Helfer-Modul auth.js.....	70
Abb. 6.4: Einstiegspunkt main.js.....	71
Abb. 6.5: SPA-Startseite index.html.....	71
Abb. 6.6: router.js.....	72
Abb. 6.7: Sicht SignUp.vue, template-Teil.....	73
Abb. 6.8: Sicht SignUp.vue, script-Teil.....	74
Abb. 6.9: Sicht Hello.vue.....	75
Abb. 6.10: Sicht Menu.vue.....	76
Abb. 6.11: Menü der SPA.....	77

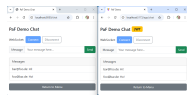
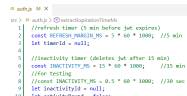
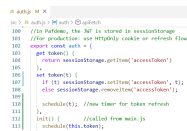
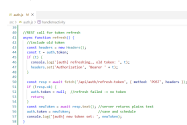
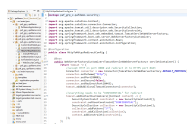
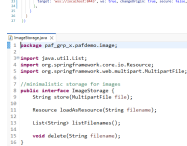
**Abb. 6.12:** Chat-Funktion der SPA..... 78**Abb. 6.13:** Konstanten und Variablen für Timer-Funktionalität im Helfer-Modul auth.js..... 79**Abb. 6.14:** Scheduling des JWT Refresh im Helfer-Modul auth.js..... 79**Abb. 6.15:** Aufruf des Scheduling im Helfer-Modul auth.js.....80**Abb. 6.16:** REST Call für JWT Refresh im Helfer-Modul auth.js..... 81**Abb. 6.17:** Inaktivitätsbehandlung im Helfer-Modul auth.js.....82**Abb. 6.18:** Event Listener Binding im Helfer-Modul auth.js.....82**Abb. 7.1:** Ablauf eines CSRF-Angriffs am Beispiel eines Web-Formulars..... 84**Abb. 7.2:** Seitenquelltext eines Formulars mit CSRF-Token..... 85**Abb. 7.3:** Konfigurationsklasse zum Weiterleiten von HTTP zu HTTPS.....89**Abb. 7.4:** vite.config.js, erweitert um https/wss-Spezifikationen..... 90**Abb. 7.5:** Interface ImageStorage..... 92**Abb. 7.6:** Klasse FileSystemImageStorage, Teil 1..... 92**Abb. 7.7:** Klasse FileSystemImageStorage, Teil 2..... 93



Abb. 7.8: Klasse FileSystemImageStorage, Teil 3..... 94

Abb. 7.9: Klasse WebConfig..... 95

Abb. 7.10: /images-Endpunkte in der Klasse PdfdemoApiController..... 96

Abb. 7.11: Template hello-jwt.html mit Image-Upload und -Anzeige, Teil 1..... 97

Abb. 7.12: Template hello-jwt.html mit Image-Upload und -Anzeige, Teil 2..... 97

Abb. 7.13: Template hello-jwt.html mit Image-Upload und -Anzeige, Teil 3..... 98

Abb. 7.14: Template hello-jwt.html mit Image-Upload und -Anzeige, Teil 4..... 99

Abb. 7.15: View Hello.vue mit Image-Upload und -Anzeige, Teil 1..... 100

Abb. 7.16: View Hello.vue mit Image-Upload und -Anzeige, Teil 2..... 101

Abb. 7.17: CountdownService, Teil 1..... 103

Abb. 7.18: CountdownService, Teil 2..... 104

Abb. 7.19: Chat-Funktion mit Countdown in der SPA..... 107

